RÉPUBLIQUE DE CÔTE D'IVOIRE

MINISTÈRE DE L'ÉDUCATION NATIONALE, DE L'ALPHABÉTISATION ET DE L'ENSEIGNEMENT TECHNIQUE

École Nouvelle Supérieure d'Ingénieurs et de Technologie (ENSIT)

Entreprise d'accueil : SYNELIA

Mémoire de fin d'études

En vue de l'obtention du diplôme d'Ingénieur en Sciences Informatiques

Option : Génie Logiciel

Thème

Étude, conception et mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée : cas de l'entreprise SYNELIA

Présenté par	Encadrement
KOUASSI EMISSAH MARIE	Pédagogique : M. ASSOHOUN
	Entreprise : M. KOUADIO INNOCENT
	Manager Backend chez SYNELIA

Année académique : 2025 - 2026

RÉPUBLIQUE DE CÔTE D'IVOIRE

MINISTÈRE DE L'ÉDUCATION NATIONALE, DE L'ALPHABÉTISATION ET DE L'ENSEIGNEMENT TECHNIQUE

École Nouvelle Supérieure d'Ingénieurs et de Technologie (ENSIT)

Entreprise d'accueil : SYNELIA

Mémoire de fin d'études

En vue de l'obtention du diplôme d'Ingénieur en Sciences Informatiques

Option : Génie Logiciel

Thème

Étude, conception et mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée : cas de l'entreprise SYNELIA

Présenté par	Encadrement
KOUASSI EMISSAH MARIE	Pédagogique : M. ASSOHOUN
	Entreprise : M. KOUADIO INNOCENT
	Manager Backend chez SYNELIA

Année académique : 2025 - 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadreur pédagogique, **M. ASSOHOUN**, pour son accompagnement, sa disponibilité et la qualité de ses conseils tout au long de ce travail.

Mes sincères remerciements s'adressent également à mon maître de stage, **M. KOUADIO INNOCENT**, pour sa confiance, son encadrement au sein de SYNELIA et la richesse des échanges techniques et méthodologiques qui ont nourri ce projet.

Je remercie l'ensemble de l'équipe SYNELIA pour son accueil, sa bienveillance et son implication dans le suivi du projet SyneliaQuizApp.

Mes remerciements vont aussi au corps enseignant et administratif de l'ENSIT, ainsi qu'aux membres du jury qui consacreront du temps à l'évaluation de ce mémoire.

Enfin, j'adresse une pensée reconnaissante à ma famille et à mes proches pour leur soutien constant durant mon parcours académique.

Résumé

Ce mémoire présente l'étude, la conception et la mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée, réalisée pour le compte de l'entreprise SYNELIA, dans le cadre du projet de fin d'études du cycle d'ingénieur en sciences informatiques.

Le recrutement constitue un levier stratégique pour les entreprises de services numériques. La capacité à attirer, évaluer et intégrer rapidement les talents conditionne directement la qualité des prestations et la compétitivité. Pourtant, dans de nombreuses structures, le processus de recrutement reste fragmenté entre emails, fichiers de suivi, formulaires externes et échanges informels, ce qui génère des pertes d'information, un manque de standardisation des évaluations et une traçabilité insuffisante des décisions.

Pour répondre à cette problématique, le projet SyneliaQuizApp a été conçu comme une plateforme centralisée couvrant l'ensemble du cycle de recrutement : création et configuration des campagnes, soumission et suivi des candidatures, évaluation technique par quiz automatisé, planification des entretiens, évaluations collaboratives, prise de décision et notifications. L'application distingue quatre profils d'utilisateurs — super administrateur, administrateur, recruteur et candidat — avec des permissions fines et des parcours adaptés à chaque rôle.

La démarche méthodologique a suivi une progression en quatre temps. Une phase d'analyse a permis de formaliser les besoins métier, les règles de gestion et les flux du recrutement à partir du cahier des charges et de l'observation des pratiques existantes. Une phase de conception a structuré l'architecture en couches, modélisé la base de données relationnelle et défini les contrats d'API REST entre le backend et le frontend. La réalisation a été menée de manière incrémentale en s'appuyant sur Spring Boot pour la logique serveur, Angular pour l'interface utilisateur et MySQL pour la persistance, avec des migrations versionnées via Flyway. Enfin, une phase de validation a permis de confronter la solution aux exigences fonctionnelles, de corriger les écarts et de mesurer les résultats obtenus.

Les principaux résultats du projet incluent une couverture fonctionnelle élevée des modules cœur du recrutement — campagnes, formulaires, quiz, entretiens, évaluations, décisions — une architecture technique robuste et maintenable, une gestion granulaire des permissions, une interface administrateur complète pour le paramétrage transverse, ainsi qu'un parcours candidat public sécurisé avec suivi en temps réel. Le projet a égale-

ment mis en évidence des limites liées au niveau d'industrialisation des tests, à la maturité de certaines zones transverses et à l'absence de mise à l'échelle en environnement de production.

En conclusion, SyneliaQuizApp démontre qu'une démarche d'ingénierie logicielle structurée peut transformer un processus métier sensible en un système cohérent, évolutif et porteur de valeur pour l'entreprise. Les perspectives ouvertes par ce travail incluent le renforcement de la chaîne CI/CD, l'enrichissement des indicateurs analytiques, l'amélioration de l'expérience candidat et l'extension fonctionnelle vers de nouveaux modules.

Sommaire

Introduction 1
1. Présentation générale du projet 5
2. Étude de l'existant et état de l'art 13
3. Analyse des besoins 23
4. Conception de la solution 38
5. Environnement technique et outils 75
6. Implémentation de la solution 80
7. Tests et validation 91
8. Présentation des résultats 97
9. Limites et perspectives 102
Conclusion 104
Références bibliographiques 106
Annexes 109

Avant-propos

Le présent mémoire s'inscrit dans le cadre académique de la formation d'ingénieur en sciences informatiques de l'École Nouvelle Supérieure d'Ingénieurs et de Technologie (ENSIT), établissement d'enseignement supérieur situé à Abidjan, en Côte d'Ivoire. L'ENSIT forme des ingénieurs capables de concevoir, développer et piloter des solutions informatiques adaptées aux enjeux technologiques actuels, en s'appuyant sur un socle scientifique solide et une pédagogie orientée vers la pratique professionnelle.

Le parcours complet à l'ENSIT s'étend sur cinq années et se décompose en deux cycles complémentaires. Les deux premières années constituent la classe préparatoire intégrée, durant laquelle les élèves acquièrent l'ensemble des bases scientifiques nécessaires à la poursuite d'études d'ingénieur : mathématiques (analyse, algèbre, probabilités), physique (électronique, électromagnétisme, optique), initiation à l'algorithmique et à la programmation (langage C, micro-informatique), ainsi que des enseignements transverses (économie, gestion, techniques d'expression et anglais). Cette prépa intégrée met l'accent sur l'acquisition de connaissances solides et la découverte de l'entreprise, à travers des projets et des conférences professionnelles.

Les trois années suivantes forment le cycle ingénieur proprement dit. La première année, dite de tronc commun, consolide les bases scientifiques et techniques — algorithmique avancée, programmation orientée objet, bases de données, systèmes d'exploitation, réseaux — tout en initiant les élèves aux premiers projets concrets. La deuxième année ouvre la voie aux spécialisations avancées : modélisation orientée objet (UML, Merise), développement web et mobile (J2EE, applications web), intelligence artificielle et machine learning, administration des systèmes et des réseaux, sécurité informatique. La troisième année, enfin, approfondit les compétences métier (gestion de projets, architectures cloud, cryptographie, méthodologie de recherche et de rédaction) et se conclut par un projet de fin d'études réalisé en entreprise, dont ce mémoire constitue l'aboutissement.

C'est dans ce cadre que j'ai eu l'opportunité d'effectuer mon stage de fin d'études au sein de SYNELIA, entreprise de services numériques africaine basée à Abidjan. Née en 2018 de la transformation de Smile Group Afrique — filiale franchisée d'un groupe européen leader de l'Open Source implanté en Côte d'Ivoire depuis 2012 — SYNELIA incarne une ambition : bâtir une industrie technologique africaine compétitive, ancrée localement et tournée vers l'international. La contraction de « Synergie » et « Nouvelle » reflète cette volonté de tracer une voie inédite pour l'écosystème numérique du continent.

Aujourd'hui, SYNELIA revendique plus de cent dix profils technologiques, plus de cent projets livrés avec succès et environ quatre-vingt-dix clients actifs répartis en Afrique de l'Ouest. Le groupe structure son offre autour de six expertises majeures — Hosting et Cloud, API Management, Big Data et Machine Learning, Cybersécurité, Intelligence Artificielle et Automatisation de processus — complétées par des unités d'activité dédiées telles que l'Open Digital Academy (formation et insertion des talents tech africains) et la distribution de services Mobile Money.

Le projet SyneliaQuizApp, qui fait l'objet de ce mémoire, s'inscrit dans l'unité « Contrôle qualité des recrutements » du groupe. Il répond à un besoin concret identifié par la direction : structurer le processus de recrutement à travers une plateforme interne unifiée, capable de centraliser les campagnes, d'automatiser l'évaluation technique des candidats et de garantir la traçabilité des décisions. Ce projet m'a permis de mobiliser l'ensemble des compétences acquises au cours de ma formation — analyse des besoins, conception logicielle, développement full-stack, gestion de base de données, sécurité applicative — dans un contexte professionnel réel, exigeant et formateur.

Je tiens à exprimer ici ma gratitude envers l'ENSIT pour la qualité de la formation dispensée, et envers SYNELIA pour la confiance accordée et l'environnement de travail stimulant qui a rendu ce projet possible.

Glossaire

API	Application Programming Interface
Angular	Framework frontend utilisé pour l'interface web monopage
ATS	Applicant Tracking System
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
JPA	Java Persistence API
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données dans la méthode Merise
MCT	Modèle Conceptuel des Traitements dans la méthode Merise
MERISE	Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise
MLD	Modèle Logique de Données
MPD	Modèle Physique de Données
MySQL	Système de gestion de base de données relationnelle
REST	Style d'architecture des services web
RGPD	Règlement Général sur la Protection des Données
SAT	Structure d'Accès Théorique
SPA	Single Page Application
Spring Boot	Framework backend Java retenu pour le projet
Typst	Système de composition utilisé pour le mémoire
UML	Unified Modeling Language
WebSocket	Protocole de communication bidirectionnelle en temps réel

Introduction

À l'ère du digital, la gestion du recrutement connaît une transformation profonde. Les entreprises ne peuvent plus se contenter d'un traitement fragmenté des candidatures, de validations manuelles dispersées et d'outils hétérogènes pour piloter l'ensemble du parcours candidat.

Dans ce contexte, le projet **SyneliaQuizApp** a été conçu pour répondre à un besoin concret de l'entreprise SYNELIA : disposer d'une plateforme centralisée de gestion des campagnes de recrutement, de suivi des candidatures, d'évaluation technique et de coordination des intervenants internes. Cette démarche de centralisation s'inscrit dans une logique d'urbanisation des systèmes d'information, telle que définie par Christophe Longépé dans son ouvrage **Le projet d'urbanisation du S.I.** (2009) : remplacer des outils hétérogènes et fragmentés par un système unifié, aligné sur les processus métier de l'organisation.

Le recrutement occupe aujourd'hui une place stratégique dans la compétitivité des entreprises de services numériques. Dans un environnement où les profils techniques sont fortement sollicités, la capacité à identifier rapidement les bons candidats, à évaluer leurs compétences avec cohérence et à maintenir une communication fluide tout au long du processus devient un avantage organisationnel décisif. Or, dans de nombreuses structures, ces opérations restent encore dispersées entre plusieurs outils, plusieurs interlocuteurs et plusieurs supports de suivi difficilement consolidables.

C'est dans ce contexte que s'inscrit le présent mémoire, consacré à l'étude, à la conception et à la mise en œuvre d'une application web de gestion du processus de recrutement avec évaluation technique automatisée pour l'entreprise SYNELIA. Le sujet ne porte pas uniquement sur la réalisation d'un logiciel, mais sur la transformation d'un processus métier complet : centraliser les candidatures, structurer les campagnes de recrutement, intégrer un quiz technique, coordonner les évaluations, suivre les décisions et améliorer l'expérience vécue par les candidats comme par les recruteurs.

La problématique à laquelle répond ce travail peut être formulée ainsi : *comment concevoir une plateforme unifiée capable de digitaliser le recrutement, de standardiser l'évaluation technique et de renforcer la traçabilité des décisions, tout en restant adaptée aux contraintes opérationnelles d'une entreprise en croissance ?* Derrière cette question se trouvent plusieurs enjeux concrets : éviter la dispersion des informations, réduire

la charge manuelle, harmoniser les critères d'analyse, sécuriser les accès et rendre le pilotage du recrutement plus lisible.

Le choix de cette thématique se justifie à la fois par sa pertinence pour SYNELIA et par son intérêt du point de vue de l'ingénierie logicielle. Pour l'entreprise, il s'agit de répondre à un besoin réel de structuration des recrutements et de maîtrise des échanges internes et externes. Pour l'étudiant ingénieur, le projet constitue un terrain particulièrement riche, car il mobilise l'analyse des besoins, la modélisation métier, la conception d'une architecture web complète, l'intégration de mécanismes de sécurité, la gestion des données, ainsi que la validation d'un produit à forte densité fonctionnelle.

Plusieurs concepts clés traversent ce mémoire. La **campagne de recrutement** désigne le cadre organisationnel dans lequel sont publiés les besoins et suivis les candidats. Le **parcours candidat** correspond à la succession des interactions publiques, depuis la soumission jusqu'au suivi de l'avancement. Le **quiz technique** constitue un mécanisme de pré-évaluation automatisée destiné à objectiver une partie de l'analyse des compétences. Enfin, la **traçabilité** renvoie à la capacité du système à conserver les états, les décisions, les permissions et les événements utiles à la compréhension du processus.

La méthodologie adoptée dans ce travail repose sur une progression en quatre temps. Il a d'abord fallu analyser le besoin métier à partir du cahier des charges, des notes fonctionnelles et des flux réellement couverts par l'application. Une phase de conception a ensuite permis de structurer l'architecture, les responsabilités des modules, les règles de gestion et les principaux objets manipulés par le système. La réalisation a été menée de manière incrémentale sur un backend Spring Boot, un frontend Angular et une base MySQL, avec des validations régulières. Enfin, un travail de recul critique a permis d'évaluer les résultats obtenus, les limites rencontrées et les perspectives d'évolution du produit.

Le mémoire est organisé en quatre parties complémentaires. La première partie pose le cadre théorique et le contexte du projet, en présentant l'entreprise, la problématique et l'état de l'existant. La deuxième partie est consacrée à l'analyse des besoins et à la conception de la solution. La troisième partie décrit la réalisation effective de l'application, son environnement technique et la démarche de validation retenue. La quatrième partie dresse le bilan des résultats obtenus, puis ouvre sur les limites du travail réalisé et les perspectives d'amélioration envisageables.

Ainsi, au-delà de la seule production d'une application web, ce mémoire vise à montrer comment une démarche d'ingénierie logicielle peut contribuer à transformer

un processus métier sensible en un système structuré, évolutif et porteur de valeur pour l’entreprise.

Première partie

Cadre théorique et contexte du projet

Fondements du besoin, positionnement du sujet et justification de la solution

CHAPITRE 1

Présentation générale du projet

Ce premier chapitre a pour rôle de cadrer le projet dans son environnement métier, organisationnel et académique. Il introduit les besoins de SYNELIA, les objectifs poursuivis et le périmètre retenu pour la solution développée.

1.1 Contexte du projet

Le projet SyneliaQuizApp est né d'un besoin métier concret exprimé par SYNELIA dans un contexte de croissance et de diversification des recrutements. En tant qu'entreprise spécialisée dans les services informatiques et le conseil en technologies, SYNELIA doit identifier, évaluer et intégrer des profils variés allant de jeunes diplômés à des consultants ou experts plus expérimentés. Cette dynamique de recrutement rend rapidement visibles les limites d'un traitement fondé sur des supports dispersés, des échanges manuels et des pratiques peu standardisées.

Avant la mise en place d'une solution dédiée, le processus de recrutement reposait sur une combinaison d'emails, de fichiers de suivi, de formulaires externes, d'échanges informels et de validations éclatées entre plusieurs acteurs. Une telle organisation peut fonctionner à faible volume, mais elle devient fragile dès que le nombre de candidatures augmente, que plusieurs recruteurs doivent collaborer sur les mêmes dossiers ou que l'entreprise souhaite objectiver davantage ses décisions.

La mise en œuvre d'une plateforme interne de recrutement répond donc à un double objectif. D'une part, il s'agit d'optimiser le traitement opérationnel des candidatures en centralisant les données, les états et les interactions. D'autre part, il s'agit de doter l'entreprise d'un système capable de produire un processus plus lisible, plus équitable et plus traçable, tant pour les recruteurs que pour les candidats.

1.1.1 L'entreprise SYNELIA et ses besoins

Synelia est une entreprise africaine de services numériques (ESN) issue de la transformation, en 2018, de *Smile Group Afrique*. La maison-mère, Smile Côte d'Ivoire, avait été implantée à Abidjan dès 2012 comme filiale franchisée d'un groupe européen leader de l'Open Source. À la suite d'une dynamique de croissance et d'expansion régionale (Burkina Faso, Libéria, puis d'autres pays d'Afrique de l'Ouest), le groupe a pris la décision

stratégique de se détacher de la franchise européenne pour devenir une entreprise africaine indépendante. Le nom *Synelia* traduit cette ambition : la contraction de « Synergie » et de « Nouvelle » exprime la volonté de tracer une nouvelle voie pour l'industrie technologique africaine, en s'appuyant sur un ancrage local fort tout en visant désormais le marché occidental depuis l'Afrique.

Aujourd'hui, Synelia revendique plus de dix années de présence en Afrique de l'Ouest, un effectif de plus de cent dix profils technologiques locaux et internationaux, plus de cent projets accomplis avec succès et environ quatre-vingt-dix clients actifs. Le groupe se positionne comme *partenaire stratégique des entreprises qui souhaitent réussir leur transition vers un avenir numérique.* Son offre s'articule autour de six grandes expertises technologiques — Hosting and Cloud, API Management, Big Data et Machine Learning, Cybersécurité, Intelligence Artificielle et Automatisation de processus — déclinées en accompagnements sur mesure auprès d'organisations privées et publiques. Cette offre est complétée par des unités d'activité dédiées, parmi lesquelles l'**Open Digital Academy** (formation et insertion des talents tech africains), la distribution Mobile Money, plusieurs solutions sectorielles (par exemple un système de gestion des ressources animales) et une offre de contrôle qualité des recrutements dans laquelle s'inscrit directement le présent projet.

L'organisation interne du groupe est portée par une gouvernance à trois niveaux : la Direction Générale assurée par Patrick KOUASSI, fondateur du groupe, l'Administration Générale exercée par Jorge VILANOVA, et le Secrétariat Général confié à Stéphane OUATTARA (Secrétaire Général et COO), auquel est rattaché le PMO Gouvernance. Cinq directions opérationnelles complémentaires se déploient ensuite : Business & International, Production & Expérience Client, Innovation & Expertise, Performance Financière & Fiscalité, et Administration & Capital Humain. Cette gouvernance structurée favorise un pilotage transverse, dans lequel les expertises technologiques et les unités d'activité communiquent étroitement avec les fonctions support.

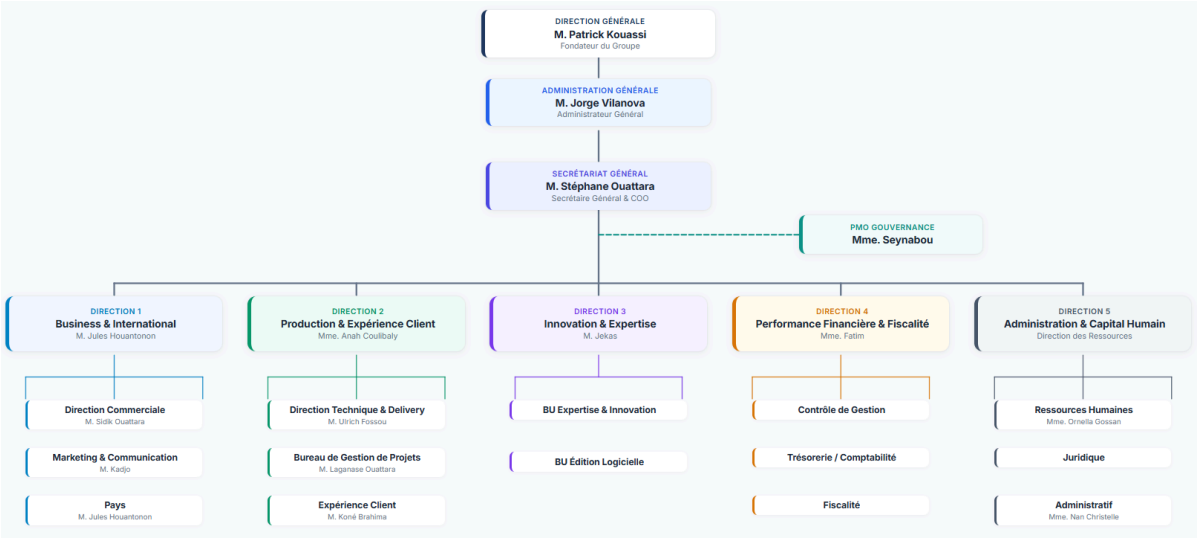


Fig. 1. – Organigramme du Groupe Synelia — Gouvernance, directions opérationnelles et services rattachés (juin 2026).

Dimension	Caractérisation du groupe Synelia
Identité	ESN africaine indépendante depuis 2018, issue de Smile Group Afrique (présence depuis 2012).
Implantations	Côte d'Ivoire (siège), Burkina Faso, Libéria et expansion régionale en Afrique de l'Ouest.
Effectif	Plus de 110 profils technologiques locaux et internationaux.
Activité	Plus de 103 projets livrés et environ 90 clients actifs accompagnés.
Expertises	Hosting and Cloud, API Management, Big Data and Machine Learning, Cybersécurité, IA, Automatisation de processus.
Offres dérivées	Open Digital Academy, distribution Mobile Money, solutions sectorielles, contrôle qualité des recrutements.
Gouvernance	Direction Générale (Patrick KOUASSI) ; Administration Générale (Jorge VILANOVA) ; SG (Stéphane OUATTARA) ; cinq directions opérationnelles.

Dans ce contexte, le recrutement n'est pas une fonction périphérique mais un levier stratégique direct. La capacité du groupe à délivrer ses six expertises technologiques, à animer son académie de formation et à honorer ses engagements clients dépend de la régularité avec laquelle il identifie, évalue et intègre de nouveaux talents — qu'il s'agisse de jeunes diplômés issus de l'Open Digital Academy, de consultants confirmés sur les

expertises techniques de pointe ou de profils transverses pour les fonctions support. Le cahier des charges interne mentionne plusieurs dizaines de recrutements par an, répartis sur plusieurs spécialités et plusieurs niveaux d'expérience, ce qui crée un besoin de structuration durable que ne peut pas couvrir une logique artisanale.

Au-delà du volume, la nature même de l'activité de Synelia impose une exigence forte sur la *qualité* et la *traçabilité* des recrutements. Le groupe communique sur l'excellence opérationnelle, la conformité aux normes et la sécurité de ses interventions ; il est donc cohérent que sa propre chaîne de recrutement adopte le même niveau de rigueur : critères homogènes, évaluations techniques objectivées, historique consolidé des décisions, gouvernance fine des permissions entre recruteurs et évaluateurs. C'est précisément ce besoin que vient adresser SyneliaQuizApp, en outillant le pôle « Contrôle qualité des recrutements » et en s'appuyant sur les compétences de la Direction Technique pour proposer une plateforme conforme aux standards techniques du groupe.

1.1.2 Le processus de recrutement traditionnel

Dans un fonctionnement traditionnel, le processus de recrutement suit une chaîne relativement connue. Une offre ou une campagne est d'abord diffusée par différents canaux, tels que l'email, les réseaux professionnels ou les plateformes d'emploi. Les candidatures sont ensuite reçues, triées et relues manuellement selon des critères qui peuvent varier d'un recruteur à l'autre.

Vient ensuite une phase d'évaluation, souvent réalisée à partir d'entretiens techniques, de questionnaires externes ou de jugements informels portés sur le dossier du candidat. Les entretiens doivent être planifiés, confirmés et suivis, ce qui multiplie les échanges. Enfin, une décision de présélection, de poursuite ou de rejet est prise, puis communiquée au candidat avec plus ou moins de formalisation.

Pris isolément, chacun de ces maillons paraît simple. Mais lorsqu'ils sont répartis entre plusieurs outils et plusieurs personnes, ils génèrent rapidement des pertes d'information, des lenteurs et une forte dépendance à la mémoire individuelle des intervenants. Comme le soulignent D. S. Chapman et J. Webster dans leur article **The use of technologies in the recruiting, selection, and evaluation of job applicants** (2003), l'usage d'outils numériques intégrés dans le recrutement ne se limite pas à un gain de productivité : il transforme la capacité même de l'organisation à standardiser, comparer et documenter ses évaluations.

1.1.3 Les défis et limitations actuels

Les principales difficultés du processus initial tiennent d'abord à la fragmentation des outils. Les candidatures peuvent arriver par plusieurs canaux, les informations sont

réparties entre boîtes mail, documents partagés et fichiers de suivi, et l'historique complet d'un candidat devient difficile à reconstituer. Cette dispersion nuit directement à la réactivité et à la qualité du pilotage.

Une seconde limite concerne l'absence de standardisation des évaluations. Sans grille commune ni mécanisme homogène de test technique, la comparaison entre candidats devient moins objective et dépend fortement de l'interprétation de chaque recruteur. D. L. Stone, D. L. Deadrick, K. M. Lukaszewski et R. Johnson, dans **The influence of technology on the future of human resource management** (2015), ont montré que la technologie, lorsqu'elle est correctement intégrée au processus RH, constitue un levier puissant pour réduire cette subjectivité et harmoniser les critères de sélection. Ce problème est particulièrement sensible dans un environnement où les compétences techniques doivent être évaluées avec rigueur.

Enfin, le manque de traçabilité fragilise la gouvernance du recrutement. Il devient difficile de savoir pourquoi une décision a été prise, quelle source de candidature est la plus efficace, quel acteur est intervenu à quel moment ou quels retards affectent l'expérience candidat. La lourdeur organisationnelle qui en découle peut impacter à la fois la qualité du recrutement et l'image de l'entreprise.

1.1.4 La nécessité d'une solution digitale

Dans ce contexte, une plateforme interne unifiée apparaît comme une réponse pertinente pour plusieurs raisons. T. Bondarouk et M. R. Olivas-Luján, dans **Electronic HRM in the Smart Era** (2013), qualifient cette transition de passage à un *electronic HRM*, où la technologie ne se contente pas de reproduire des processus papier mais redéfinit la manière dont l'organisation interagit avec ses candidats. Elle permet d'abord de centraliser les informations et d'offrir un référentiel commun à tous les acteurs du recrutement. Elle permet ensuite d'automatiser certaines opérations répétitives, comme la vérification d'email, l'envoi de notifications, la mise à disposition du quiz ou la consolidation de certains états métier.

Une telle solution favorise également la standardisation du processus. En structurant les campagnes, les statuts, les évaluations et les permissions, elle réduit la variabilité des pratiques et facilite la comparaison entre candidatures. Enfin, elle améliore la transparence du parcours candidat en donnant accès à un suivi plus clair, ce qui contribue à renforcer la qualité de l'expérience utilisateur et, par extension, l'image employeur de SYNELIA.

1.2 Objectifs du projet

Les objectifs du projet doivent être compris à deux niveaux complémentaires. D'un côté, il existe des objectifs stratégiques liés à la transformation du processus de recrutement et à l'amélioration de la performance organisationnelle. De l'autre, le projet poursuit des objectifs opérationnels précis, directement traduits dans les modules et parcours de SyneliaQuizApp.

1.2.1 Objectifs généraux

L'objectif général principal consiste à **digitaliser le processus de recrutement** en remplaçant une logique éclatée par une plateforme web centralisée. Cette digitalisation vise à réduire les manipulations manuelles, à structurer les échanges et à rendre plus visibles les différentes étapes du parcours candidat.

Le deuxième objectif général est **d'automatiser l'évaluation technique** grâce à l'intégration d'un système de quiz configurable, permettant de pré-filtrer ou de préqualifier certains profils de manière plus homogène. Cette automatisation n'a pas vocation à remplacer totalement le jugement humain, mais à l'appuyer par un socle de vérification plus objectif.

Enfin, le projet cherche à **améliorer la collaboration interne** entre les acteurs du recrutement. Cela suppose de partager les informations utiles, de distribuer les droits de manière adaptée, de suivre les interventions de chacun et de permettre une meilleure coordination des évaluations, des entretiens et des décisions.

1.2.2 Objectifs spécifiques

Sur le plan opérationnel, SyneliaQuizApp vise à couvrir plusieurs besoins concrets. Il s'agit d'abord de permettre la création de campagnes de recrutement configurables, associées à des formulaires et à des droits d'accès adaptés. L'application doit ensuite offrir un parcours candidat cohérent, depuis la soumission initiale jusqu'au suivi public de la candidature.

Le projet doit également prendre en charge le quiz technique, sa diffusion, sa reprise éventuelle, sa correction automatique ou manuelle et sa restitution dans le contexte global du recrutement. À cela s'ajoutent la planification des entretiens, la collecte d'évaluations collaboratives, la production de décisions, la gestion des notifications et l'administration des données de référence nécessaires au fonctionnement du système.

En pratique, ces objectifs spécifiques traduisent une ambition claire : couvrir le cœur du cycle de recrutement dans une même plateforme, avec une profondeur fonctionnelle suffisante pour soutenir un usage réel.

1.3 Périmètre fonctionnel

Le périmètre de la solution a été défini de manière à couvrir le cœur du besoin exprimé par SYNELIA, tout en conservant une limite raisonnable pour une première version cohérente. L'application se concentre donc sur la gestion du recrutement, de la candidature à la décision, avec les modules nécessaires à la circulation de l'information et à l'évaluation des profils.

1.3.1 Modules couverts par l'application

Les modules effectivement couverts par l'application comprennent l'authentification et la gestion des accès, la gestion des campagnes de recrutement, les formulaires et questionnaires, le dépôt et le suivi des candidatures, le quiz technique, la planification des entretiens, les évaluations collaboratives, les décisions, les notifications et les tableaux de bord d'administration ou de suivi.

À ces modules s'ajoutent plusieurs briques transverses indispensables, comme la gestion des modèles d'email, la configuration SMTP, les référentiels métier, les permissions par rôle et par entité, ainsi que les vues techniques d'administration permettant d'exploiter l'application dans la durée.

1.3.2 Acteurs du système

Le système met en interaction quatre grandes catégories d'acteurs. Le **super administrateur** intervient sur les paramètres sensibles et les ressources globales de l'application. L'**administrateur** gère les données de référence, certains écrans techniques et une partie des configurations nécessaires au fonctionnement de la plateforme.

Le **recruteur** ou gestionnaire métier constitue l'acteur principal du processus opérationnel : il crée les campagnes, suit les candidatures, planifie les entretiens, consulte les évaluations et participe aux décisions. Le **candidat**, quant à lui, interagit avec l'application à travers des parcours publics sécurisés, principalement pour soumettre sa candidature, passer un quiz, consulter son suivi ou répondre à des sollicitations liées à son dossier.

1.3.3 Frontières du projet

Le projet inclut donc le cœur du processus de recrutement, mais il n'a pas vocation à couvrir l'ensemble du système d'information RH de l'entreprise. Des fonctionnalités comme l'intégration directe avec les job boards, la visioconférence intégrée, la gestion de la paie ou des contrats, l'application mobile native, le multilingue ou le matching automatique par intelligence artificielle sont volontairement laissées hors du périmètre de cette version.

Cette délimitation est importante d'un point de vue méthodologique. Elle permet de concentrer l'effort sur une version robuste du processus principal, plutôt que de disperser le développement sur des fonctions annexes encore prématurées. Le choix retenu privilégie donc la cohérence métier, la stabilité de l'architecture et la qualité du noyau fonctionnel.

1.4 Contraintes du projet

Le projet s'inscrit dans un ensemble de contraintes qu'il convient de reconnaître explicitement. Sur le plan technique, l'application doit rester accessible depuis un navigateur web moderne, proposer des temps de réponse acceptables sur les opérations courantes et conserver une architecture suffisamment claire pour être maintenue et étendue. Elle doit également composer avec plusieurs technologies complémentaires et avec un modèle de données évolutif.

Sur le plan organisationnel, la solution doit pouvoir être prise en main par des profils différents, ce qui implique un effort de lisibilité des écrans, une gestion des droits compréhensible et un accompagnement au changement. Le recrutement étant un processus transverse, le système ne peut être utile que s'il parvient à articuler les besoins de plusieurs acteurs sans les noyer dans une complexité excessive.

Des contraintes réglementaires et opérationnelles pèsent également sur le projet, en particulier autour de la protection des données personnelles des candidats, de la traçabilité des décisions et de la maîtrise des communications sortantes. Enfin, comme tout projet mené dans le cadre d'un stage, SyneliaQuizApp a dû être développé dans un temps limité, avec une nécessité permanente de prioriser entre profondeur fonctionnelle, qualité technique et faisabilité dans le délai imparti.

CHAPITRE 2

Étude de l'existant et état de l'art

Ce chapitre positionne la solution développée par rapport aux pratiques existantes et aux solutions du marché. Il sert également à justifier les choix technologiques retenus pour la mise en œuvre du système.

2.1 Étude de l'existant

L'existant chez SYNELIA se caractérisait par une organisation fonctionnelle identifiable, mais fortement dispersée sur le plan outillé. Les candidatures pouvaient être collectées par différents canaux, puis suivies à l'aide d'emails, de fichiers bureautiques, de tableaux de suivi et de formulaires externes. Cette organisation avait l'avantage d'être rapide à mettre en place au départ, sans nécessiter de solution métier spécifique. Elle permettait aussi une certaine souplesse locale dans la manière de traiter les dossiers.

Toutefois, cette souplesse avait pour contrepartie un manque de centralisation. Les informations utiles à l'analyse d'une candidature n'étaient pas toujours réunies au même endroit, les critères d'évaluation variaient selon les acteurs, et la reconstitution du parcours complet d'un candidat devenait coûteuse en temps. L'existant présentait donc une valeur opérationnelle minimale, mais peinait à soutenir une logique de croissance, de collaboration structurée et de traçabilité fiable.

Il convient néanmoins de souligner que cet existant mettait déjà en évidence les besoins réels de l'entreprise. Les recruteurs avaient déjà identifié la nécessité de diffuser des campagnes, de recueillir des informations standardisées, de tester les compétences, de suivre les statuts et de produire des décisions. En ce sens, le projet SyneliaQuizApp ne part pas d'un vide fonctionnel ; il cherche plutôt à rationaliser et à intégrer des pratiques déjà présentes, mais encore insuffisamment outillées.

2.2 Benchmark des solutions existantes

Le marché propose aujourd'hui de nombreuses solutions de recrutement, parmi lesquelles des ATS généralistes comme Workable ou Greenhouse, ainsi que des plateformes plus spécialisées dans l'évaluation technique comme HackerRank, Codility ou TestGorilla. Les premières se distinguent généralement par leur capacité à structurer les campagnes,

centraliser les candidats, suivre les statuts et produire des tableaux de bord. Les secondes mettent davantage l'accent sur la conception de tests, la notation automatisée et l'évaluation comparée des compétences.

Ces solutions offrent des fonctionnalités avancées et une forte maturité produit, mais elles ne répondent pas toujours de manière optimale à un besoin interne spécifique. D'une part, elles peuvent imposer des modèles de workflow peu flexibles par rapport à l'organisation réelle de l'entreprise. D'autre part, elles introduisent souvent une dépendance à une plateforme externe, à ses coûts, à ses conventions et à ses limites de personnalisation. Enfin, lorsqu'il faut articuler très finement les permissions, les parcours publics, les états métier et les règles propres à l'entreprise, une solution générique peut s'avérer moins adaptée qu'un outil pensé sur mesure.

Le benchmark met donc en évidence un arbitrage classique. Les solutions du marché permettent de démarrer rapidement avec un haut niveau de finition. En revanche, une solution interne comme SyneliaQuizApp peut offrir une meilleure maîtrise du modèle de données, des parcours utilisateurs, des règles métier et de l'évolution future du produit, à condition de supporter l'effort initial de conception et de développement.

2.3 État de l'art technologique

L'état de l'art retenu pour ce projet repose sur une architecture web en couches, devenue aujourd'hui un standard pour les applications métier. Le backend est porté par un framework serveur capable d'exposer des services structurés, d'assurer la sécurité, de gérer la persistance et d'orchestrer les règles métier. Le frontend, quant à lui, s'appuie sur un framework orienté composants permettant de construire des interfaces riches, interactives et organisées autour de parcours utilisateurs distincts.

Dans ce paysage, **Spring Boot** s'impose comme une référence pour les applications Java orientées API. Il facilite la mise en place de services REST, l'intégration avec la sécurité, la persistance via JPA et l'organisation modulaire du code. **Angular** représente de son côté une solution robuste pour les applications monopage complexes, en particulier lorsqu'il faut structurer de nombreux écrans, partager des composants et maintenir une logique de navigation cohérente entre plusieurs profils d'utilisateurs.

La couche de données relationnelle reste également un choix pertinent pour un domaine aussi structuré que le recrutement. Une base comme **MySQL** permet de modéliser proprement les relations entre campagnes, candidatures, formulaires, tentatives de quiz, entretiens, évaluations, décisions et permissions. Enfin, l'architecture **REST** fournit un cadre clair pour les échanges entre frontend et backend, tandis que les principes modernes

de sécurité web imposent une séparation rigoureuse entre authentification, autorisation, validation des entrées, contrôle des accès publics et protection des données sensibles.

2.4 Justification des choix technologiques

Le choix d'une pile technologique ne peut se réduire à une préférence d'usage. Dans un projet d'ingénierie destiné à durer, il doit être justifié au regard d'alternatives crédibles, à la lumière de critères stables : adéquation au domaine métier, maturité de l'écosystème, qualité des outils de sécurité, capacité de structuration d'un projet de taille intermédiaire, courbe d'apprentissage, et compatibilité avec les contraintes d'exploitation de l'entreprise. La démarche retenue ici a donc consisté à présenter, pour chaque couche de la solution, les options envisagées, à les confronter selon ces critères, puis à argumenter la sélection finale.

2.4.1 Critères de sélection retenus

Avant toute comparaison, il était nécessaire d'expliciter une grille de lecture commune. Les six critères suivants ont structuré l'évaluation des technologies envisagées et resteront utilisés tout au long du chapitre.

Critère	Lecture appliquée au projet SyneliaQuizApp
Adéquation métier	Capacité à modéliser un domaine riche en règles de gestion, transitions d'état, permissions contextualisées et historisation.
Maturité et écosystème	Stabilité du framework, fréquence de mises à jour, qualité des bibliothèques tierces et taille de la communauté.
Sécurité native	Disponibilité d'outillage de sécurité éprouvé : authentification, autorisation, validation, protection contre les vulnérabilités courantes.
Structuration d'un projet moyen	Aptitude à imposer une organisation claire des couches et à supporter la croissance progressive du code.
Productivité et apprentissage	Temps d'amorçage, lisibilité du code et adéquation avec les compétences déjà disponibles dans l'équipe.
Coût et licence	Modèle open source, absence de coût récurrent et compatibilité avec un déploiement interne maîtrisé par SYNELIA.

2.4.2 Choix du framework backend

Plusieurs frameworks backend pouvaient légitimement être retenus pour porter la logique métier de SyneliaQuizApp. Le projet exige un cadre capable de soutenir un domaine fortement orienté données, de gérer une matrice de permissions complexe et d'exposer une API REST stable. L'évaluation a donc porté sur les principales options actuelles du marché.

Option envisagée	Forces principales	Limites par rapport au besoin
Spring Boot (Java)	Écosystème mature, intégration native avec JPA et Spring Security, modèle en couches imposant une structure claire, large adoption en entreprise.	Verbosité du langage Java et courbe d'apprentissage initiale plus marquée que pour un framework dynamique.
Node.js avec Express ou NestJS	Productivité initiale élevée, écosystème JavaScript unifié avec le frontend, démarrage rapide pour des API simples.	Faiblesse structurelle d'Express sur les domaines complexes ; absence d'un standard sécurité comparable à Spring Security ; gestion des transactions et de la persistance moins robuste sur un domaine fortement relationnel.
Django (Python)	Productivité forte, ORM intégré, écosystème éprouvé en applications de gestion, batteries-included.	Moins adapté à une API REST riche en règles métier différenciées par contexte ; gouvernance des permissions fines plus artisanale ; moindre familiarité de l'équipe avec Python en contexte SYNELIA.
ASP.NET Core	Performances élevées, sécurité solide, outillage Microsoft de qualité, structuration claire.	Couplage culturel à l'écosystème Microsoft, coût d'environnement Windows historiquement plus marqué, ressources internes Java plus disponibles que C# chez SYNELIA.
Laravel (PHP)	Productivité rapide pour CRUD, syntaxe expressive, ORM agréable.	Modèle de threading et de typage moins adapté à un domaine fortement transactionnel et à des permissions complexes ;

		positionnement plus orienté sites de gestion classiques que plateformes métier critiques.

Sur l'ensemble de la grille, **Spring Boot** s'est imposé comme le choix le plus équilibré. Comme le souligne Craig Walls dans **Spring in Action** (6e édition, 2022), il offre un cadre éprouvé en environnement professionnel, une intégration native avec Spring Security indispensable au modèle de permissions du projet, une couche de persistance JPA cohérente avec le caractère fortement relationnel du domaine, et une organisation en couches qui supporte sans difficulté la croissance du code. L'inconvénient principal, lié à la verbosité de Java, est compensé par la lisibilité durable du code et par la fiabilité accrue qu'apporte un typage statique strict dans un projet manipulant des règles de gestion sensibles. Le contexte SYNELIA, où Java reste la pile dominante, a achevé de consolider ce choix sur le plan opérationnel.

2.4.3 Choix du framework frontend

Côté frontend, le projet impliquait un nombre élevé d'écrans, des parcours distincts par profil utilisateur, des règles d'affichage dépendantes des permissions, et la nécessité de maintenir une cohérence visuelle sur la durée. Trois familles de frameworks ont été comparées.

Option envisagée	Forces principales	Limites par rapport au besoin
Angular	Framework opinionné, TypeScript par défaut, architecture modulaire imposée, injection de dépendances native, routage et formulaires intégrés, outillage CLI complet.	Courbe d'apprentissage plus marquée et verbosité initiale supérieure à des frameworks plus légers.
React	Souplesse, large écosystème, communauté très active, productivité élevée pour les interfaces composables.	Non opinionné : nécessite des choix complémentaires sur le routage, les formulaires, l'état global et l'organisation modulaire, ce qui fragilise la cohérence d'un projet à long terme s'il n'est pas strictement encadré.

Vue.js	Bonne ergonomie, courbe d'apprentissage douce, qualité de la documentation.	Écosystème plus restreint en contexte entreprise francophone ; intégration native de la sécurité et des grands modules métier moins structurée qu'Angular.

Le choix d'**Angular** répond directement à la nature du produit. Comme le documente Adam Freeman dans **Pro Angular 16** (2023), la richesse du domaine, la coexistence d'une surface publique candidate et d'une surface interne, la nécessité d'imposer une structure modulaire stable et le besoin d'un typage strict dès la couche présentation rendent un framework opinionné plus pertinent qu'un framework librement composable. L'usage par défaut de TypeScript, l'injection de dépendances, le routage intégré et la gestion native des formulaires réactifs apportent un cadre de travail homogène, ce qui réduit le coût de maintenance et facilite l'arrivée de nouveaux contributeurs sur le projet. React et Vue restent techniquement valables, mais leur souplesse devient un risque lorsqu'aucune équipe dédiée n'est en mesure de figer durablement les conventions transverses.

2.4.4 Choix du système de gestion de base de données

Le domaine du recrutement est intrinsèquement relationnel : campagnes, formulaires, versions, candidatures, étapes, entretiens, évaluations, décisions et permissions sont liés par des contraintes d'intégrité fortes. Le choix du SGBD devait donc privilégier la cohérence transactionnelle, la richesse des types relationnels et la qualité de l'outillage de migration.

Option envisagée	Forces principales	Limites par rapport au besoin
MySQL 8	Maturité éprouvée, large adoption en entreprise, intégration naturelle avec Spring Boot et Flyway, outillage d'exploitation accessible, support solide des contraintes d'intégrité.	Quelques limitations historiques sur les types avancés, partiellement comblées par MySQL 8.
PostgreSQL	Très riche fonctionnellement : types avancés, JSONB performant, contraintes d'exclusion, extensions nombreuses.	Surdimensionné par rapport aux besoins immédiats du projet ; sa richesse aurait introduit une complexité d'exploitation

		supérieure sans bénéfice métier immédiat.
MongoDB	Souplesse documentaire, schéma flexible, productivité initiale pour des modèles peu contraints.	Inadapté au domaine : la richesse relationnelle du recrutement aurait dû être réimplémentée applicativement, au détriment de la cohérence et de la traçabilité exigées par le projet.
MariaDB	Compatibilité étroite avec MySQL et gouvernance communautaire.	Aucun gain fonctionnel décisif par rapport à MySQL 8 dans le contexte du projet.

MySQL 8 a été retenu parce qu'il offre le meilleur compromis entre richesse fonctionnelle, cohérence transactionnelle et simplicité d'exploitation. Sa parfaite intégration avec JPA et Hibernate, dont C. Bauer, G. King et G. Gregory, dans **Java Persistence with Hibernate** (2e édition, 2016), décrivent les principes de mapping objet-relationnel, ainsi qu'avec Flyway pour les migrations versionnées, permet d'industrialiser les évolutions de schéma, ce qui est essentiel dans un projet en évolution continue. PostgreSQL aurait été un choix techniquement défendable, mais son surplus fonctionnel n'apportait pas de valeur métier additionnelle au regard des besoins identifiés. MongoDB, en revanche, a été écarté sans ambiguïté : substituer un modèle documentaire à un domaine relationnel aussi structuré aurait conduit à déplacer dans le code applicatif les contraintes que la base sait nativement garantir, au prix d'une fragilité accrue.

2.4.5 Synthèse comparative

La pile retenue se distingue donc moins par une supériorité absolue de chacune de ses briques que par la cohérence d'ensemble qu'elle produit. Spring Boot, Angular et MySQL partagent une logique commune : framework opinionné, typage strict, structure imposée et écosystème stable. Cette homogénéité de philosophie est, pour un projet métier de cette taille, un avantage déterminant.

Couche	Choix retenu	Alternatives écartées	Raison principale du choix
Backend	Spring Boot	Node.js, Django, ASP.NET Core, Laravel	Structure imposée, sécurité intégrée et adéquation à un

			domaine fortement relationnel.
Frontend	Angular	React, Vue.js	Framework opinionné et typage strict adaptés à une application multi-profils riche en règles d'affichage.
Base de données	MySQL 8	PostgreSQL, MongoDB, MariaDB	Équilibre entre intégrité relationnelle, simplicité d'exploitation et intégration avec JPA et Flyway.
Migrations	Flyway	Liquibase, scripts manuels	Versionnage simple, lisible et compatible avec une démarche de livraison incrémentale.
Architecture	REST en couches	GraphQL, monolithe sans couches, microservices	Adapté au volume et à la maturité du projet sans complexité d'exploitation inutile.

Au-delà de la justification individuelle de chaque brique, ces choix servent directement les objectifs du projet. La séparation frontend/backend rend le système plus évolutif. L'usage d'une base relationnelle soutient la richesse des relations métier. Les frameworks retenus facilitent la mise en œuvre de la sécurité, des notifications, de la documentation d'API et de la maintenance progressive du produit. Le choix technologique n'est donc pas seulement un choix d'outils ; il participe pleinement à la qualité architecturale de la solution proposée.

2.5 Critique de l'existant et solution proposée

L'analyse de l'existant et du marché montre clairement que le problème de SYNELIA n'est pas l'absence totale d'outils, mais l'absence d'un outil unificateur, cohérent avec ses besoins réels. Les pratiques historiques permettaient certes d'assurer un traitement minimal des candidatures, mais au prix d'une forte charge manuelle, d'une faible visibilité

globale et d'une standardisation insuffisante. À l'inverse, les solutions du marché offrent une réponse industrialisée, mais pas toujours parfaitement alignée sur les contraintes, les préférences et le niveau de contrôle attendu par l'entreprise.

SyneliaQuizApp se positionne précisément dans cet espace. La solution proposée vise à centraliser les campagnes, les candidatures, les quiz, les entretiens, les évaluations et les décisions dans une plateforme unique, tout en intégrant des mécanismes de permissions fines, de suivi public candidat et de configuration transverse. Sa valeur ajoutée tient donc moins à la reproduction de fonctions déjà connues qu'à leur articulation dans un système unifié, pensé pour le contexte spécifique de SYNELIA.

Ce chapitre permet ainsi de justifier le projet à un double niveau : fonctionnellement, il répond à de réelles insuffisances du processus existant ; technologiquement, il s'appuie sur des choix cohérents avec l'état de l'art tout en assumant une logique de personnalisation et de maîtrise interne. La suite du mémoire montrera comment cette orientation a été traduite en besoins structurés, puis en solution concrète.

Deuxième partie

Analyse et conception

Formalisation des besoins, règles de gestion et architecture de la solution

CHAPITRE 3

Analyse des besoins

Ce chapitre a pour objectif de démontrer que la solution répond à des besoins clairement identifiés, priorisés et structurés selon une logique d'ingénierie logicielle. Il ne s'agit pas seulement d'énumérer des fonctionnalités, mais de montrer comment les attentes des acteurs, les contraintes d'exploitation et les règles du recrutement ont été transformées en exigences cohérentes, traçables et concevables.

3.1 Méthodologie d'analyse

L'analyse des besoins a été menée selon une démarche progressive et itérative, ancrée dans les usages réels du projet plutôt que dans une modélisation abstraite déconnectée de l'implémentation. Cette approche rejoint les principes du développement itératif et de la conception dirigée par les cas d'utilisation décrits par Craig Larman dans **UML et les design patterns** (2005). Le point de départ a été le cahier des charges fonctionnel, enrichi ensuite par les cartographies métier, les notes de debug, les listes de tâches, les inventaires techniques et les observations issues des validations successives du produit. Cette approche a permis de confronter en permanence le besoin théorique, le comportement du code et les arbitrages effectivement retenus pendant le développement.

Les besoins ont ensuite été regroupés en modules fonctionnels, puis relus à travers des critères transverses : sécurité, permissions, cohérence des états, traçabilité, qualité de l'expérience utilisateur et capacité d'évolution. Ce choix de lecture par parcours et par règles métier est particulièrement important dans un projet comme SyneliaQuizApp, où une fonctionnalité ne peut pas être correctement comprise si elle est isolée de son contexte de campagne, de permission ou de pipeline.

3.1.1 Parties prenantes et périmètre étudié

La compréhension du besoin a d'abord reposé sur l'identification des acteurs ayant une influence directe sur le système. Le mémoire retient quatre profils structurants, chacun portant des attentes propres et des risques métier spécifiques. Cette lecture par parties prenantes permet de justifier les choix d'interface, de sécurité et de structuration des modules.

Acteur	Attentes principales	Risque métier à maîtriser	Conséquence directe sur la conception
Candidat	Postuler sans friction, comprendre les étapes, conserver un suivi autonome.	Abandon en cours de parcours, incompréhension des statuts, perte de confiance dans le processus.	Surface publique simple, sécurisée, guidée et clairement séparée des écrans internes.
Recruteur / évaluateur	Piloter les campagnes, filtrer les dossiers, collaborer sur les entretiens et les évaluations.	Décisions prises hors contexte, dispersion de l'information, difficulté à partager l'évaluation.	Vues consolidées par campagne, historique d'avancement et permissions contextualisées.
Administrateur	Gérer les utilisateurs, les référentiels et le paramétrage d'exploitation.	Dérive des droits, données incohérentes, configuration fragile.	Référentiels administrables, contrôle d'accès fin et traçabilité des actions techniques.
Super administrateur	Gouverner la plateforme, superviser la sécurité et maintenir la configuration globale.	Mauvaise gouvernance des rôles, défaut de supervision ou de paramétrage global.	Séparation nette entre administration métier et administration de plateforme.

3.1.2 Démarche de formalisation

La formalisation des besoins a suivi quatre opérations complémentaires : recenser les parcours métier réels, regrouper les attentes par modules applicatifs, exprimer les règles de gestion qui conditionnent les transitions d'état, puis vérifier la cohérence de cette modélisation avec le schéma de données, les écrans existants et les validations déjà réalisées. Cette méthode a permis d'éviter une double impasse fréquente dans les mémoires techniques : d'un côté un discours purement théorique sans ancrage logiciel, de l'autre une description du code dépourvue de logique d'analyse.

3.2 Besoins fonctionnels

Les besoins fonctionnels du projet peuvent être structurés en plusieurs blocs applicatifs complémentaires, chacun correspondant à un segment du processus de recrutement. Cette structuration relie les attentes métier aux modules effectivement conçus dans la

solution et montre que le produit répond à une logique de parcours complet plutôt qu'à une accumulation de fonctionnalités indépendantes.

3.2.1 Authentification et gestion des utilisateurs

Le système doit permettre l'authentification des utilisateurs internes, la gestion de leurs rôles et le contrôle précis de leurs droits d'accès. Les administrateurs et recruteurs ont besoin d'accéder à des espaces différenciés, avec des permissions cohérentes selon leur responsabilité réelle. L'application doit donc gérer à la fois des droits globaux liés au rôle système et des droits contextualisés, portés par une campagne ou par une entité particulière.

Pour les parcours publics, le besoin est différent : le candidat ne doit pas disposer d'un accès interne classique, mais d'un ensemble de points d'entrée sécurisés, restreints à sa propre candidature. Cela implique des mécanismes dédiés de vérification d'email, de codes de suivi et de liens contextualisés, distincts du modèle d'authentification des utilisateurs internes.

3.2.2 Gestion des données de référence

Le projet nécessite un socle de données de référence administrables, sans lequel le recrutement ne peut pas être structuré durablement. Cela inclut notamment les départements, les spécialités, les questions de quiz, les réponses attendues, les formulaires, les groupes métier et plusieurs référentiels transverses nécessaires au fonctionnement métier et aux écrans d'administration.

Le besoin ne porte pas seulement sur la création de ces données, mais aussi sur leur cohérence, leur réutilisation et leur évolution. Une question peut être partagée par plusieurs formulaires, un formulaire peut être versionné, et certaines données doivent être historisées indirectement pour éviter que des modifications ultérieures ne déforment la lecture des campagnes déjà utilisées. L'administration des référentiels constitue donc une base essentielle de la qualité du système.

3.2.3 Campagnes de recrutement

La campagne de recrutement constitue l'unité de pilotage centrale du système. Le besoin métier impose de pouvoir créer une campagne, définir son état de cycle de vie, lui associer un pipeline, lui rattacher un ou plusieurs formulaires, générer des liens de diffusion et partager son accès avec des collaborateurs internes selon des permissions fines.

La campagne doit également servir de point de convergence pour le suivi des candidatures, des entretiens, des évaluations et des décisions. Elle doit donc supporter à la fois une logique de configuration et une logique de lecture consolidée. L'entreprise attend en

particulier une meilleure visibilité sur les campagnes actives, sur les profils qu'elles visent et sur les actions qu'elles autorisent aux différents intervenants.

3.2.4 Candidatures et suivi candidat

Le système doit offrir un parcours de candidature public simple et fiable, permettant au candidat de renseigner ses informations, de transmettre ses pièces, de vérifier son adresse email et d'obtenir un moyen de suivi autonome de son dossier. Ce besoin est fondamental, car il conditionne à la fois la qualité de la donnée collectée et la perception du sérieux de l'entreprise par les candidats.

Au-delà de la soumission initiale, la solution doit permettre un suivi public contextualisé : consultation de l'état d'avancement, visibilité sur les actions possibles, réponse à une convocation, et, lorsque la règle métier l'autorise, demande de correction d'information ou abandon du processus. Le système doit donc gérer une frontière claire entre les actions publiques autorisées et les opérations strictement réservées aux équipes internes.

3.2.5 Quiz technique

Le quiz technique répond à un besoin de standardisation et d'objectivation de l'évaluation initiale des candidats. L'application doit pouvoir rattacher un formulaire publié à une campagne, présenter les questions autorisées, gérer le temps imparti si nécessaire, sauvegarder les réponses et calculer un score exploitable dans la suite du pipeline.

Ce besoin va plus loin qu'un simple test ponctuel. Le système doit également distinguer l'état du quiz, la progression du candidat, la correction manuelle éventuelle et le rattachement des réponses à une version de formulaire figée. Cette exigence est déterminante pour garantir la comparabilité des résultats, la relecture des copies et la traçabilité des corrections.

3.2.6 Entretiens, évaluations et décisions

Le projet doit permettre la planification d'entretiens reliés à une étape précise du pipeline, la communication de la convocation au candidat, la prise en compte de sa réponse et la mise à jour du statut correspondant. L'entretien n'est donc pas conçu comme un simple rendez-vous, mais comme un événement métier ayant des effets sur l'état du dossier, sur la disponibilité des actions et sur la suite du processus de recrutement.

L'évaluation doit être collaborative, détaillée et contextualisée. Plusieurs évaluateurs peuvent être amenés à produire des observations ou des notes à propos d'un même candidat. Enfin, le système doit permettre la prise de décision, qu'il s'agisse d'une présélection, d'une décision finale, d'une annulation ou d'un retour arrière métier. Le besoin central ici est la cohérence entre jugement, historisation et évolution du dossier candidat.

3.2.7 Notifications et dashboard

Le besoin ne se limite pas à stocker des données ; il consiste aussi à faire circuler l'information au bon moment. L'application doit donc générer des notifications internes et des emails pour signaler les événements significatifs du processus : vérification, convocation, réponse candidat, résultat d'une demande, décision ou défaut de configuration critique.

Parallèlement, le dashboard doit fournir des vues synthétiques permettant de suivre l'activité, de filtrer les dossiers, d'identifier les actions en attente et de faciliter le pilotage quotidien par les équipes. Le système doit ainsi rendre visibles les éléments réellement utiles à la décision, sans noyer l'utilisateur dans une masse d'informations non contextualisées.

3.2.8 Matrice synthétique des besoins fonctionnels

Afin de transformer ces constats en cahier des charges exploitable, les besoins fonctionnels ont été reformulés sous forme de matrice. Chaque ligne exprime un objectif métier, les fonctionnalités attendues et la traduction opérationnelle qui devra être vérifiable lors de la recette.

ID	Module	Attentes métier	Réponse attendue du système
BF-01	Authentification et droits	Sécuriser les accès internes et différencier précisément les responsabilités.	Gestion des rôles système, overrides, permissions par campagne et contrôle des accès publics.
BF-02	Référentiels	Disposer de données métier stables, administrables et réutilisables.	CRUD des départements, spécialités, questions, réponses, groupes métier et paramètres transverses.
BF-03	Campagnes	Créer et piloter une campagne de recrutement de bout en bout.	Configuration du cycle de vie, association d'un pipeline, génération de liens de diffusion et partage collaboratif.
BF-04	Candidatures	Collecter des dossiers fiables et offrir un suivi autonome au candidat.	Soumission publique, vérification d'email, suivi par code candidat,

			consultation des statuts et actions publiques autorisées.
BF-05	Quiz	Objectiver l'évaluation initiale des candidats.	Publication d'un formulaire versionné, passation chronométrée, sauvegarde des réponses, score, correction manuelle et historisation.
BF-06	Entretiens et décisions	Coordonner la suite du processus après le quiz.	Planification, convocation, réponse candidat, évaluation collaborative, décision et mise à jour du pipeline.
BF-07	Notifications	Informer les acteurs au moment utile.	Emails, notifications internes et retours d'action cohérents avec les événements métier du recrutement.
BF-08	Pilotage	Donner de la visibilité aux équipes RH.	Tableaux de bord, listes filtrables, statistiques utiles et vues consolidées par campagne, dossier et intervenant.

3.2.9 Matrice acteurs et responsabilités

La matrice suivante synthétise la place de chaque acteur dans le système. Elle complète la vision purement fonctionnelle par une lecture centrée sur la responsabilité, indispensable dans une application gouvernée par les droits d'accès.

Acteur	Opérations principales	Données manipulées	Niveau de contrôle
Candidat	Postuler, vérifier son email, passer un quiz, suivre sa candidature, répondre à un entretien.	Son propre dossier, ses réponses, ses pièces jointes, ses statuts publics.	Contrôle limité à son parcours personnel.
Recruteur	Créer une campagne, suivre les candidatures, corriger un quiz, planifier	Campagnes assignées, candidatures, entretiens, évaluations,	Contrôle opérationnel contextualisé par

	un entretien, évaluer, décider.	décisions, notifications.	campagne et par entité.
Administrateur	Gérer les utilisateurs, les référentiels, les permissions et les paramètres d'exploitation.	Utilisateurs, groupes métier, référentiels, modèles d'email, paramètres fonctionnels.	Contrôle d'administration métier.
Super administrateur	Superviser la plateforme, les réglages globaux et la gouvernance de sécurité.	Configuration globale, paramètres techniques, rôles système, supervision transverse.	Contrôle maximal et structurant.

La responsabilité générale de chaque acteur doit cependant être complétée par une lecture plus opérationnelle des droits. La matrice suivante synthétise les permissions nominales, avant prise en compte des droits contextualisés sur campagne ou sur entité.

Fonctionnalité	Super	Admin	Recruteur	Candidat
Connexion et tableau de bord interne	Oui	Oui	Oui	Non
Créer ou modifier un utilisateur	Oui	Oui	Non	Non
Gérer départements, spécialités, questions et réponses	Oui	Partiel	Non	Non
Créer et configurer une campagne	Oui	Oui	Oui	Non
Gérer les formulaires et leurs versions	Oui	Oui	Oui	Non
Consulter et traiter les candidatures	Oui	Oui	Oui	Non
Planifier un entretien	Oui	Oui	Oui	Non
Évaluer et décider	Oui	Oui	Oui	Non

Soumettre une candidature et passer le quiz	Non	Non	Non	Oui
Suivre sa propre candidature	Non	Non	Non	Oui

3.3 Besoins non fonctionnels

Les besoins non fonctionnels jouent un rôle majeur dans la réussite du projet. En matière de **performance**, l'application doit offrir des temps de réponse compatibles avec un usage fluide sur les opérations courantes, en particulier sur les listes de candidatures, les écrans de détail et les parcours publics. La **sécurité** exige une séparation stricte entre surface publique et surface interne, une authentification robuste des utilisateurs internes et un contrôle d'accès fin sur chaque action sensible. Sur ce plan, le projet s'appuie sur les recommandations du référentiel **OWASP Top 10** (OWASP Foundation, 2025) pour la protection contre les vulnérabilités web courantes et sur les mécanismes de sécurité de Spring Security décrits par Laurențiu Spilca dans **Spring Security in Action** (2e édition, 2024).

L'**ergonomie** constitue un autre besoin important, car le système s'adresse à des profils différents et doit rester compréhensible malgré la richesse du domaine métier. Les critères ergonomiques de J. C. Bastien et D. L. Scapin, dans leur rapport **Ergonomical criteria for the evaluation of user interfaces** (1993), — guidage, charge de travail, contrôle explicite, adaptabilité, gestion des erreurs, homogénéité, signifiance des codes et compatibilité — fournissent une grille de lecture pertinente pour évaluer la qualité d'une interface multi-profils comme celle de SyneliaQuizApp. La **maintenabilité** impose une architecture claire, un découpage modulaire, des conventions stables d'API et une capacité à faire évoluer les règles métier sans dégrader l'ensemble. La **disponibilité**, la **fiabilité** et la **conformité** complètent cet ensemble, dans un contexte où les données de candidature doivent rester exploitables, auditables et protégées.

ID	Axe	Exigence	Critère d'acceptation
BNF-01	Performance	Les listes, écrans de détail et parcours candidat doivent rester fluides en usage nominal.	Navigation sans attente perceptible sur les parcours courants et pagination exploitable sur les volumes métier.

BNF-02	Sécurité	Séparer rigoureusement les surfaces publique et interne, puis contrôler chaque action sensible.	Authentification fiable, permissions vérifiées côté backend et absence d'exposition non autorisée de données RH.
BNF-03	Fiabilité	Garantir la cohérence des états de campagne, candidature, quiz, entretien et décision.	Transitions d'état contrôlées, validations métier explicites et historique cohérent des opérations.
BNF-04	Ergonomie	Rendre l'application compréhensible pour des profils très différents.	Écrans lisibles, vocabulaire métier homogène, parcours publics courts et tableaux de bord actionnables.
BNF-05	Maintenabilité	Permettre l'évolution des règles métier sans réécriture globale.	Architecture en couches, modules identifiables, contrats d'API stables et migrations versionnées.
BNF-06	Traçabilité	Pouvoir justifier une décision, une correction ou une évolution d'état.	Historique exploitable des candidatures, scores, évaluations, décisions et actions d'administration.
BNF-07	Conformité	Encadrer la manipulation des données personnelles et leur conservation.	Données strictement nécessaires, droits d'accès limités, justification des décisions sensibles et audit possible.

3.4 Diagrammes des cas d'utilisation

Les diagrammes de cas d'utilisation permettent de représenter les interactions entre les acteurs et le système en se concentrant sur la valeur métier produite. Dans ce mémoire, ils

servent à montrer la frontière entre la surface publique candidate et la surface interne de pilotage, ainsi que la spécialisation progressive des usages entre recruteur, administrateur et super administrateur.

3.4.1 Diagramme global

Le diagramme global doit faire apparaître quatre acteurs principaux et les macro-fonctions suivantes : gestion des utilisateurs, gestion des référentiels, gestion des campagnes, dépôt de candidature, passation de quiz, planification des entretiens, évaluation, décision et pilotage. Il synthétise l'ensemble du système, sans détailler les variantes de permission.

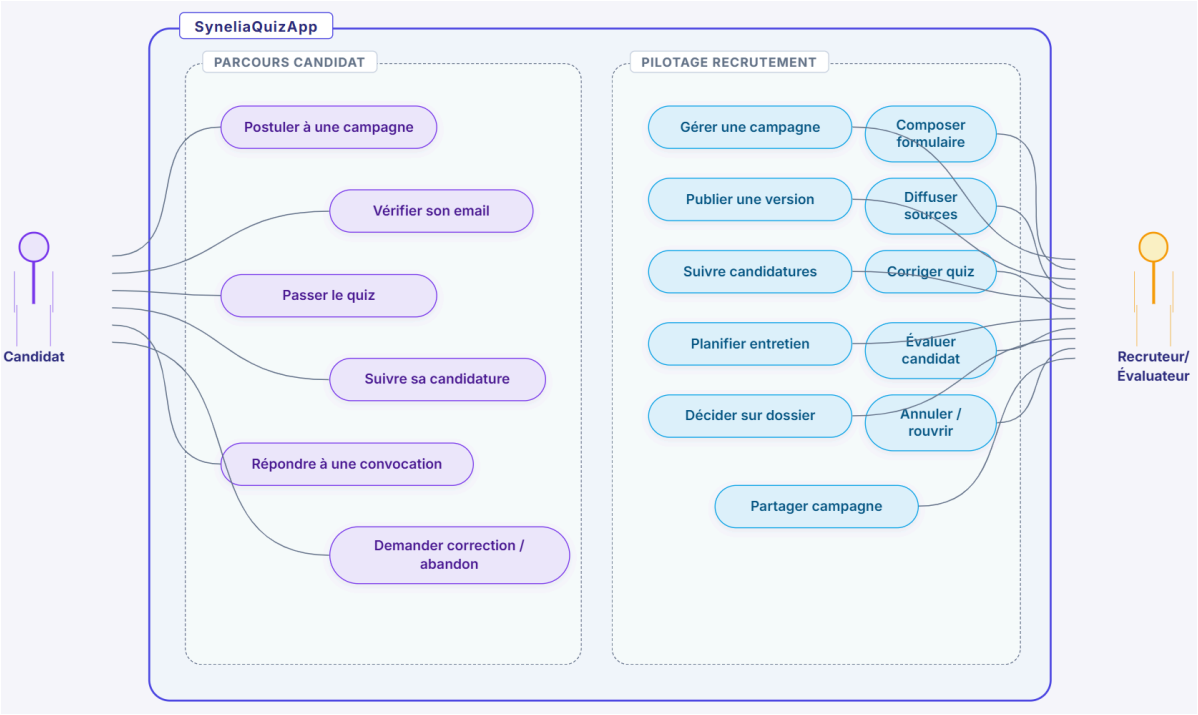


Fig. 2. – Cas d'utilisation — parcours candidat (6 UC).

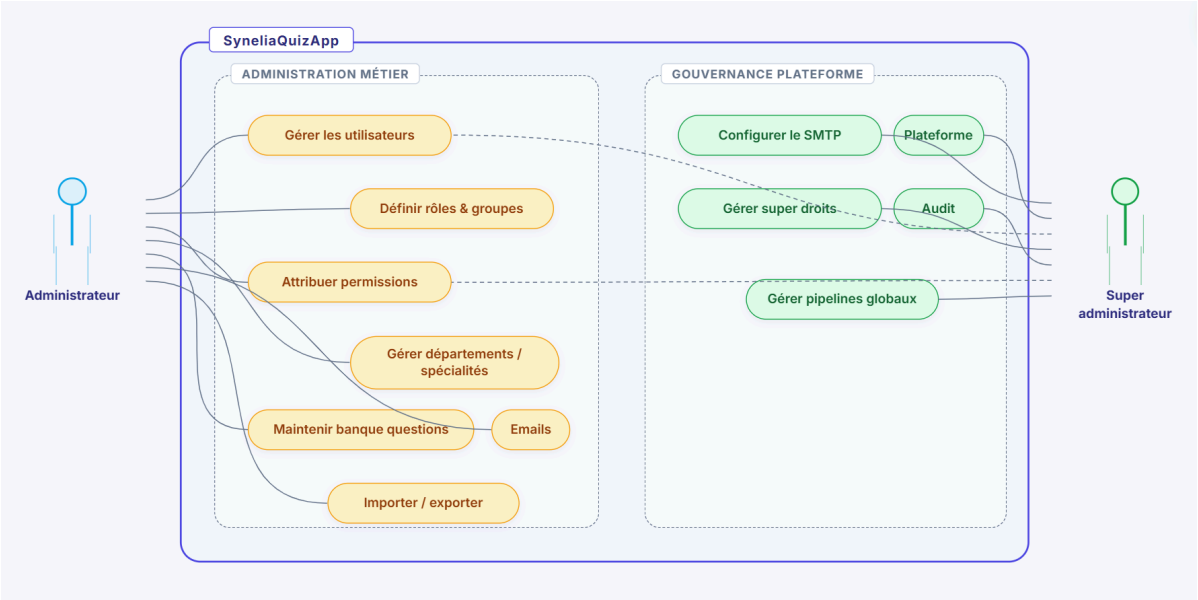


Fig. 3. – Cas d'utilisation — administration métier et gouvernance plateforme (12 UC).

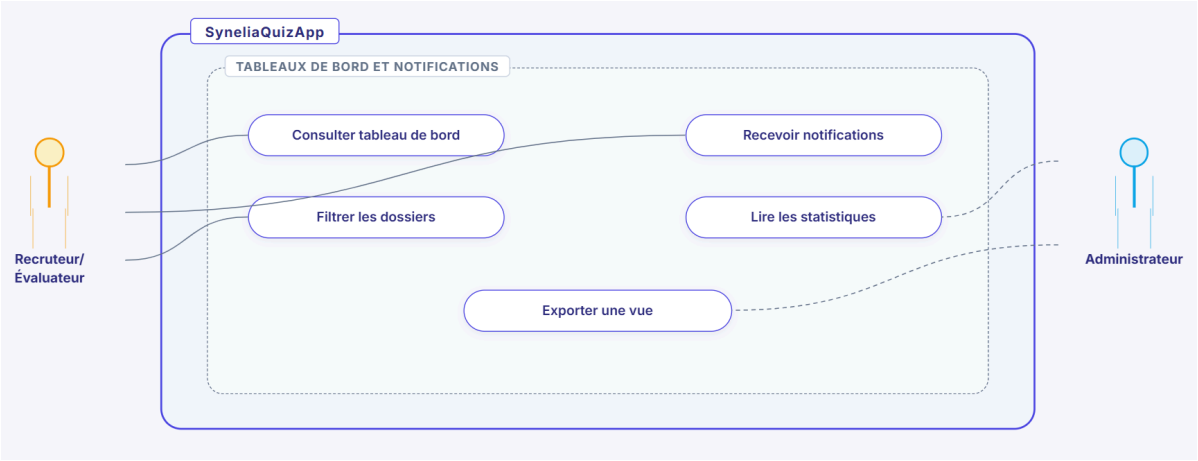


Fig. 4. – Cas d'utilisation — tableaux de bord, notifications et exports (5 UC transverses).

3.4.2 Description textuelle des cas d'utilisation principaux

Afin de compléter le diagramme global, les cas d'utilisation majeurs ont été décrits textuellement. Cette description fait le lien entre l'attente utilisateur, le déclencheur métier et le résultat attendu par l'organisation.

ID	Acteur	Déclencheur	Résultat attendu
UC-01	Candidat	Accès à une campagne via un lien public ou un token de source.	Le système affiche l'offre, autorise la saisie du dossier et crée une candidature exploitable par les recruteurs.

UC-02	Candidat	Validation d'email puis passation du quiz technique.	Le système sécurise l'accès, enregistre les réponses, calcule l'état du quiz et rend le dossier évaluable.
UC-03	Recruteur	Création ou mise à jour d'une campagne de recrutement.	La campagne est configurée avec un pipeline, un formulaire, des permissions et des informations de diffusion cohérentes.
UC-04	Recruteur	Planification d'un entretien à partir d'une candidature.	Une convocation est produite, le candidat peut répondre et l'historique du pipeline est mis à jour.
UC-05	Évaluateur	Saisie d'une appréciation après entretien.	L'évaluation est rattachée au bon entretien, agrégée et rendue visible au décideur légitime.
UC-06	Administrateur	Gestion des utilisateurs, rôles et référentiels.	Le système maintient une base de travail propre, gouvernée et sécurisée pour l'ensemble de l'organisation.
UC-07	Super administrateur	Pilotage des paramètres globaux et supervision de la sécurité.	La plateforme reste configurable, stable et auditée sur l'ensemble de ses usages.

3.4.3 Lecture par acteur

Dans la version finale du mémoire, le diagramme global pourra être complété par trois figures ciblées : un diagramme orienté administration, un diagramme orienté recruteur et un diagramme orienté candidat. Cette décomposition améliore la lisibilité tout en conservant la cohérence d'ensemble du modèle d'usage.

3.5 Priorisation des besoins

L'ensemble des besoins identifiés ne possède pas la même criticité. Une priorisation de type MoSCoW a donc été retenue afin de distinguer les fonctionnalités indispensables à la valeur métier immédiate, celles qui renforcent l'efficacité opérationnelle et celles qui relèvent d'une extension ultérieure. Cette hiérarchisation est essentielle dans un projet

de recrutement, car le cœur de valeur se situe avant tout dans la collecte fiable des candidatures, l'évaluation et la prise de décision.

Priorité	Besoins concernés	Justification
Must have	Soumettre une candidature ; gérer les campagnes ; consulter les candidatures ; s'authentifier ; passer le quiz ; évaluer les candidats.	Sans ces fonctions, le système ne remplit pas sa mission principale de recrutement.
Should have	Planifier des entretiens ; vérifier l'email ; suivre sa candidature ; recevoir des notifications ; prendre une décision finale structurée.	Ces fonctions complètent le workflow métier et sécurisent la qualité du processus.
Must have	Tableaux de bord avancés ; export de données ; raffinements de diffusion et de reporting.	Ces fonctions renforcent le pilotage opérationnel et la visibilité du processus de recrutement, conditions essentielles pour une exploitation professionnelle de la plateforme.
Won't have	Intégration calendrier poussée, matching avancé, automatisations non critiques hors périmètre immédiat.	Ces éléments relèvent d'extensions futures et ne doivent pas brouiller le périmètre du noyau métier.

3.6 Règles de gestion

Les règles de gestion constituent le noyau normatif du système. Elles expriment les contraintes qui ne peuvent pas être laissées à la seule interface et qui doivent être garanties par le modèle métier et par le backend. Leur formalisation est indispensable pour justifier ensuite les choix Merise et UML du chapitre suivant.

ID	Domaine	Règle	Effet sur le système
RG-01	Campagnes	Une campagne ne peut être ouverte que si son cadre métier est cohérent : période valide, pipeline exploitable et diffusion contrôlée.	Le backend doit bloquer les activations incohérentes et

			garantir l'intégrité du paramétrage.
RG-02	Accès public	L'accès candidat dépend de la validité du lien de diffusion, de l'état de la campagne et des limites métiers imposées.	Les points d'entrée publics doivent vérifier le contexte avant toute création de dossier.
RG-03	Vérification email	Certaines actions publiques ne sont autorisées qu'après validation de l'adresse email du candidat.	La vérification devient une précondition métier et non un simple confort d'interface.
RG-04	Versionning formulaire	Une version de formulaire ayant servi à une candidature ne doit plus être altérée de manière destructive.	Le verrouillage et la publication des versions protègent la comparabilité des réponses.
RG-05	Pipeline	application_progress constitue la source de vérité de l'avancement d'une candidature.	Les écrans de synthèse doivent dériver de l'historique métier plutôt que d'un seul statut simplifié.
RG-06	Entretiens et décisions	Un entretien, une évaluation ou une décision doivent être rattachés à la bonne candidature et à l'étape pertinente du pipeline.	Le système conserve un historique lisible des événements qui ont conduit au verdict final.
RG-07	Permissions	Une action ne dépend pas seulement d'un rôle global ; elle peut aussi dépendre d'un droit contextualisé sur une campagne ou une entité.	Les contrôles d'accès doivent être réalisés côté backend en tenant compte du contexte métier.

RG-08	Correction quiz	Toute correction manuelle d'une réponse doit rester identifiable, datée et attribuable.	La traçabilité de l'évaluation technique est préservée et révisable.
RG-09	Audit	Les décisions critiques et certaines modifications doivent pouvoir être justifiées a posteriori.	Le modèle doit conserver suffisamment d'informations pour soutenir la traçabilité et la conformité.

CHAPITRE 4

Conception de la solution

Ce chapitre traduit les besoins en architecture, structures de données, flux applicatifs et conventions d'implémentation. Il expose la cible conceptuelle du système tout en l'adossant au schéma réel et aux modules effectivement identifiés dans le projet.

4.1 Architecture globale du système

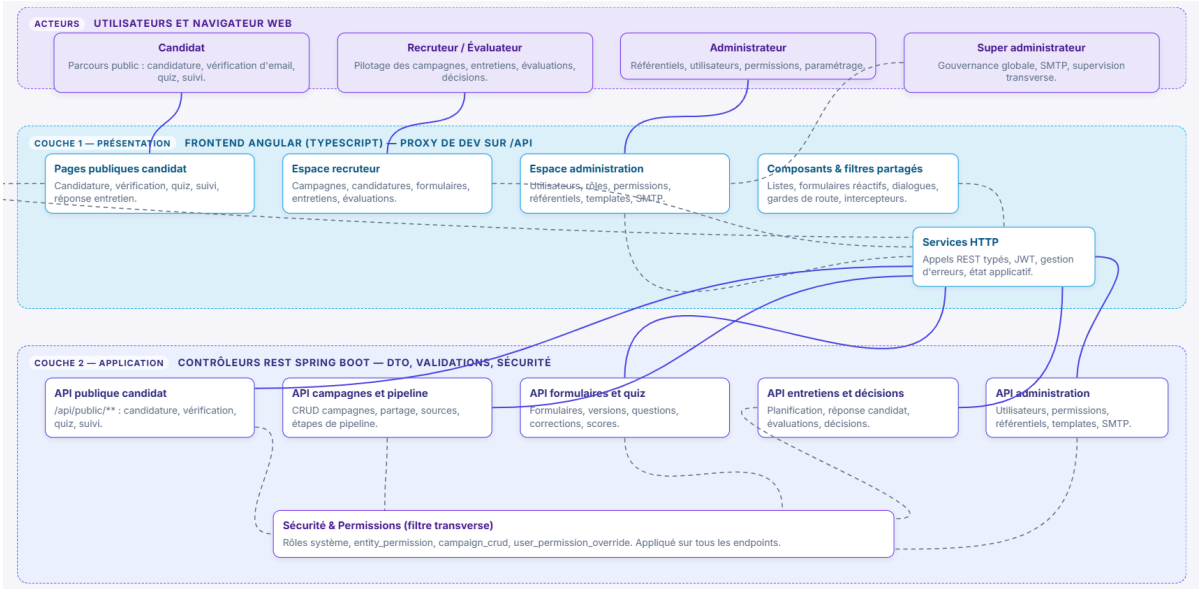


Fig. 5. – Architecture 1/2 — acteurs, frontend Angular et API REST Spring Boot.

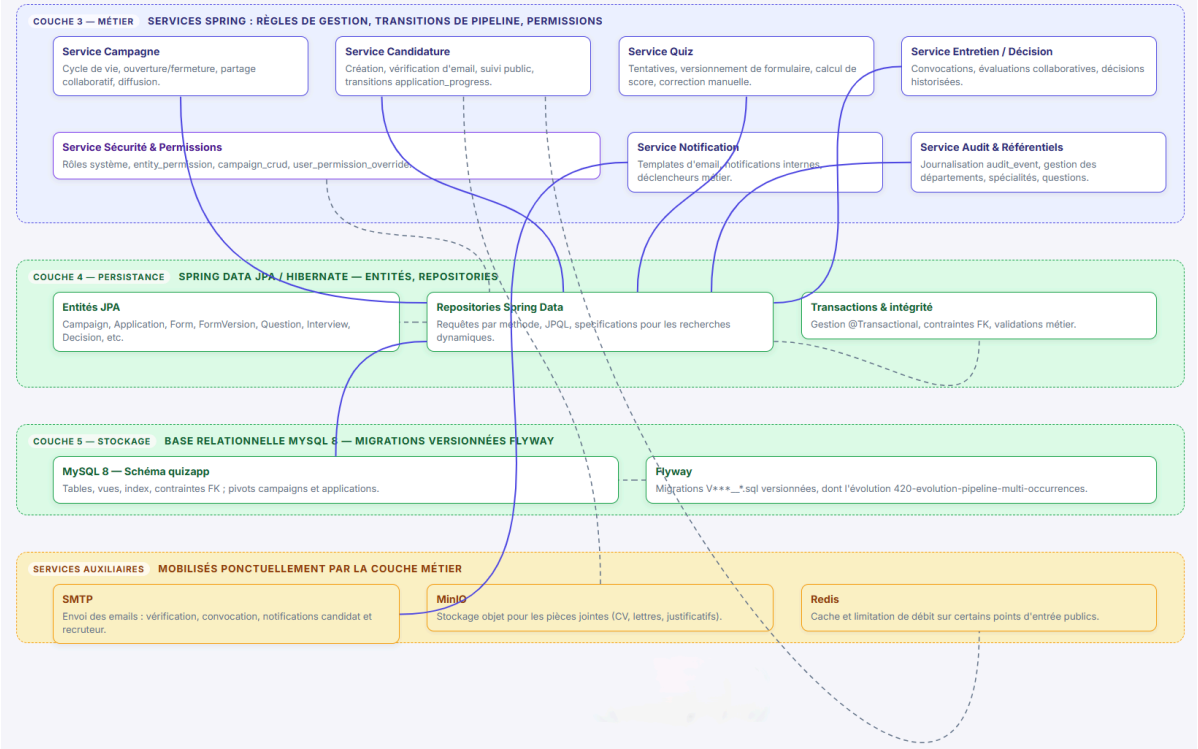


Fig. 6. – Architecture 2/2 — services métier, persistance JPA/MySQL et services auxiliaires (SMTP, MinIO, Redis).

La solution adopte une architecture web en couches, fondée sur une séparation nette entre l'interface utilisateur, les services métier et la persistance. Le frontend Angular constitue la couche de présentation. Il prend en charge les pages publiques candidat, les tableaux de bord internes, les composants partagés, les services d'appel API et la logique d'interaction propre à chaque profil utilisateur. Le backend Spring Boot joue le rôle de couche applicative centrale : il expose les endpoints, applique les règles métier, contrôle les permissions, orchestre les transitions de pipeline et dialogue avec la base de données.

La persistance repose sur une base relationnelle MySQL enrichie par des migrations Flyway, ce qui permet de faire évoluer le schéma de manière versionnée. Autour de ce noyau, des services auxiliaires comme Redis, MinIO ou les composants d'envoi d'emails complètent l'architecture physique lorsque le scénario métier l'exige. Cette organisation en couches, où chaque niveau possède une responsabilité clairement délimitée, s'inspire des principes de Clean Architecture formalisés par Robert C. Martin dans **Clean Architecture** (2017) : isolation du domaine métier, indépendance des interfaces et testabilité des composants. L'ensemble forme une architecture distribuée simple, mais extensible, adaptée à une application métier de taille intermédiaire.

Sur le plan logique, le backend est lui-même organisé en sous-couches lisibles : exposition web, services métier, accès aux données, sécurité, configuration et utilitaires. Cette organisation limite le couplage direct entre l'interface et la persistance. Elle permet

surtout de centraliser les règles métier dans des services spécialisés, ce qui est indispensable dans un domaine où les transitions d'état, les permissions, les agrégats et les effets de bord doivent rester cohérents dans tout le système.

4.1.1 Vue logique en couches

La vue logique suivante résume la responsabilité de chaque couche et son rôle dans la maîtrise du processus de recrutement.

Couche	Composants principaux	Responsabilité dans la solution
Présentation	Pages Angular, composants partagés, formulaires, services HTTP.	Gérer les interactions utilisateur, afficher l'état métier et transmettre les commandes au backend.
Application	Contrôleurs REST, DTO, sécurité, orchestration métier Spring Boot.	Recevoir les demandes, valider les droits, appliquer les règles de gestion et coordonner les traitements.
Métier	Services de campagne, candidature, quiz, entretien, évaluation, décision, permissions.	Centraliser les règles métier, les transitions d'état et les effets transverses.
Persistance	Entités JPA, repositories, MySQL, migrations Flyway.	Conserver les données, garantir l'intégrité relationnelle et versionner le schéma.
Services auxiliaires	SMTP, MinIO, Redis, notifications.	Supporter les besoins transverses de communication, stockage et exploitation.

4.2 Conception de la base de données

La conception de la base de données suit une logique relationnelle riche, cohérente avec la nature fortement structurée du domaine. Les entités principales s'organisent autour de quelques noyaux : utilisateurs et permissions, campagnes et pipelines, formulaires et questions, candidatures et parcours candidat, entretiens et évaluations, décisions et notifications. Chaque noyau possède ses propres relations, mais l'ensemble converge vers la candidature comme objet pivot du recrutement.

4.2.1 Démarche Merise retenue

Le choix d'une méthode de conception ne peut être implicite dans un mémoire d'ingénieur. Deux approches dominent aujourd'hui la modélisation des systèmes d'information : la méthode **Merise**, structurée autour d'une séparation explicite des données et des traitements à plusieurs niveaux d'abstraction, et le langage **UML**, orienté objet et particulièrement adapté à la modélisation comportementale des logiciels. Avant d'en retenir une comme colonne vertébrale de la conception, ces deux approches ont été comparées au regard du domaine et des objectifs du projet.

Critère	Merise	UML
Nature de la méthode	Méthode systémique française orientée systèmes d'information.	Langage de modélisation orienté objet, indépendant d'une méthode.
Découpage proposé	Séparation explicite données / traitements et niveaux conceptuel, logique et physique.	Ensemble de diagrammes statiques et dynamiques sans démarche imposée.
Adéquation aux domaines riches en données	Très forte : MCD, MLD et MPD donnent un cadre rigoureux à la modélisation relationnelle.	Possible via les diagrammes de classes, mais sans la rigueur des cardinalités et des dépendances fonctionnelles.
Adéquation aux comportements dynamiques	Couverte par MCT et MOT, mais avec un formalisme moins répandu.	Très riche : séquences, activités, états, cas d'utilisation.
Maîtrise pédagogique	Maîtrisée dans le cadre de la formation et largement enseignée en école d'ingénieur en Côte d'Ivoire.	Maîtrisée également, mais avec un usage plus dispersé selon les diagrammes.
Compatibilité avec le projet	Excellente : permet de structurer l'analyse autour de la base, qui est ici le pivot du système.	Pertinente en complément pour décrire certains flux applicatifs.

À l'issue de cette comparaison, la méthode **Merise** a été retenue comme méthode principale de conception, conformément aux principes fondateurs posés par H. Tardieu, A. Rochfeld et R. Colletti dans **La méthode Merise : Principes et outils** (2003), aux guides pratiques de J. Gabay dans **Merise et UML : Guide pratique** (2008) et aux outils conceptuels et organisationnels détaillés par J. P. Matheron dans **Comprendre Merise** (2005).

Trois raisons justifient ce choix. D'abord, le domaine de SyneliaQuizApp est fortement orienté données : campagnes, pipelines, formulaires versionnés, candidatures, étapes, entretiens, évaluations et décisions s'organisent par des relations stables, qui se prêtent naturellement à une modélisation MCD/MLD/MPD. Ensuite, la séparation explicite entre données et traitements (MCD/MCT) permet d'aligner la documentation du mémoire sur le raisonnement réel suivi durant le projet : analyse du modèle de données d'abord, puis raisonnement sur les flux. Enfin, la chaîne Merise produit des artefacts visibles et hiérarchisés qui renforcent la lisibilité académique du chapitre de conception.

UML n'est pas pour autant écarté. Il est positionné comme un *complément* mobilisé ponctuellement pour illustrer certains comportements dynamiques que Merise exprime moins finement, notamment les diagrammes de cas d'utilisation pour cartographier les acteurs et les diagrammes de séquence pour expliciter les échanges techniques entre frontend, backend et services auxiliaires. Cette articulation Merise-first / UML-complémentaire permet de cumuler les forces des deux approches sans diluer la cohérence méthodologique du mémoire.

L'usage de Merise est donc particulièrement pertinent ici, car le domaine métier est fortement orienté données : une campagne comporte un pipeline, un formulaire possède plusieurs versions, une candidature progresse dans un pipeline, un entretien donne lieu à des évaluations, puis à une décision. L'enjeu n'est donc pas seulement de stocker des informations, mais de faire apparaître un système cohérent de dépendances et de contraintes.

4.2.2 Modèle Conceptuel de Données (MCD)

Au niveau conceptuel, le système peut être lu comme un ensemble d'entités métier reliées par des associations stables. La candidature occupe une place centrale : elle est émise dans le cadre d'une campagne, s'appuie sur une version de formulaire figée, progresse dans un pipeline, reçoit éventuellement des entretiens, des évaluations et des décisions, et peut générer des notifications ou des demandes spécifiques.

Les figures suivantes présentent le modèle conceptuel de données, décomposé en cinq sous-ensembles pour en faciliter la lecture.

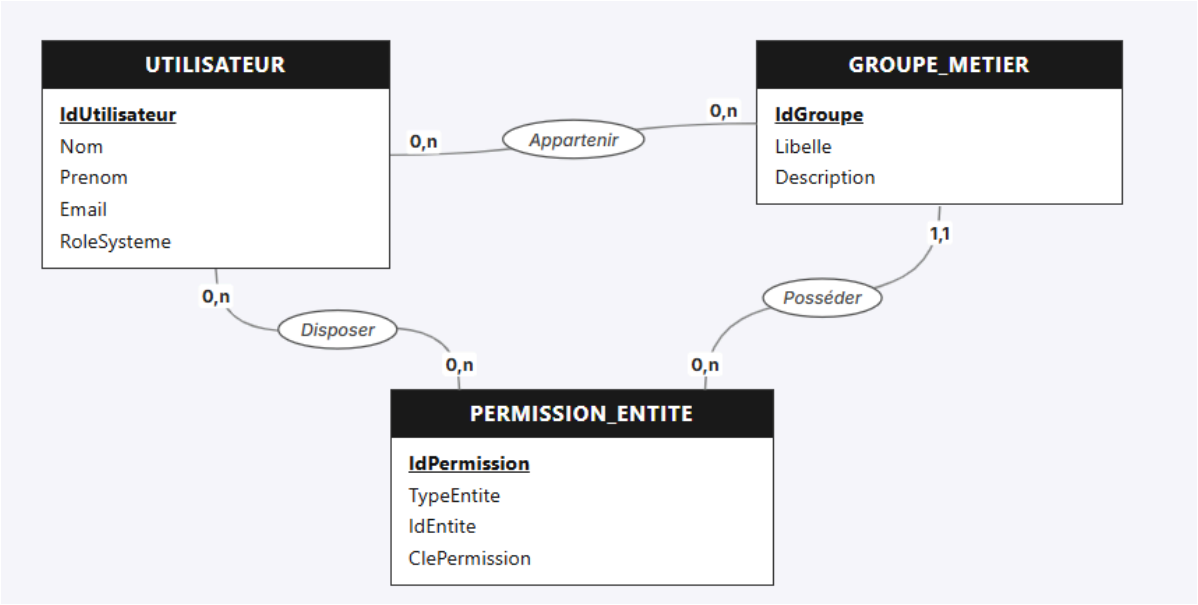


Fig. 7. – MCD — Identité, groupes & permissions.

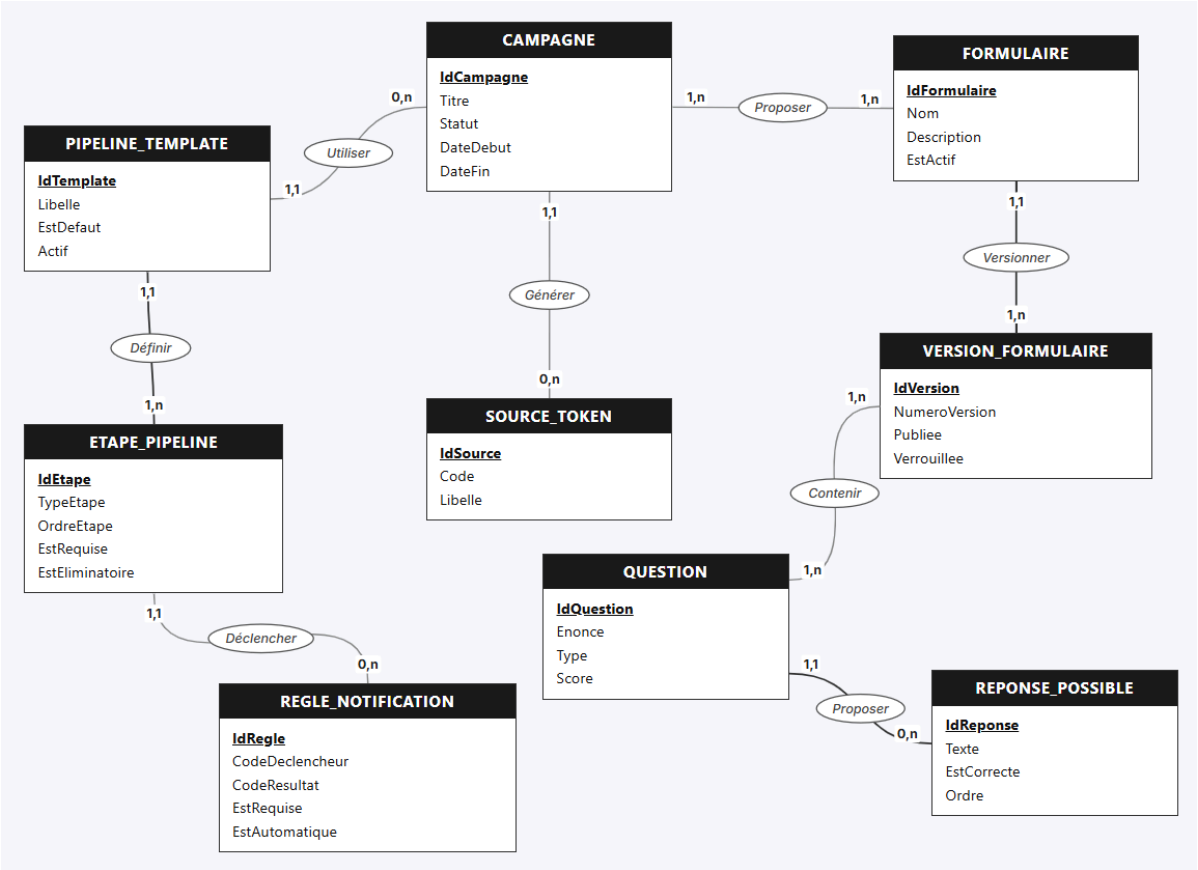


Fig. 8. – MCD — Campagnes, pipeline & formulaires.

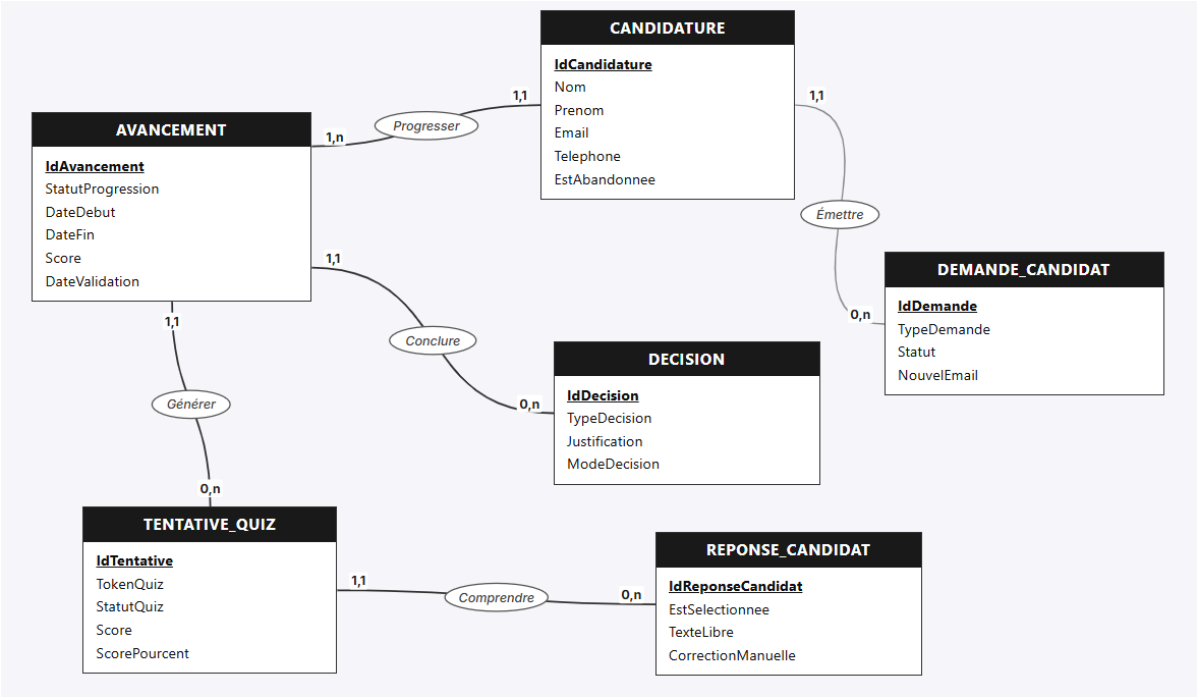


Fig. 9. – MCD — Candidature, avancement & quiz.

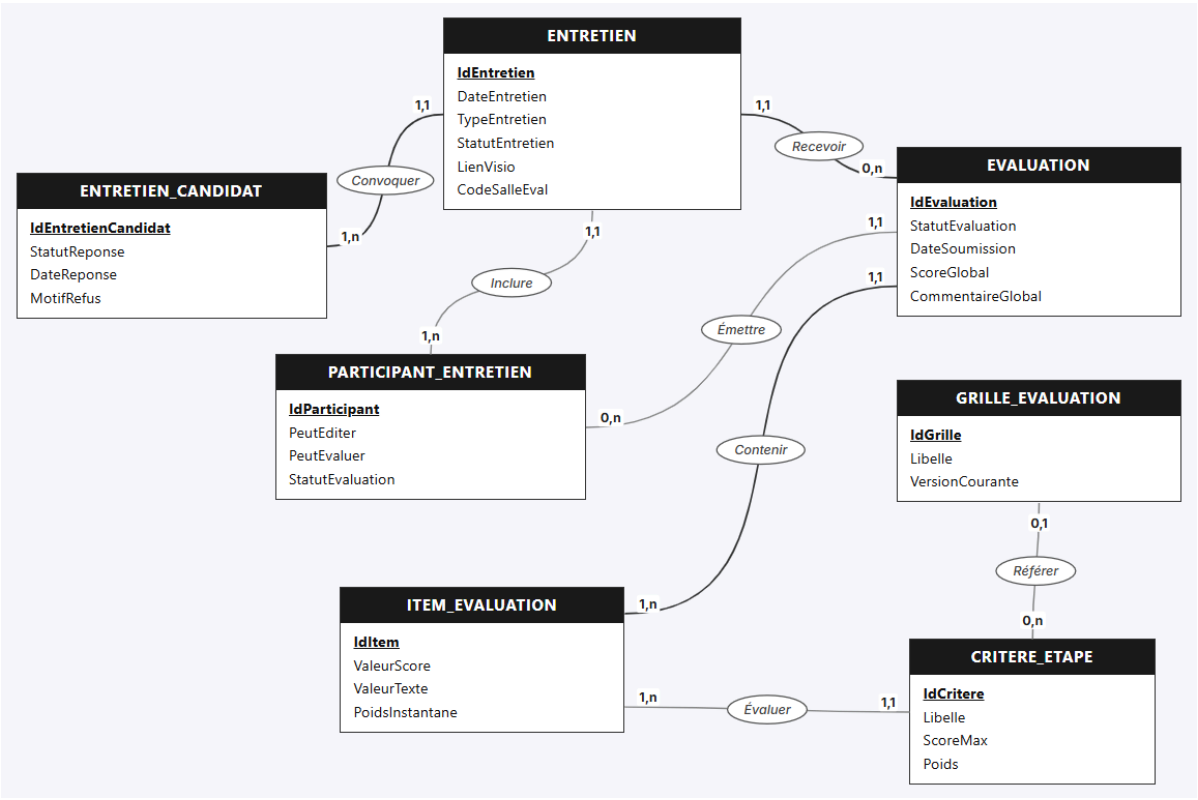


Fig. 10. – MCD — Entretiens & évaluations.

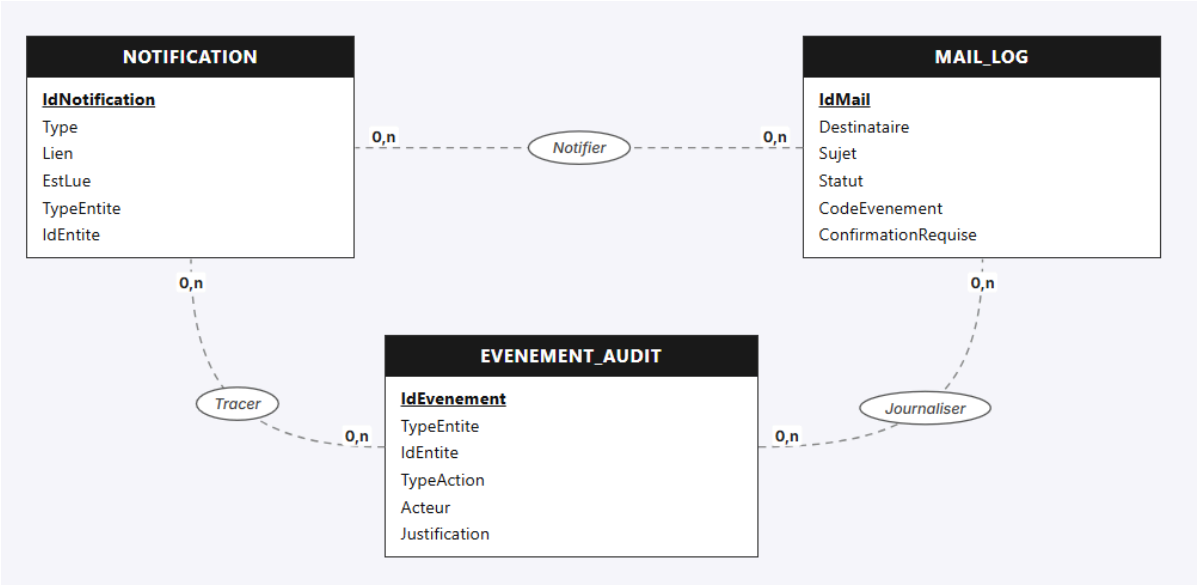


Fig. 11. – MCD — Notifications, mails & audit.

Association conceptuelle	Cardinalité	Interprétation métier
Campagne - Formulaire	1:N	Une campagne peut mobiliser plusieurs formulaires, chacun restant rattaché à une campagne donnée.
Formulaire - VersionFormulaire	1:N	Le contenu d'un formulaire évolue par versions afin de figer les questionnaires.
VersionFormulaire - Question	N:N	Une version assemble plusieurs questions, et une question peut être réutilisée dans plusieurs versions.
PipelineTemplate - Campagne	1:N	Un même pipeline peut être réutilisé par plusieurs campagnes, mais une campagne active n'emploie qu'un pipeline de référence.
SourceToken - Candidature	1:N	Un lien de diffusion peut être à l'origine de plusieurs candidatures et permet de tracer la source d'acquisition.
Candidature - Progression	1:N	Le parcours d'une candidature est détaillé par étapes de progression plutôt que réduit à un seul statut global.

Candidature - RéponseCandidat	1:N	Les réponses du quiz sont rattachées au dossier candidat et donc historisables.
Candidature - EntretienCandidat	1:N	Une candidature peut donner lieu à plusieurs événements d'entretien dans le cadre de son parcours.
EntretienCandidat - Évaluation	1:N	Un entretien peut recevoir plusieurs évaluations, produites par différents intervenants.
Candidature - Décision	1:N	Plusieurs décisions peuvent jalonner le parcours, par exemple une pré-sélection puis une décision finale.
Utilisateur - PermissionEntité	1:N	Les droits fins ne se limitent pas au rôle global ; ils peuvent cibler une entité métier donnée.

4.2.3 Modèle Logique de Données (MLD)

Le modèle logique traduit les associations conceptuelles en relations concrètes, portées par des clés étrangères ou par des tables d'association. Dans SyneliaQuizApp, cette traduction est lisible dans l'inventaire de la base, notamment autour des tables campaigns, form, form_version, form_questions, applications, application_progress, candidate_interview, evaluations, decisions, campaign_crud et entity_permission.

Les figures suivantes présentent le modèle logique de données, décliné selon les mêmes sous-ensembles que le MCD.

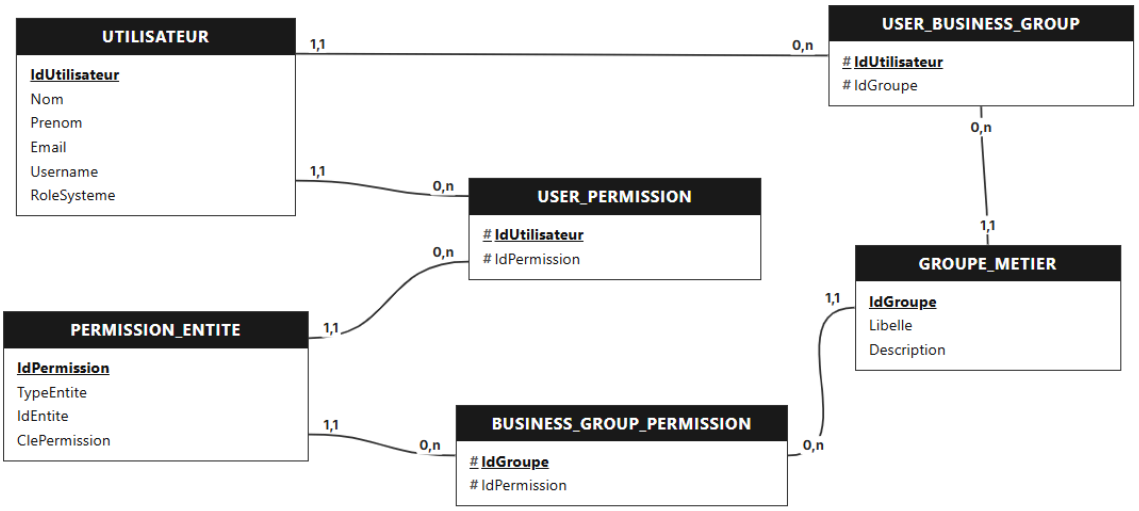


Fig. 12. – MLD — Identité, groupes & permissions.

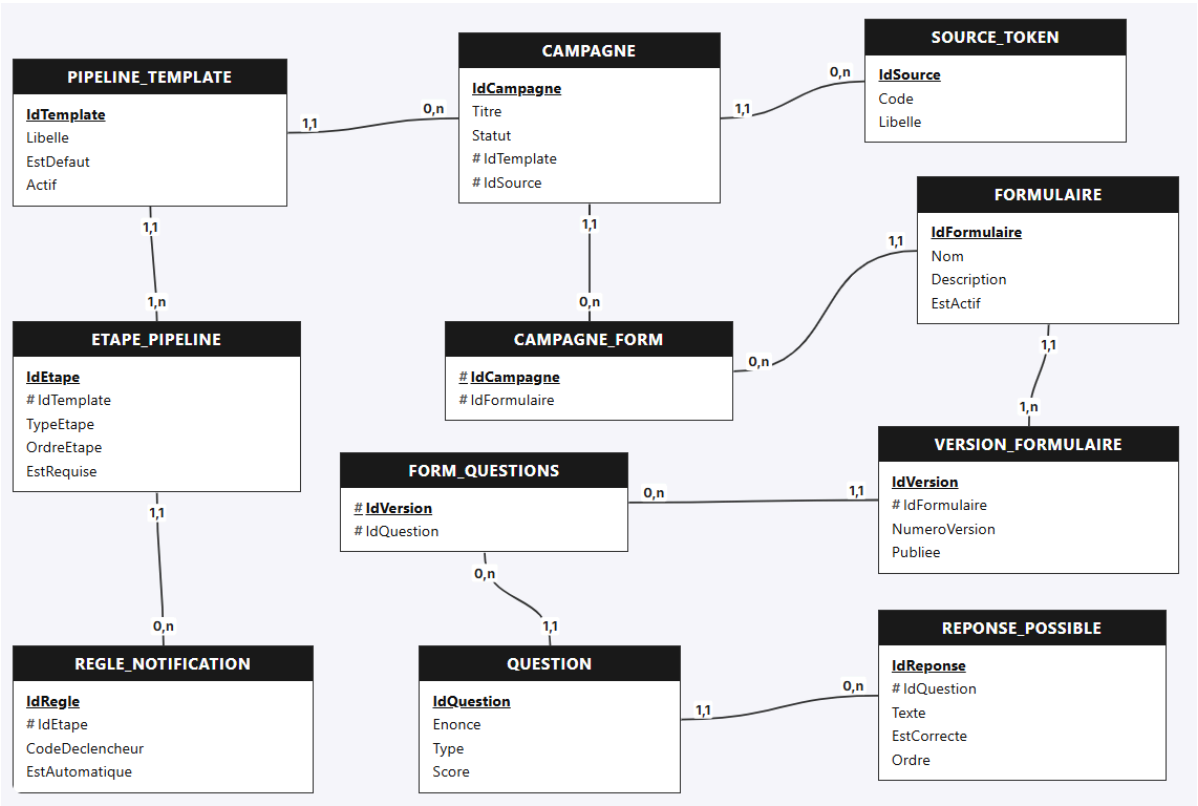


Fig. 13. – MLD — Campagnes, pipeline & formulaires.

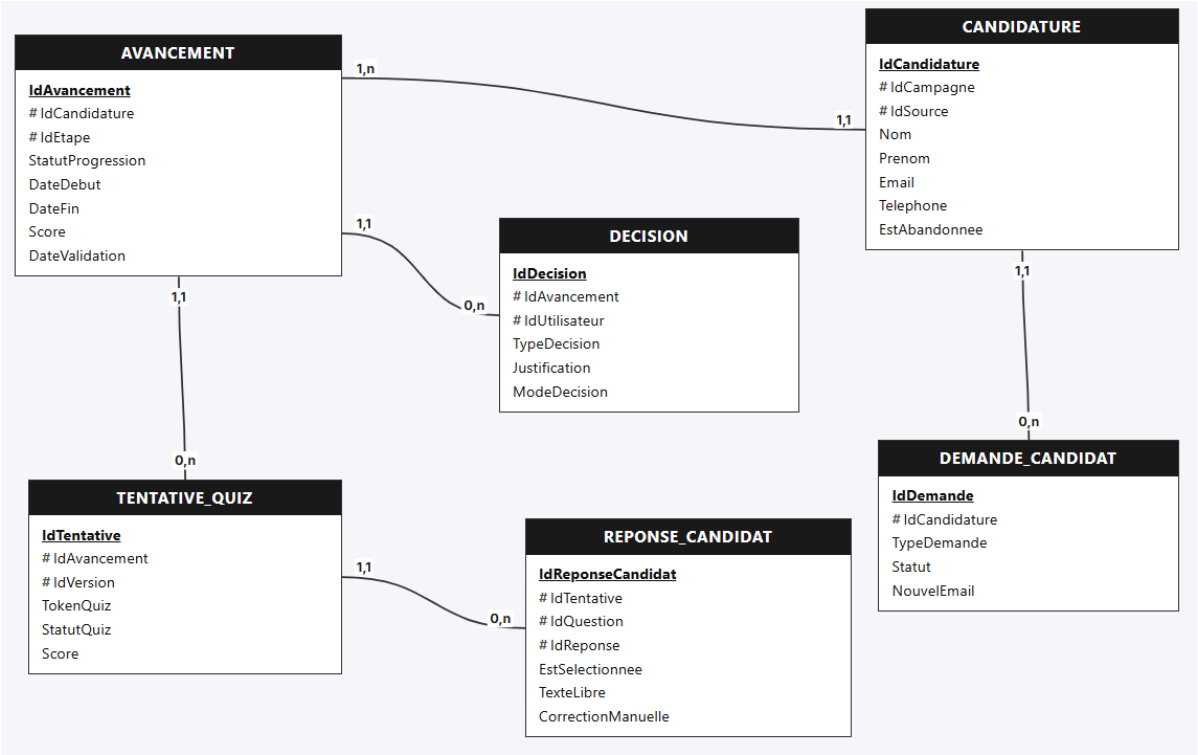


Fig. 14. – MLD — Candidature, avancement & quiz.

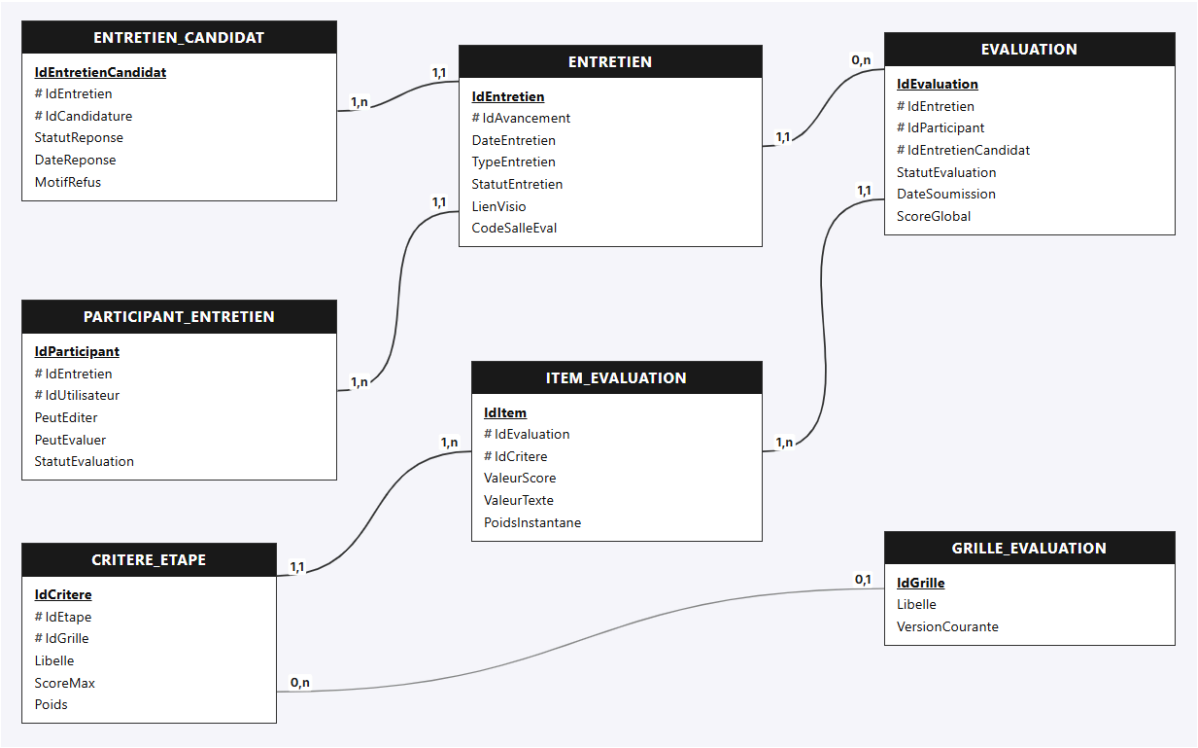


Fig. 15. – MLD — Entretiens & évaluations.

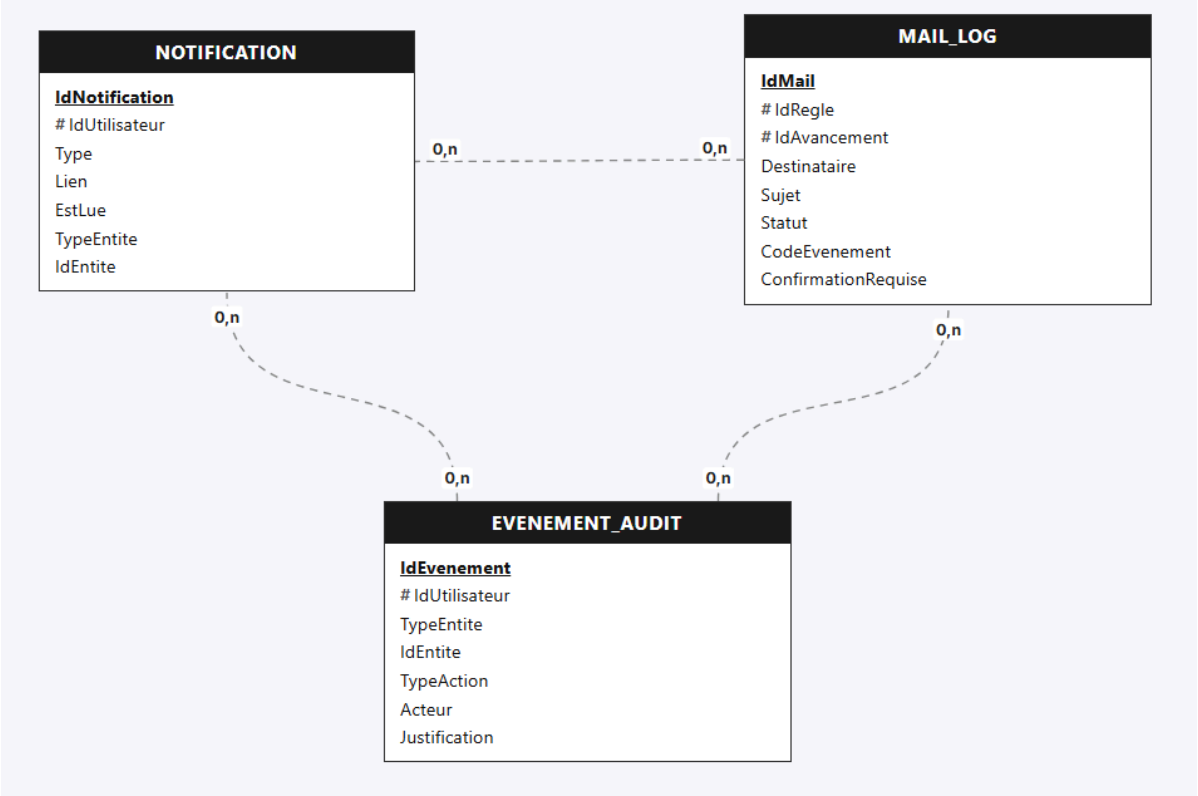


Fig. 16. – MLD — Notifications, mails & audit.

Relation logique	Implémentation	Cardinalité	Finalité métier
Campagne - pipeline	campaigns.pipeline_template_id	N:1	Associer à chaque campagne un pipeline réutilisable et ordonné.
Formulaire - version	form_version.form_id	N:1	Versionner les questionnaires sans perdre l'historique des campagnes déjà ouvertes.
Version - question	form_questions(form_version_id, question_id)	N:N	Composer un quiz, définir l'ordre des questions et éventuellement surcharger le chrono.
Campagne - formulaire	form.campaign_id	N:1	Rattacher les formulaires au contexte de campagne qui les exploite.

Candidature - source	applications.source_token_id	N:1	Tracer l'origine d'une candidature et la diffusion de l'offre.
Candidature - version de formulaire	applications.form_version_id	N:1	Figera la version exacte du questionnaire subie par le candidat.
Candidature - progression	application_progress(application_id, pipeline_stage_id)	1:N	Historiser l'avancement par étape et conserver les scores ou notes liés à cette étape.
Candidature - entretien	candidate_interview(application_id, interview_id)	N:1 + N:1	Relier un candidat à une session d'entretien et conserver sa réponse éventuelle.
Entretien - évaluation	evaluations.candidate_interview_id	N:1	Consolider plusieurs appréciations sur une même occurrence d'entretien.
Candidature - décision	decisions(application_id, pipeline_stage_id, user_id)	N:1	Tracer la nature de la décision, l'étape concernée et l'auteur.
Permissions contextuelles	campaign_crud ; entity_permission ; user_permission_override	Associations	Exprimer des droits fins complémentaires au rôle système.

4.2.4 Modèle Physique de Données (MPD)

Le modèle physique tient compte des choix effectivement retenus pour l'implémentation MySQL. Le schéma repose majoritairement sur des identifiants entiers non signés, des colonnes techniques homogènes d'audit, des indicateurs booléens pour les états simples et des colonnes textuelles ou numériques pour les paramètres métier. Les migrations Flyway assurent la version du schéma et rendent reproductibles les évolutions de structure.

Les figures suivantes présentent le modèle physique de données, reflet direct du schéma MySQL implémenté.

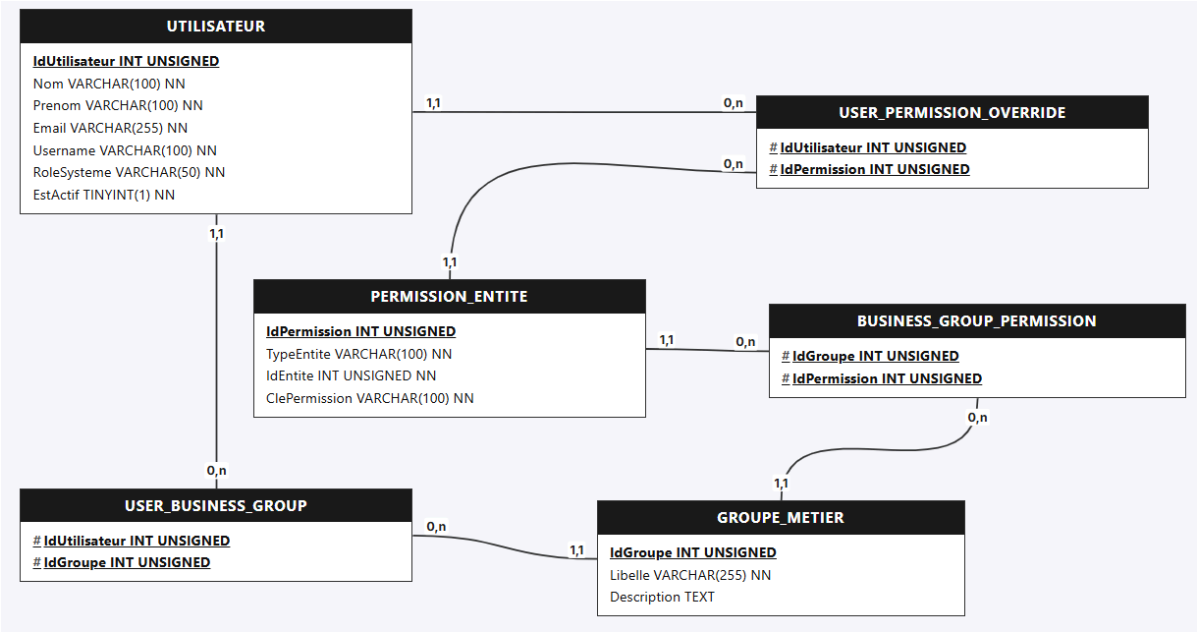


Fig. 17. – MPD — Identité, groupes & permissions.

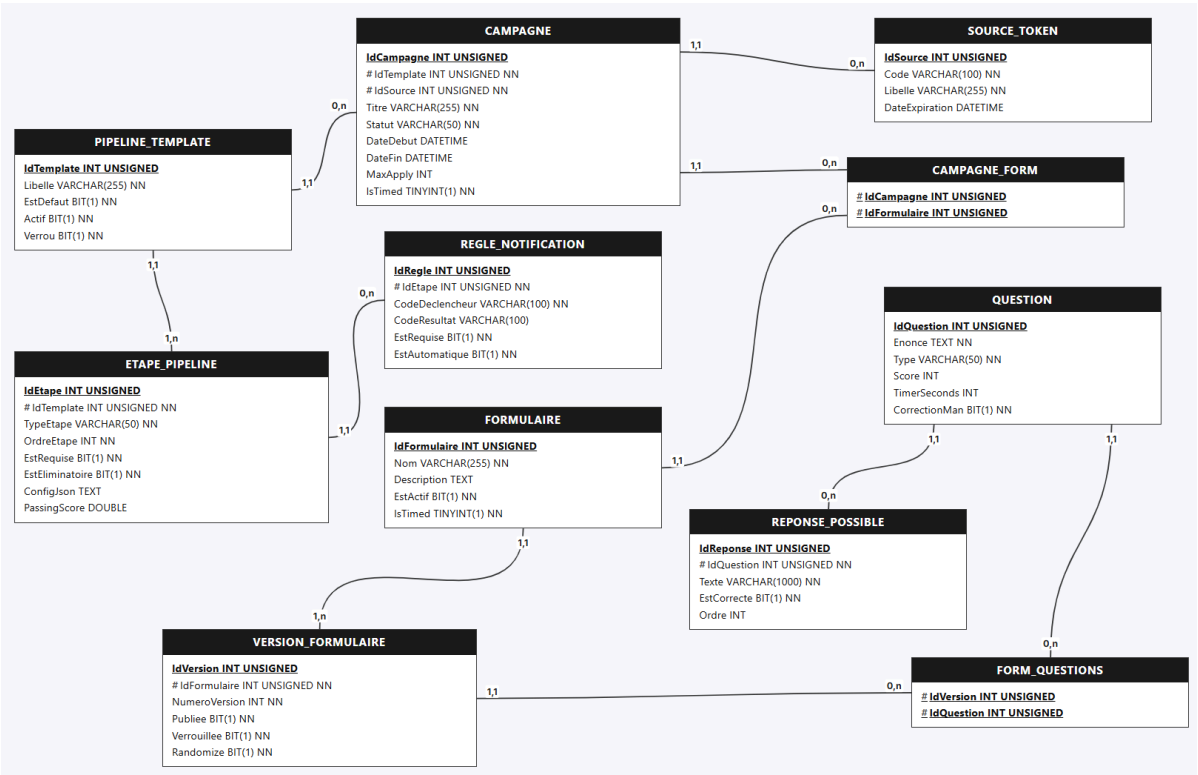


Fig. 18. – MPD — Campagnes, pipeline & formulaires.

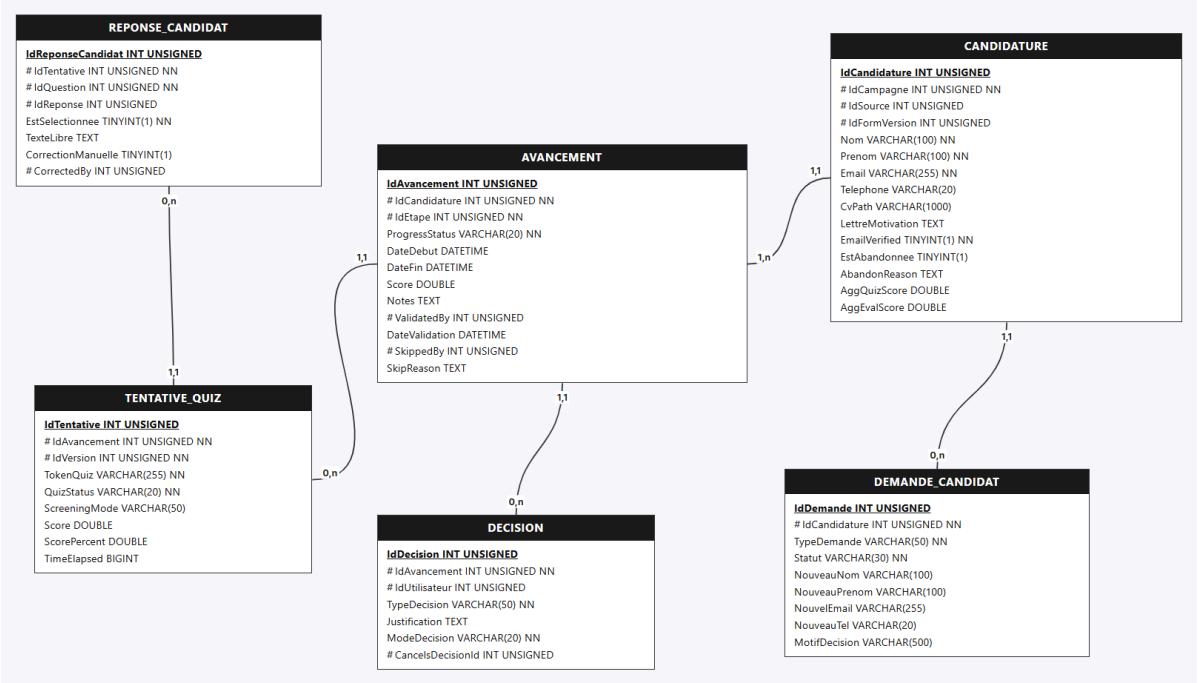


Fig. 19. – MPD — Candidature, avancement & quiz.

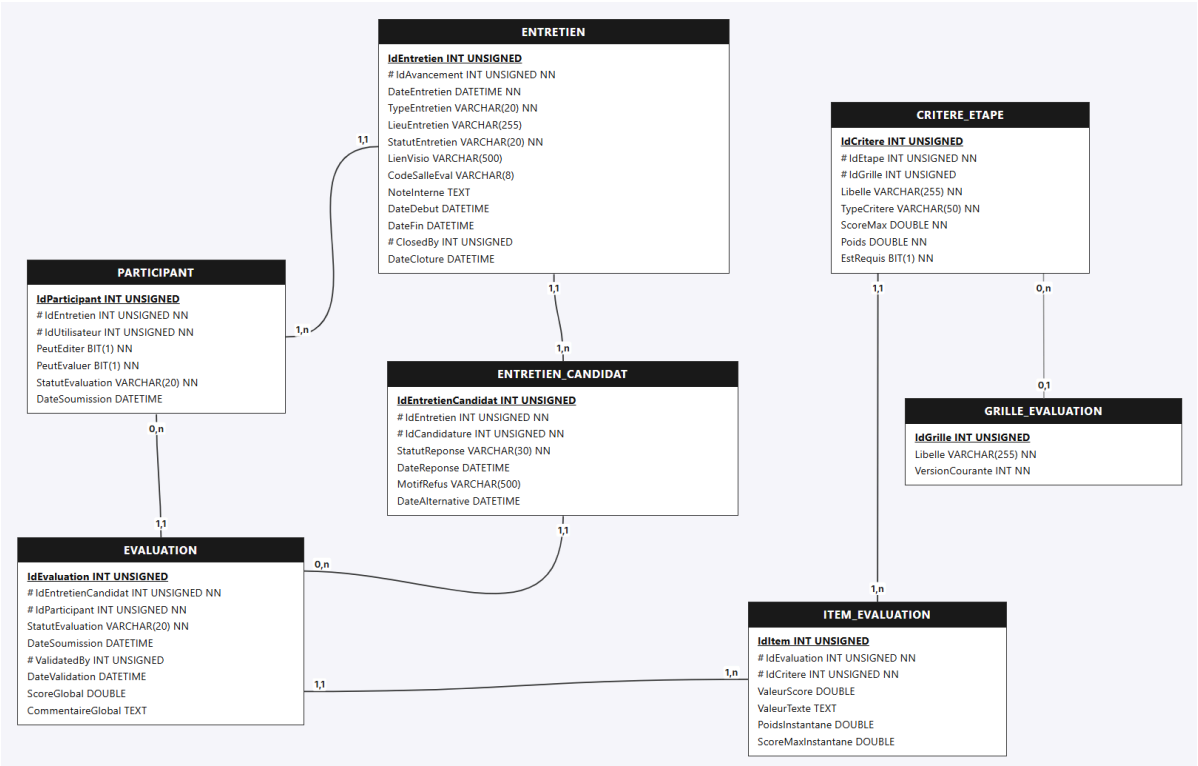


Fig. 20. – MPD — Entretiens & évaluations.

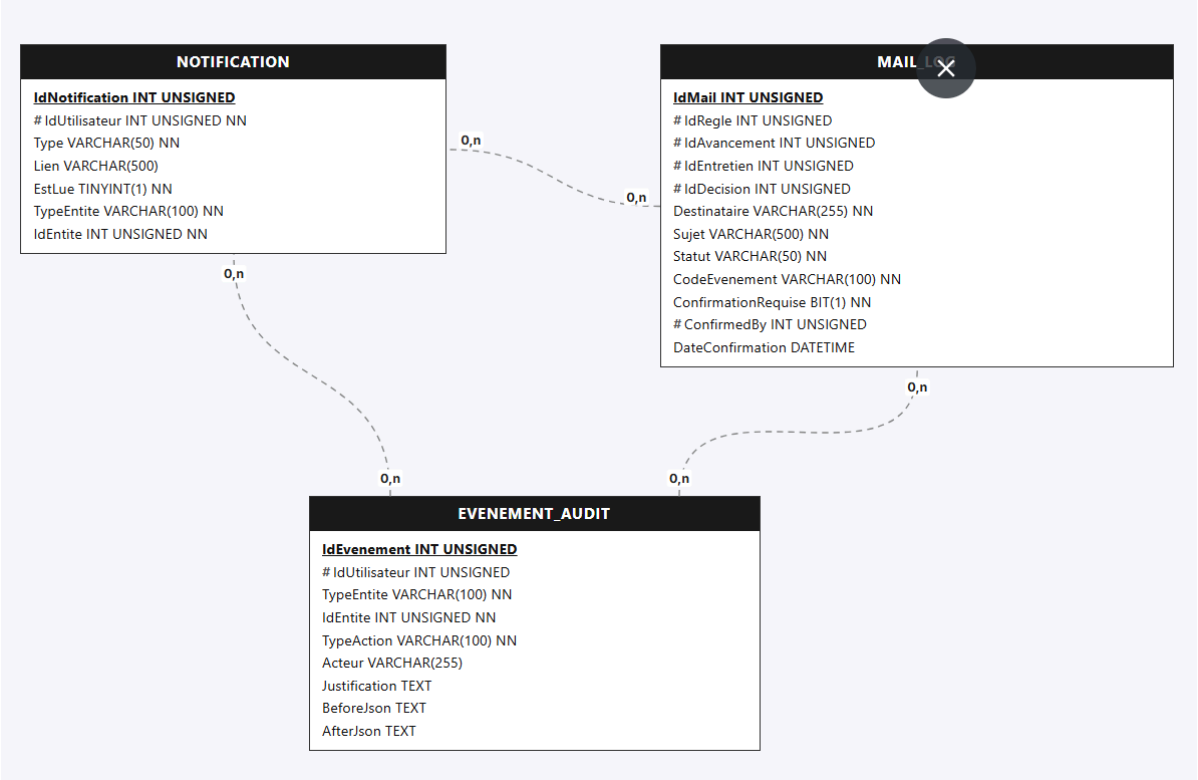


Fig. 21. – MPD — Notifications, mails & audit.

Convention physique	Implémentation retenue	Justification
Clés primaires	Identifiants de type int unsigned.	Simplicité de jointure, homogénéité de schéma et compatibilité avec les conventions existantes du projet.
Colonnes d'audit	created_at, created_by, updated_at, updated_by.	Traçabilité technique des créations et modifications d'enregistrements.
Soft delete	Colonne is_deleted plutôt qu'une suppression physique.	Préserver l'historique et limiter les ruptures de cohérence sur les données liées.
Recherche	Colonne search_string sur les tables concernées.	Faciliter la recherche textuelle et les filtres fonctionnels sans dupliquer la logique côté interface.
Indicateurs booléens	bit(1) ou tinyint(1) selon les tables.	Représenter les états simples tels que is_locked, is_active, email_verified ou is_cancel.

Paramètres dynamiques	config_json sur pipeline_stage.	Permettre une configuration flexible des étapes sans casser le noyau relationnel.
Enums métier	Valeurs stockées en varchar ou enum selon les cas.	Conserver une expressivité métier tout en matérialisant progressivement les listes fermées dans le code.
Migrations	Versionnement Flyway du schéma.	Rendre maîtrisable l'évolution de la base dans le temps et synchroniser code et structure.

4.2.5 Dictionnaire de données du noyau métier

Le dictionnaire de données suivant synthétise quelques attributs structurants du noyau métier. Il ne remplace pas un inventaire exhaustif de la base, mais il matérialise les données indispensables à la compréhension du système et au raisonnement de conception.

Entité	Attribut	Type / format	Signification
campaigns	campaign_title	varchar	Libellé métier de la campagne visible dans les écrans de pilotage.
campaigns	campaign_status	varchar(7)	État de la campagne, utilisé pour contrôler son cycle de vie et sa diffusion.
campaigns	pipeline_template_id	FK	Pipeline de référence appliqué à la campagne.
form_version	version_number	int	Numéro de version du formulaire, garantissant l'historique des questionnaires.
form_version	is_locked	bit(1)	Indique qu'une version ne doit plus être modifiée de manière destructive.
question	question_type	varchar(20)	Nature de la question : choix unique, multiple ou

			réponse libre selon la modélisation retenue.
applications	candidate_email	donnée identité	Adresse utilisée pour la vérification, la communication et l'identification du candidat.
applications	email_verified	tinyint(1)	Indique si le candidat a validé son adresse email.
applications	application_status	varchar(8)	Synthèse de l'état du dossier au niveau candidature.
applications	quiz_status	varchar(9)	État du quiz pour la candidature : non commencé, en cours, terminé, etc.
applications	form_version_id	FK	Version de formulaire effectivement subie par le candidat.
application_progress	progress_status	varchar(20)	État de l'étape de pipeline concernée.
application_progress	score	double	Valeur quantitative liée à l'étape, par exemple score de quiz ou résultat partiel.
pipeline_stage	stage_type	varchar(50)	Nature métier de l'étape : candidature, quiz, entretien, décision, etc.
pipeline_stage	stage_order	int	Ordre d'exécution de l'étape dans le pipeline.
candidate_interview	alternative_date_requested	datetime	Date proposée par le candidat en cas de demande de reprogrammation.

decisions	decision_type	varchar(50)	Type de décision porté sur le dossier : pré-sélection, refus, décision finale, etc.

4.2.6 Correspondance entre objets Merise et tables physiques

Un mémoire d'ingénierie doit montrer explicitement que les objets conceptuels ne restent pas abstraits. La matrice suivante établit donc un pont entre le vocabulaire Merise du projet et les tables réellement présentes dans l'implémentation.

Objet conceptuel	Tables physiques	Lecture d'implémentation
Campagne de recrutement	campaigns ; campaign_crud ; source_token	La campagne est prolongée par les droits collaboratifs et par les mécanismes de diffusion publique.
Formulaire versionné	form ; form_version ; form_questions	La stabilité du questionnaire repose sur le couple versionnement-composition.
Banque de questions	question ; answer	Les questions restent réutilisables et spécialisées par métier.
Dossier candidat	applications ; candidate_answer ; application_progress	La candidature concentre données personnelles, réponses et historique d'avancement.
Entretien	interviews ; candidate_interview ; evaluations	L'entretien sépare la session planifiée, la réponse candidat et l'évaluation collaborative.
Décision	decisions	La décision est historisée comme un objet autonome, rattaché à un auteur et à une étape.
Gouvernance des droits	user ; system_role_permission ; entity_permission ; user_permission_override	La sécurité combine rôles système et permissions ciblées.

4.2.7 Dépendances fonctionnelles

Les dépendances fonctionnelles permettent de clarifier quels attributs dépendent directement d'un identifiant donné. Elles constituent un intermédiaire important entre le dictionnaire de données et le passage vers le MLD, car elles mettent en évidence les déterminants du système et aident à raisonner sur la cohérence des relations.

Déterminant	Attributs dépendants ou lecture fonctionnelle
campaign_id	campaign_title, campaign_status, campaign_start, campaign_end, pipeline_template_id, creator_id, speciality_id
form_version_id	form_id, version_number, is_published, is_locked
question_id	question_text, question_type, question_chrono, score, difficulty_level, speciality_id
application_id	candidate_first_name, candidate_last_name, candidate_email, source_token_id, form_version_id, application_status, quiz_status
application_progress_id	application_id, pipeline_stage_id, progress_status, started_at, completed_at, score, decided_by
interview_id	campaign_id, application_id, interview_date, interview_status, interview_type, interview_mail_sent
candidate_interview_id	application_id, interview_id, interview_start, interview_duration, decline_reason, alternative_date_requested
decision_id	application_id, user_id, pipeline_stage_id, decision_type, is_cancel

4.2.8 Structure d'accès théorique (SAT)

La SAT formalise les chemins d'accès privilégiés entre les identifiants structurants du système. Dans SyneliaQuizApp, elle montre que les pivots majeurs sont la campagne pour la configuration du recrutement, la candidature pour l'exécution du parcours candidat, et la version de formulaire pour la traçabilité du quiz.

Les figures suivantes présentent la structure d'accès théorique, décomposée en deux phases du processus de recrutement.

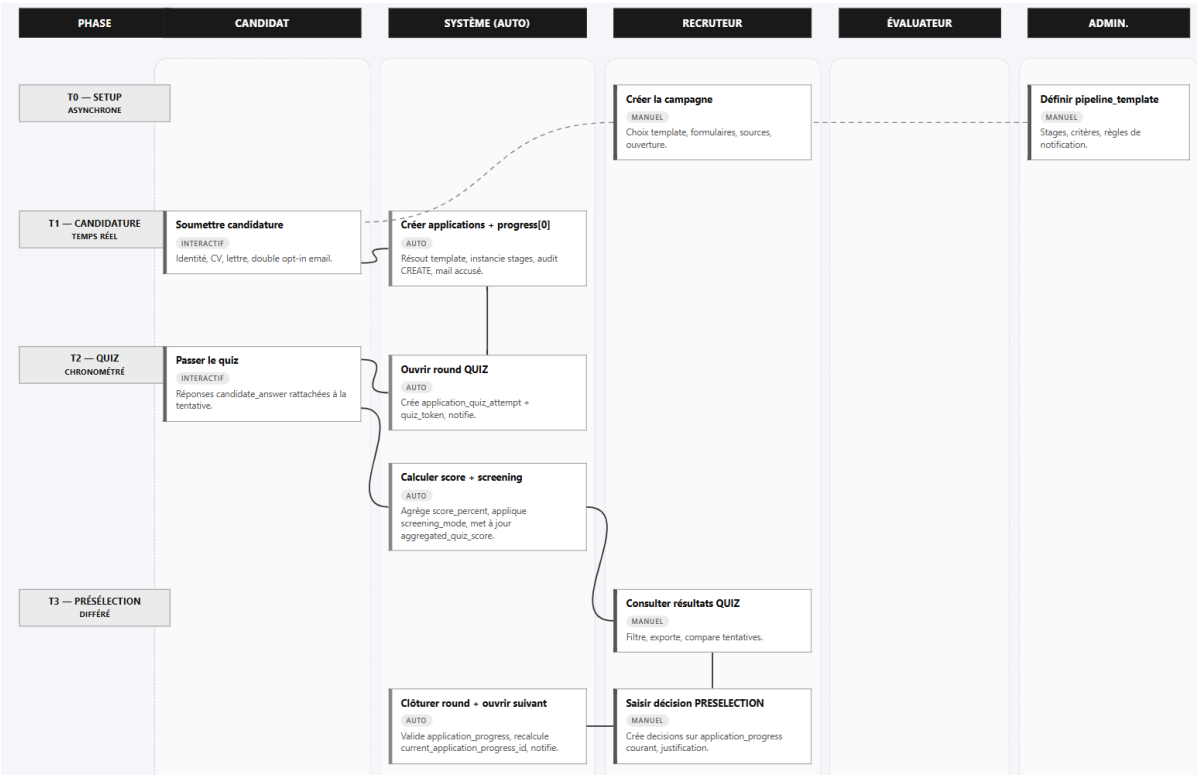


Fig. 22. – SAT — Configuration, candidature, quiz & présélection (T0 ⟶ T3).

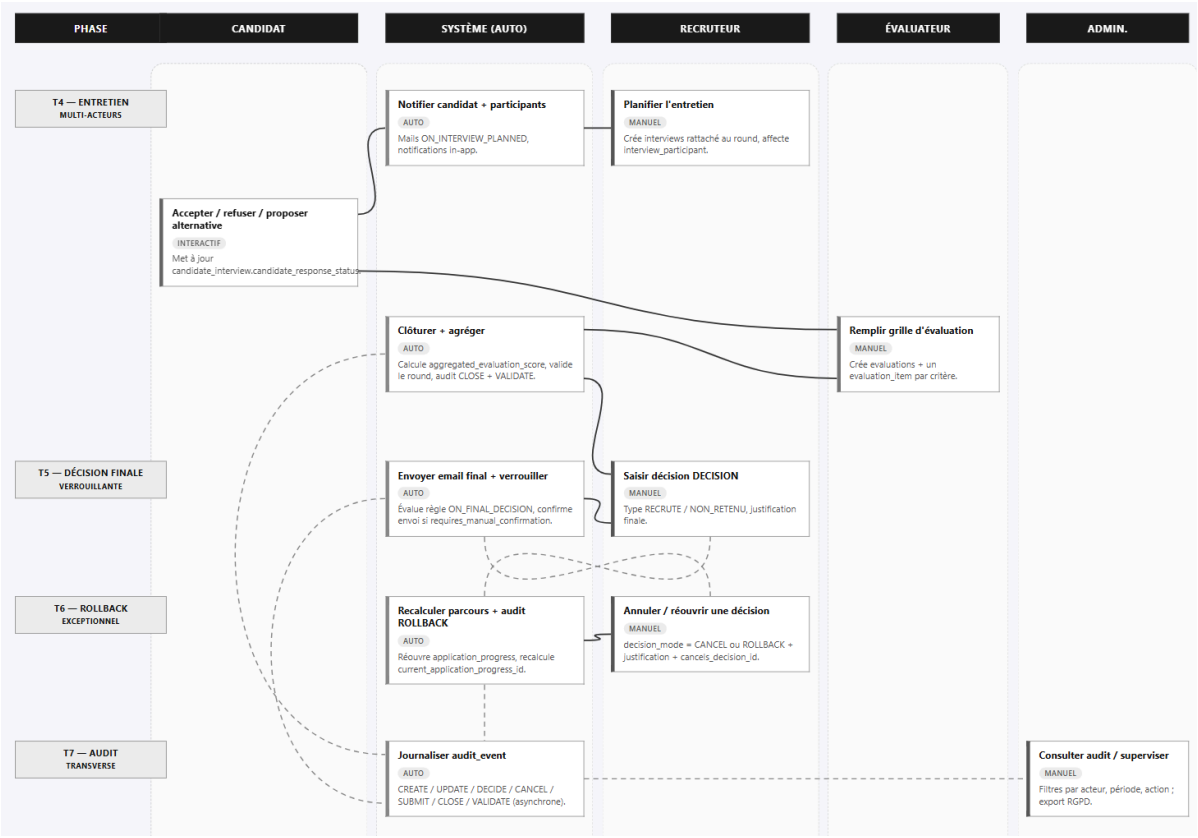


Fig. 23. – SAT — Entretien, décision, rollback & audit (T4 ⟶ T7).

Point d'entrée	Accès structurés dans le système

campaigns.id	form.campaign_id ; interviews.campaign_id ; source_token.campaign_id ; campaign_crud.campaign_id
form.id	form_version.form_id
form_version.id	form_questions.form_version_id ; applications.form_version_id
applications.id	candidate_answer.application_id ; application_progress.application_id ; candidate_interview.application_id ; decisions.application_id
pipeline_template.id	pipeline_stage.pipeline_template_id ; campaigns.pipeline_template_id
interviews.id	candidate_interview.interview_id ; interview_participant.interview_id
candidate_interview.id	evaluations.candidate_interview_id
user.id	campaign_crud.user_id ; decisions.user_id ; entity_permission.user_id ; user_permission_override.user_id

Cette lecture met en évidence deux carrefours informationnels essentiels. D'une part, `applications` concentre le suivi du candidat, ses réponses, ses étapes de pipeline, ses entretiens et ses décisions. D'autre part, `campaigns` concentre le paramétrage global du recrutement et la gouvernance collaborative autour de la campagne.

4.2.9 Modèle Conceptuel des Traitements (MCT)

Pour rester cohérent avec une logique Merise-first, la conception ne doit pas s'arrêter aux seules données. Les traitements majeurs du système peuvent eux aussi être formalisés à travers quelques MCT représentatifs du fonctionnement réel de SyneliaQuizApp.

MCT	Déclencheur	Transformation métier attendue
MCT-1 Soumission d'une candidature	Le candidat ouvre une campagne et soumet son dossier.	Le système contrôle le contexte de campagne, crée la candidature, rattache la version de formulaire et initialise le suivi.

MCT-2 Qualification initiale	Le dossier est vérifié et l'accès au quiz devient possible.	Le système gère la vérification d'email, ouvre les actions autorisées et fait progresser la candidature dans son pipeline.
MCT-3 Passation et correction du quiz	Le candidat démarre ou termine son quiz technique.	Les réponses sont enregistrées, le score est calculé ou bien un processus de correction manuelle est déclenché.
MCT-4 Entretien, évaluation et décision	Le recruteur planifie un entretien ou statue sur le dossier.	La convocation, la réponse candidat, les évaluations et la décision finale alimentent le suivi et l'historique métier.

Les figures suivantes présentent le modèle conceptuel des traitements, décomposé en trois phases du cycle de recrutement.

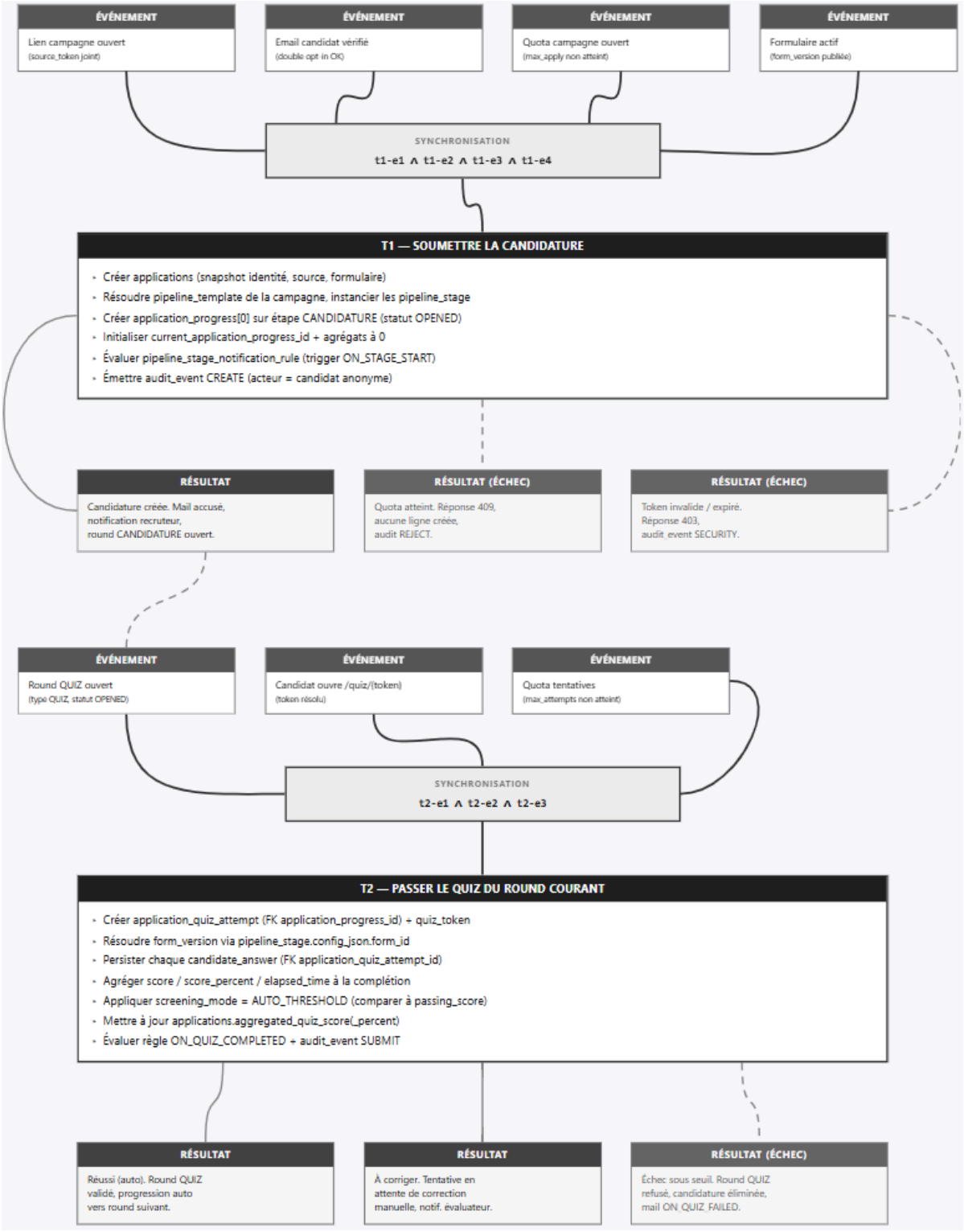


Fig. 24. – MCT — Soumettre une candidature & passer le quiz (T1 + T2).

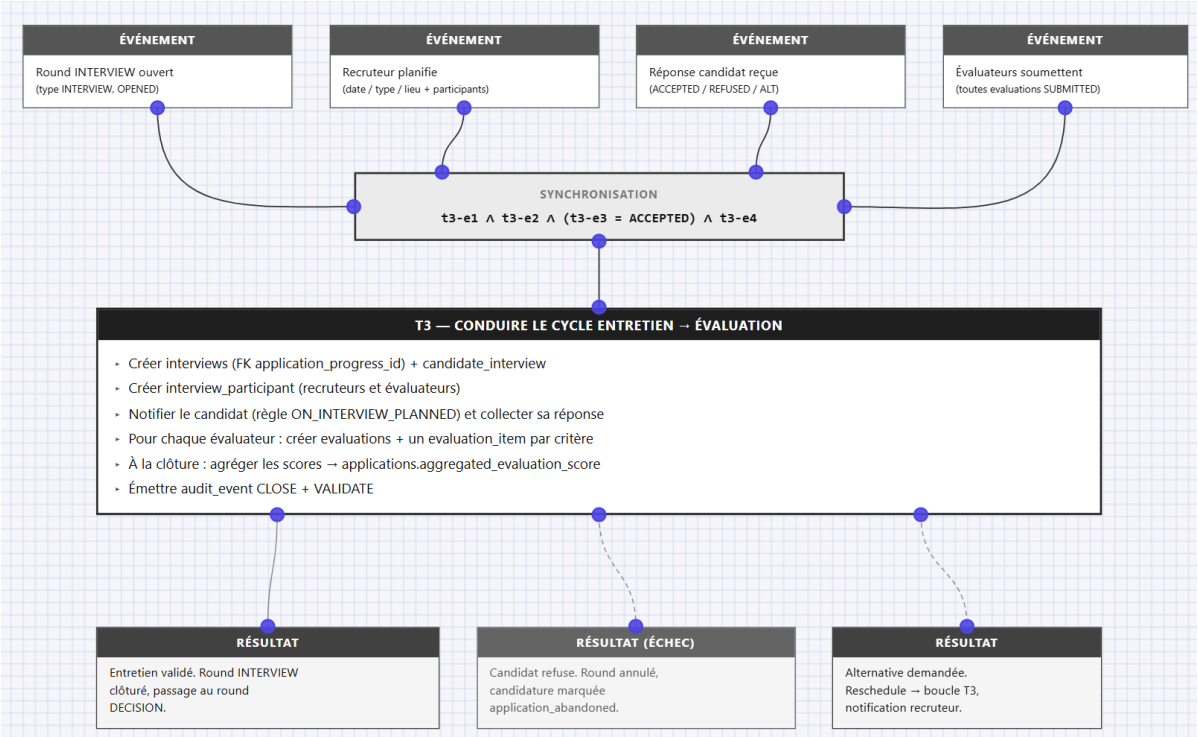


Fig. 25. – MCT — Conduire le cycle entretien ⟶ évaluation (T3).

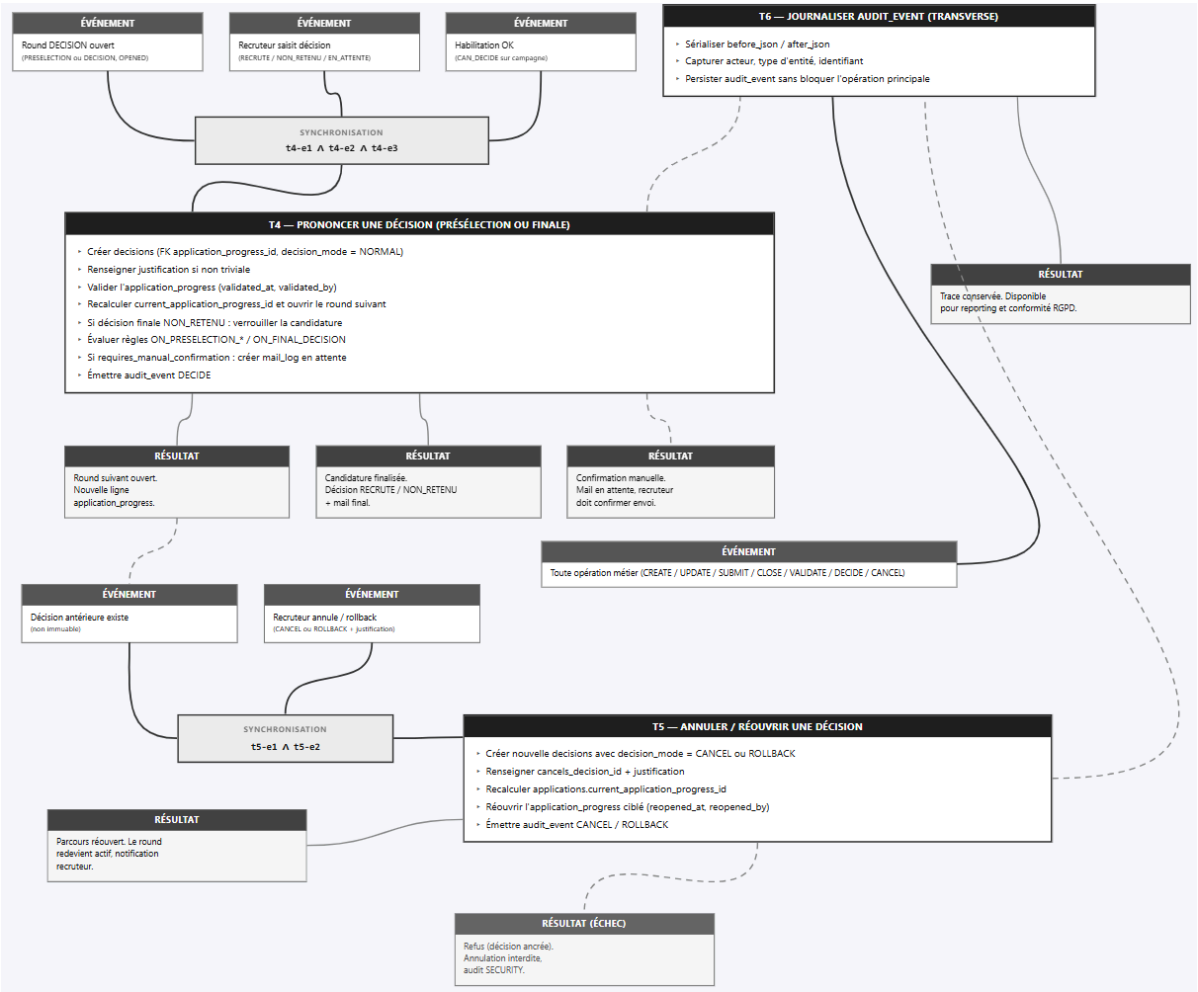
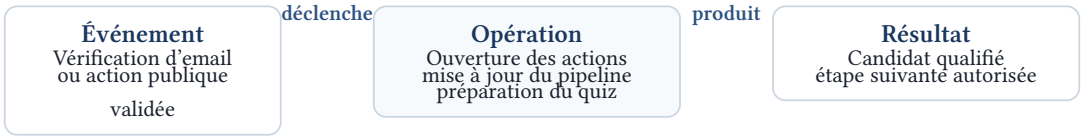
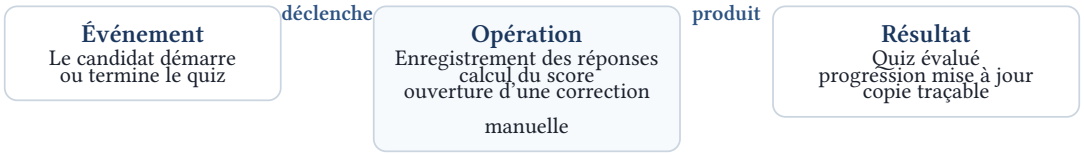


Fig. 26. – MCT — Prononcer une décision, annuler & auditer (T4 + T5 + T6).

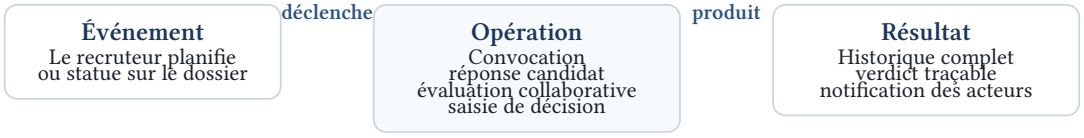
MCT-2 : Qualification initiale



MCT-3 : Passation et correction du quiz



MCT-4 : Entretien, évaluation et décision



4.2.10 Modèle Physique des Traitements (MPT)

Le MPT descend au niveau de l'implémentation et décrit l'architecture physique des traitements : IHM (Angular), contrôleur REST (Spring Boot), service métier, couche d'accès aux données (JPA), SGBD (MySQL) et services externes mobilisés. Il met en évidence l'empilement technique réel qui supporte chaque traitement métier.

MPT	Périmètre couvert	Lecture d'implémentation
MPT-1 Candidature et quiz	Soumission d'une candidature (T1) et passation du quiz technique (T2).	Angular → POST /api/applications → QuizService → Repository → MySQL → Notification SMTP.
MPT-2 Entretien et décision	Planification d'un entretien (T3) et prise de décision avec audit (T4).	Angular → POST /api/interviews → InterviewService → DecisionService → Repository → MySQL → SMTP + audit_event.

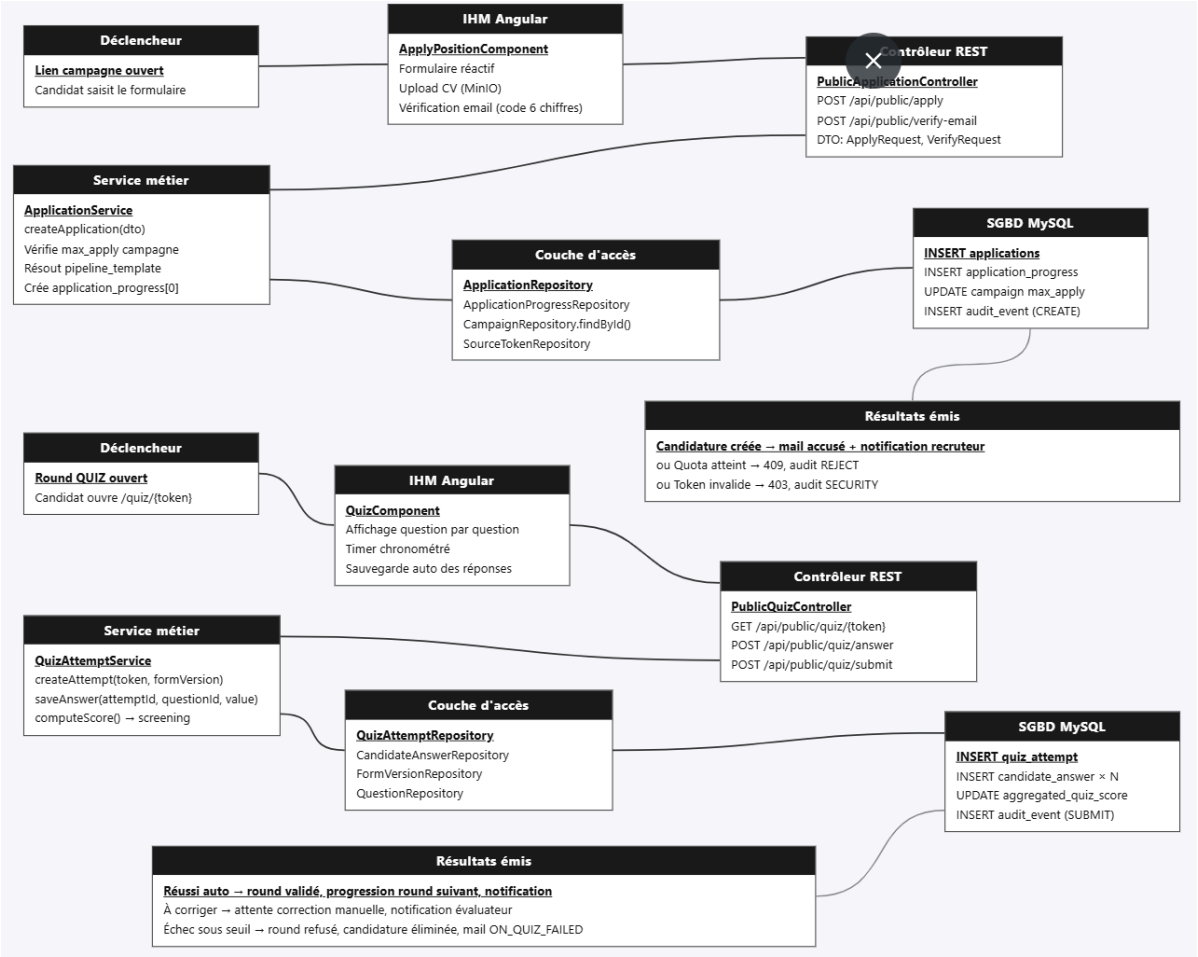


Fig. 27. – MPT 1/2 — Soumettre une candidature (T1) et passer le quiz (T2). Architecture physique : IHM, API, services, JPA, MySQL et services auxiliaires.

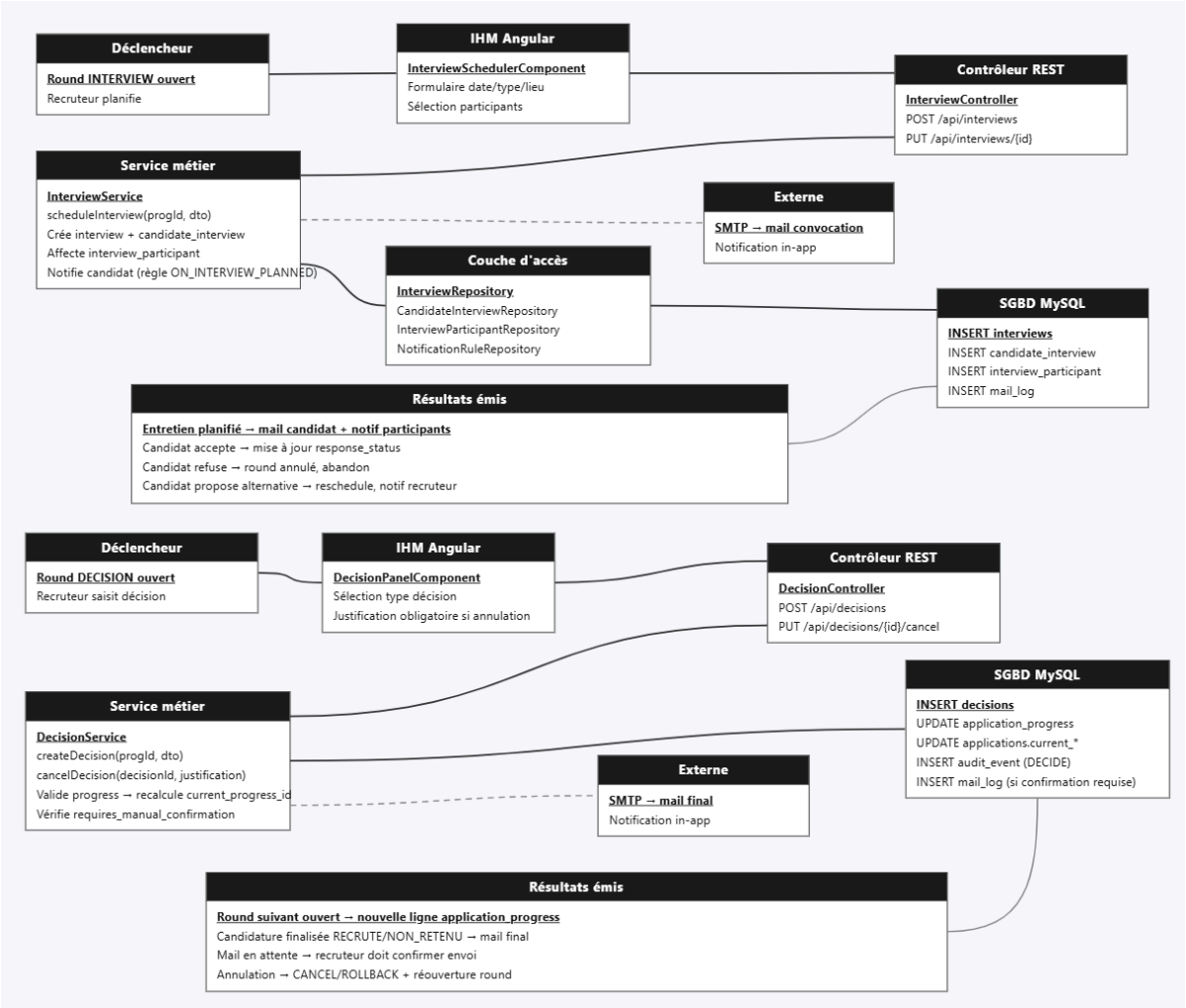


Fig. 28. – MPT 2/2 — Planifier un entretien, évaluer, décider et auditer (T3 + T4). L'audit transverse est journalisé en parallèle sur chaque opération.

4.2.11 Modèle Conceptuel de Communication (MCC)

Le MCC modélise les flux d'information entre les acteurs du système, l'application SyneliaQuizApp et les services externes. Il ne décrit pas le contenu fonctionnel des échanges, mais leur existence, leur direction et leur nature (synchrone ou asynchrone, interactif ou automatique).

Flux	Sens	Nature de l'échange
Candidat ↔ Système	Bidirectionnel	Soumission de candidature, quiz, suivi, réponse à une convocation — interface publique.
Recruteur ↔ Système	Bidirectionnel	Création de campagne, évaluation, planification, décision — interface interne.

Administrateur ↔ Système	Bidirectionnel	Gestion des utilisateurs, permissions, référentiels, templates et paramètres.
Super admin ↔ Système	Bidirectionnel	Gouvernance de la plateforme, supervision de sécurité, configuration SMTP et supervision.
Système ⟶ SMTP	Unidirectionnel	Envoi automatique de mails transactionnels : vérification, convocation, décision, notifications.
Système ↔ MinIO	Bidirectionnel	Stockage et récupération des pièces jointes candidat (CV, documents).
Système ↔ Redis	Bidirectionnel	Cache de session, jetons temporaires, files d'attente de traitements.
Système ↔ MySQL	Bidirectionnel	Persistance de l'ensemble des données métier, migrations versionnées par Flyway.

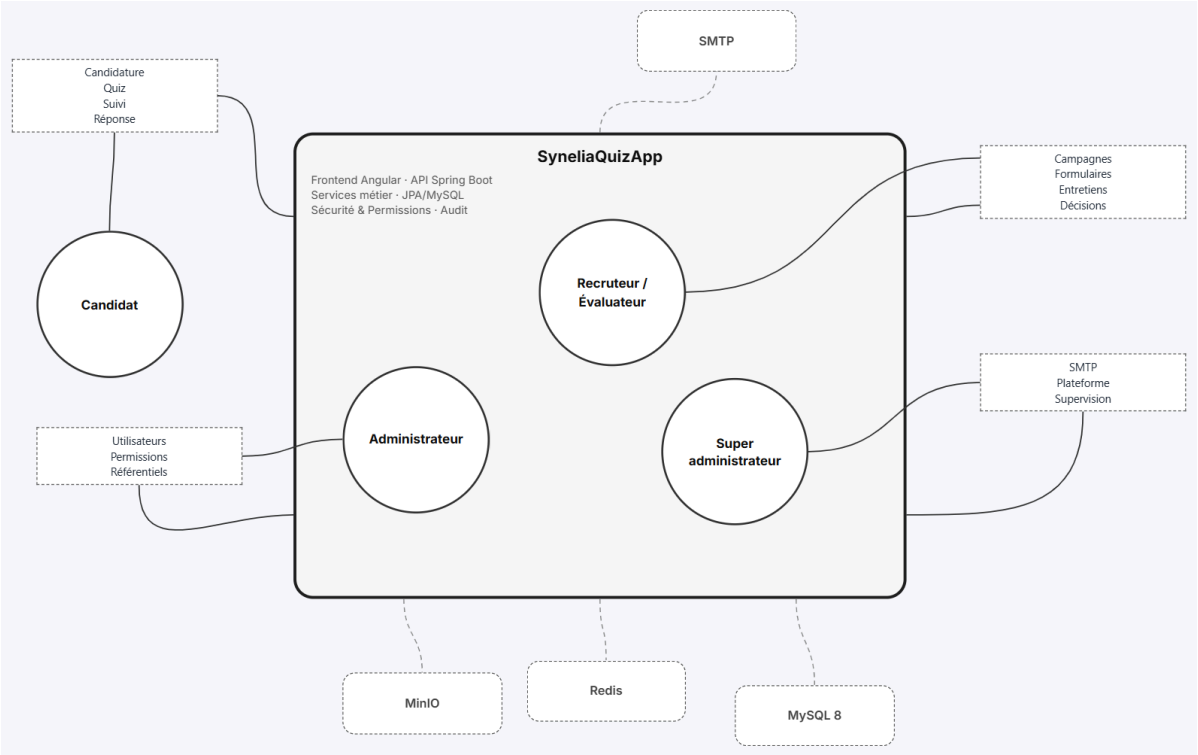


Fig. 29. – MCC — Flux d'information entre acteurs (Candidat, Recruteur, Admin, Super admin), le système SyneliaQuizApp et les services externes (SMTP, MinIO, Redis, MySQL).

4.3 Compléments éventuels en UML

Les diagrammes qui suivent ne constituent pas la colonne vertébrale de la conception. Ils peuvent être utiles en complément, notamment pour certains échanges applicatifs ou comportements dynamiques, mais ils viennent après les artefacts Merise et non avant eux.

4.3.1 Diagrammes de classes

Les diagrammes de classes du backend doivent mettre en évidence les responsabilités principales de l'application plutôt qu'une simple hiérarchie technique. Les éléments centraux de la modélisation résident d'une part dans les entités persistées, d'autre part dans les services métier qui orchestrent les cas d'usage. Le mémoire gagnera donc en lisibilité en présentant plusieurs vues ciblées plutôt qu'un diagramme unique trop dense.

Vue de classes	Objets à faire apparaître	Intérêt pour le mémoire
Vue métier globale	Campagne, Formulaire, VersionFormulaire, Question, Candidature, Progression, Entretien, Évaluation, Décision.	Montrer le noyau fonctionnel du recrutement et les dépendances entre objets métier.
Vue utilisateurs et permissions	Utilisateur, Groupe métier, Permission système, Permission d'entité, Override utilisateur.	Expliquer la gouvernance des accès et la différence entre rôle global et droit contextualisé.
Vue campagnes et quiz	Campagne, PipelineTemplate, PipelineStage, Formulaire, VersionFormulaire, Question, Réponse, RéponseCandidat.	Mettre en évidence la préparation de l'évaluation technique et son ancrage dans la campagne.
Vue entretiens et décisions	Entretien, EntretienCandidat, Évaluation, Décision, Notification.	Illustrer la fin du processus de recrutement et la production du verdict.
Vue applicative	Contrôleurs, services métier, repositories, DTO principaux.	Montrer comment les cas d'usage sont orchestrés dans la couche backend.

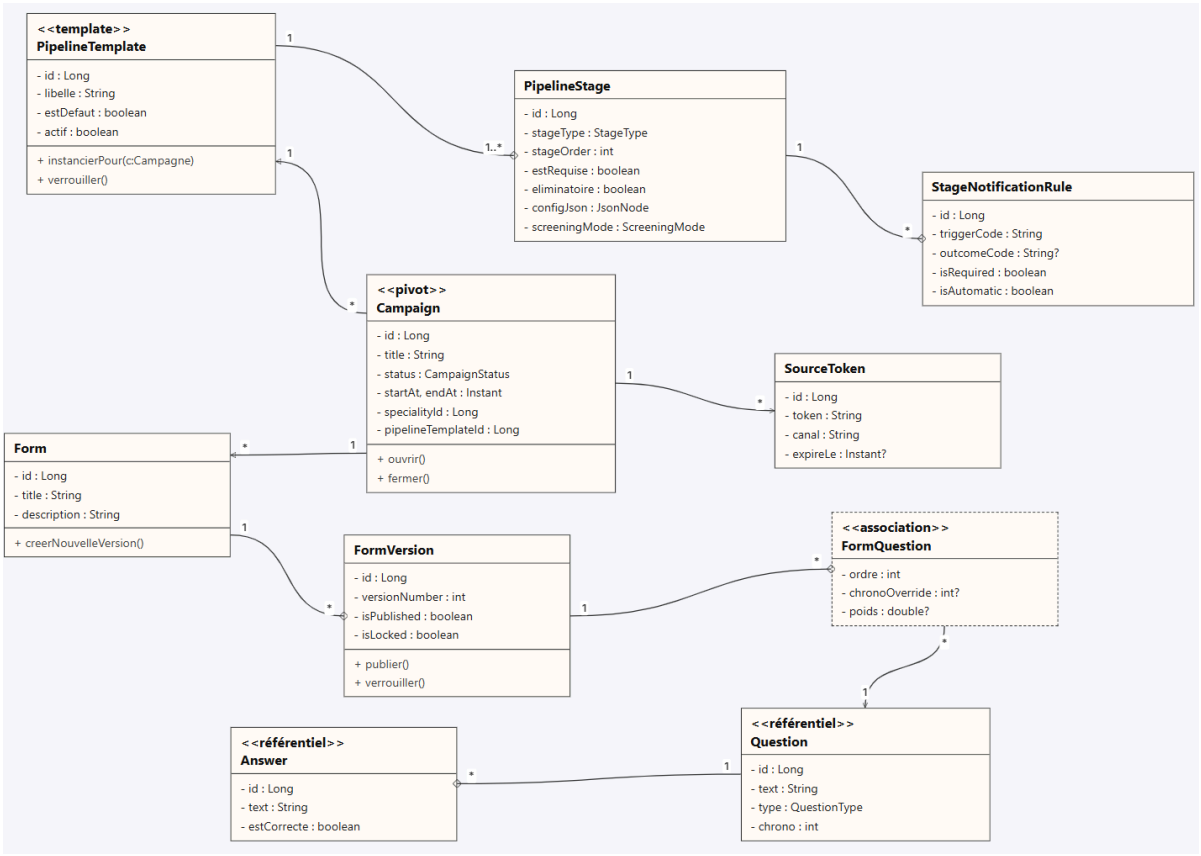


Fig. 30. – Classes UML 1/4 — Pipeline, Campagnes et Formulaires (9 classes).

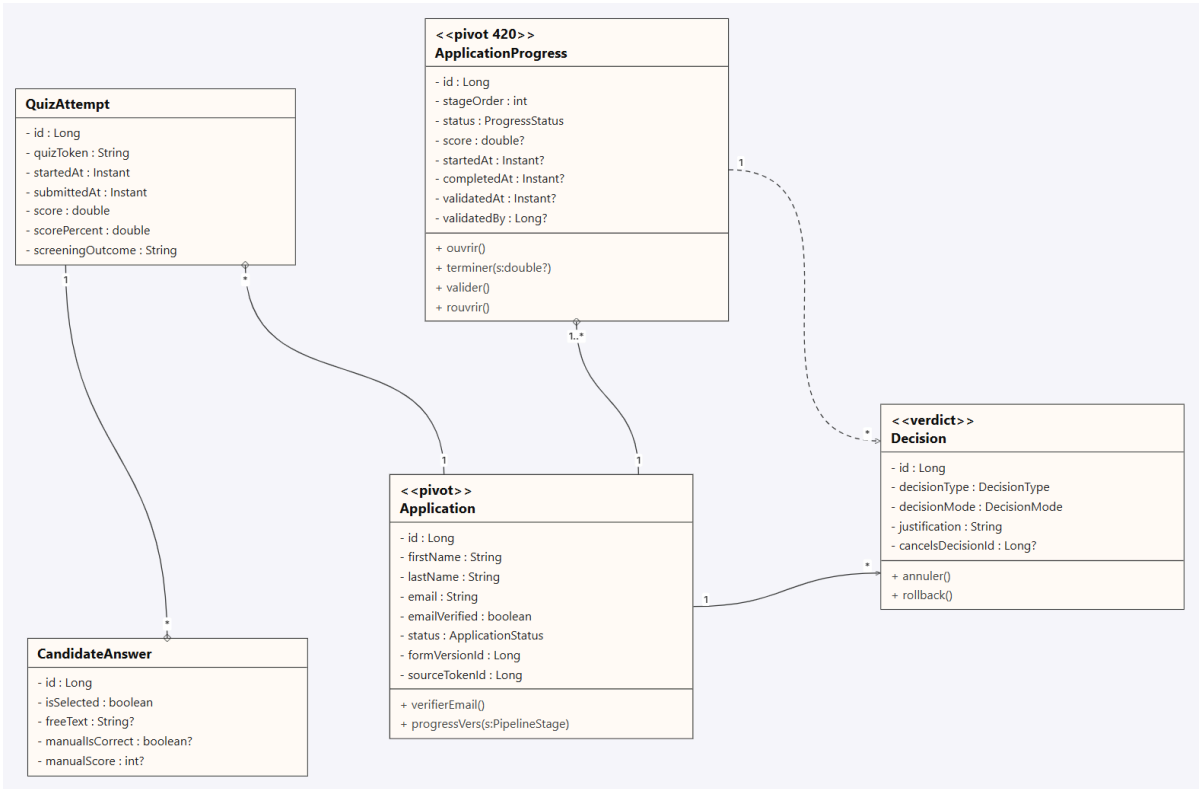


Fig. 31. – Classes UML 2/4 — Candidature, Avancement pipeline, Quiz et Décisions (5 classes).

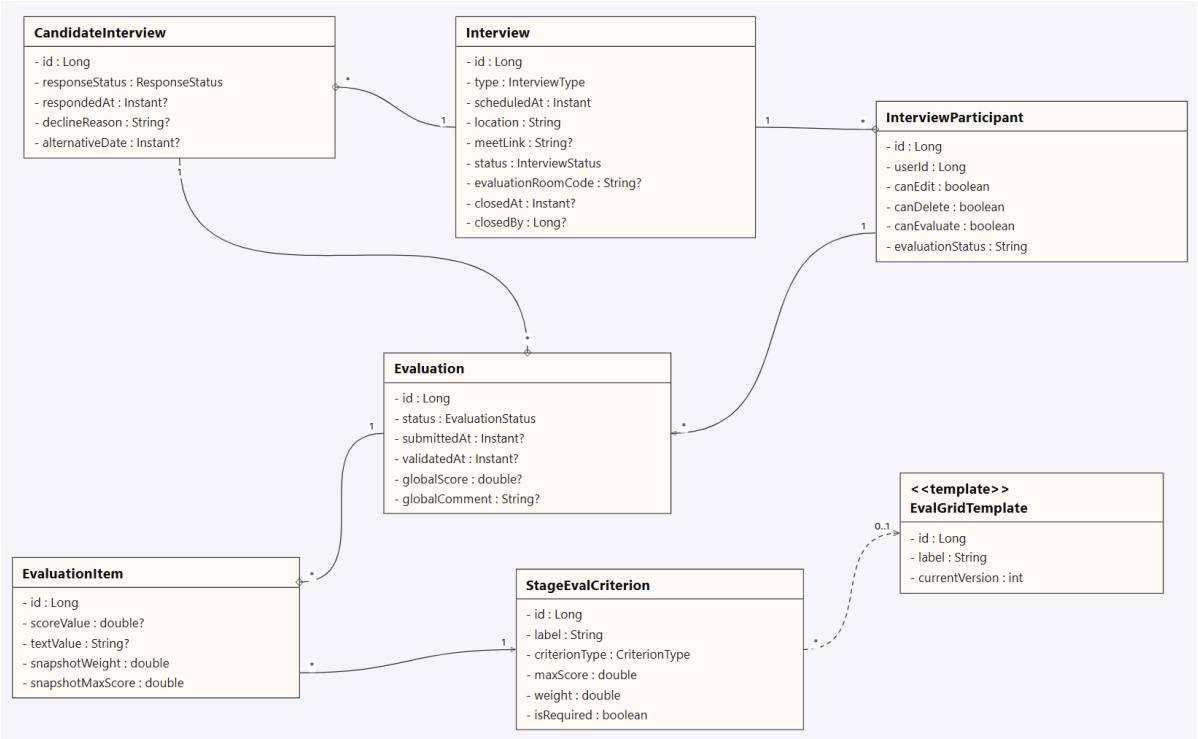


Fig. 32. – Classes UML 3/4 — Entretiens, Participants et Évaluations (7 classes).

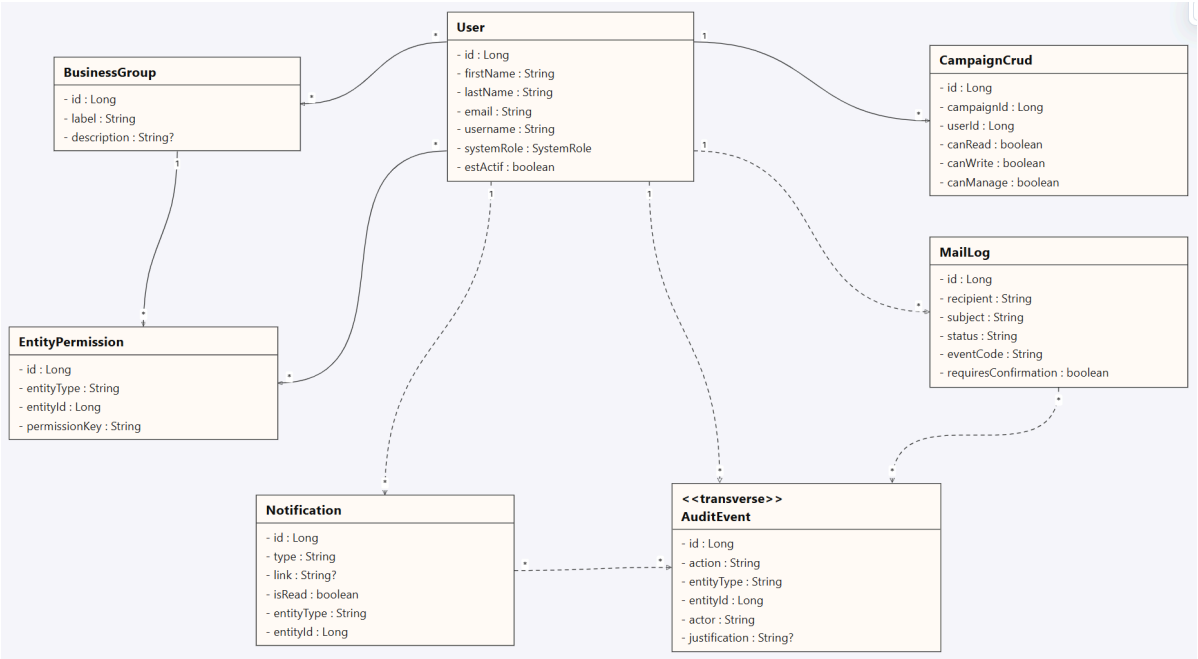


Fig. 33. – Classes UML 4/4 — Sécurité, Permissions, Notifications et Audit (7 classes).

4.3.2 Diagrammes de séquence

Les diagrammes de séquence ont pour rôle de montrer le comportement dynamique du système et l'enchaînement réel des composants lors d'un cas d'usage. Ils sont particulièrement utiles pour expliciter les interactions entre le frontend, l'API, les services métier, la persistance et les mécanismes de notification.

Séquence	Objets principaux	Ce que le diagramme doit démontrer
Candidature publique	Candidat, interface publique, API candidature, base de données, service email.	La validation du contexte de campagne, la création du dossier, la génération des informations de suivi et la notification initiale.
Passation du quiz	Candidat, interface quiz, API quiz, formulaire versionné, réponses candidat.	Le chargement des questions autorisées, l'enregistrement progressif des réponses et la clôture du quiz.
Planification d'entretien	Recruteur, API entretien, candidature, entretien, notification.	Le rattachement de l'entretien à la bonne candidature, la production de la convocation et la mise à jour du pipeline.
Décision finale	Recruteur, évaluateurs, API décision, candidature, progression, notification.	L'agrégation des éléments d'évaluation, la saisie de la décision et la synchronisation des états du dossier.

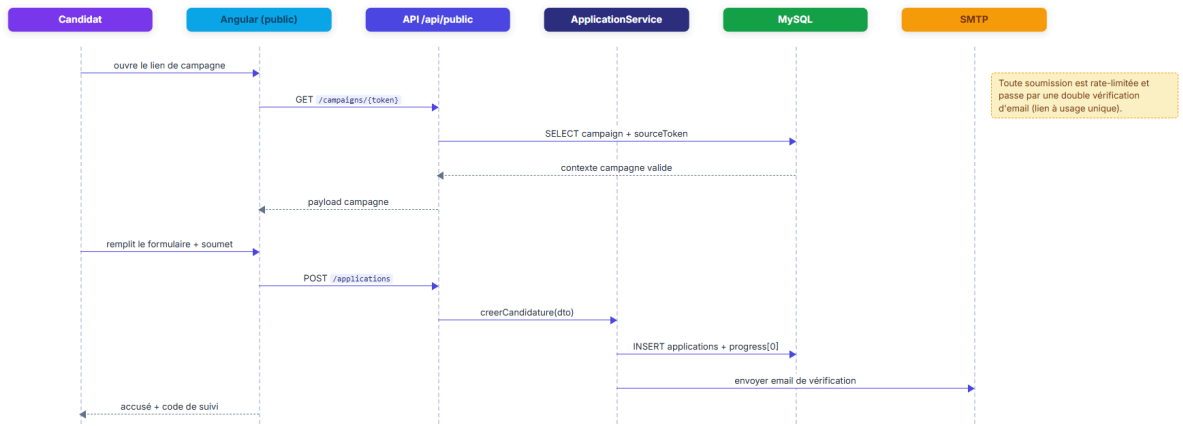


Fig. 34. – Séquence — soumission d'une candidature publique.

Les diagrammes de séquence complémentaires (passation du quiz, planification d'entretien et décision finale) sont présentés en Annexe B.

4.3.3 Diagrammes d'activités et d'états

Les diagrammes d'activités et d'états doivent représenter la candidature comme un processus évolutif plutôt que comme un simple enregistrement doté d'un champ de statut. Cette modélisation est essentielle pour expliquer comment la logique de pipeline

structure les actions possibles et comment les différents sous-états du système produisent une lecture globale du dossier.

Diagramme	Périmètre	Information apportée
Activité globale candidature	Du dépôt du dossier à la décision finale.	Vue macro du parcours candidat, des bifurcations possibles et des points de contrôle.
États de campagne	Brouillon, ouverture, pause, clôture et variantes de diffusion.	Conditions de disponibilité de l'offre et effets sur l'accès public.
États du quiz	Non commencé, en cours, terminé, corrigé manuellement, invalidé.	Règles de gestion liées à la passation et à l'évaluation technique.
États entretien / décision	Planifié, répondu, évalué, décidé, annulé, replanifié.	Traçabilité de la fin de parcours et cohérence des verdicts RH.

4.4 Maquettes et interfaces

La conception des interfaces de SyneliaQuizApp a été guidée par les principes d'ergonomie logicielle formalisés par J.-F. Nogier et J. Bouillot dans **Ergonomie du logiciel et design web** (6e édition, 2020), qui insistent sur l'adéquation entre l'interface et le modèle mental de l'utilisateur. Les maquettes et interfaces de la solution s'organisent autour de trois univers d'usage distincts. Le premier est l'espace public candidat, qui comprend les écrans de candidature, de vérification d'email, de quiz et de suivi. L'enjeu principal de cette interface est la simplicité : le candidat doit comprendre rapidement ce qu'on attend de lui, où il en est et quelles actions lui sont encore accessibles.

Le deuxième univers est celui du recruteur. Il comprend les listes de campagnes, les vues de candidatures, les écrans de planification d'entretien, les interfaces d'évaluation, les espaces de décision, la gestion des formulaires, des versions de formulaire, des questions et des liens de diffusion. Ici, la logique d'interaction doit favoriser la lecture rapide, le filtrage, l'accès au contexte utile et la sécurisation des actions sensibles par des droits explicites ou par des confirmations adaptées.

Le troisième univers est celui de l'administration. Il regroupe les référentiels, les paramètres globaux, les templates d'email, la configuration SMTP, les permissions et certaines vues techniques du système. Les maquettes associées devront mettre en évidence une

exigence particulière de lisibilité et de cohérence, car elles supportent des opérations de gouvernance dont dépend une grande partie du comportement applicatif.

Univers	Écrans clés	Objectif de conception
Candidat	Candidature publique ; vérification d'email ; quiz ; suivi de candidature ; réponse entretien.	Réduire la friction, expliciter l'état du dossier et sécuriser les actions disponibles.
Recruteur	Campagnes ; formulaires ; versions ; questions ; candidatures ; entretiens ; évaluations ; décisions ; dashboard.	Accélérer le traitement des dossiers et garantir la lecture du contexte métier avant action.
Administration	Utilisateurs ; rôles ; permissions ; départements ; spécialités ; templates d'email ; paramètres globaux.	Permettre la gouvernance, la configuration et la maintenance fonctionnelle du système.

4.5 Conception de l'API REST

La conception de l'API repose sur une logique de services orientés métier, en cohérence avec les principes architecturaux REST de Roy T. Fielding, dans sa thèse **Architectural Styles and the Design of Network-based Software Architectures** (2000), et les règles de conception d'API proposées par M. Massé dans **REST API Design Rulebook** (2011). Les endpoints ne se limitent pas à un CRUD canonique ; ils exposent aussi des actions applicatives explicites, telles que la progression dans le pipeline, la réponse à un entretien, la publication d'un formulaire, la vérification d'un email ou la production d'une décision. Cette orientation rend l'API plus proche du langage métier du système et facilite le pilotage des cas d'usage complexes.

Les échanges sont essentiellement réalisés au format JSON, via des DTO dédiés servant de contrats entre frontend et backend. Les structures de requête permettent de transmettre soit des recherches par critères, soit des objets métier à créer ou à mettre à jour, parfois regroupés dans des tableaux de données. Les réponses ont pour objectif de renvoyer des objets directement exploitables par l'interface, y compris lorsqu'elles portent des champs de synthèse ou des informations dérivées utiles à l'affichage.

L'API suit également des conventions de mise à jour partielles importantes pour la stabilité du projet : certaines valeurs omises ne doivent pas être écrasées, certaines valeurs vides doivent être ignorées, et certains champs calculés ou historisés restent strictement

propriétaires du backend. La gestion des erreurs doit enfin préserver un équilibre entre rigueur technique et lisibilité fonctionnelle : contrôle des entrées, refus explicite des incohérences critiques, messages exploitables pour le frontend et distinction claire entre erreur métier, erreur de validation et défaut d'infrastructure.

Famille de services	Données et actions couvertes	Intention de conception
Authentification et permissions	Connexion interne, gestion des utilisateurs, rôles, groupes, overrides et permissions d'entité.	Sécuriser l'accès et exprimer les responsabilités réelles des acteurs.
Campagnes et pipeline	Création de campagne, paramétrage du pipeline, partage, filtrage et pilotage des dossiers.	Faire de la campagne le centre du workflow de recrutement.
Formulaires et quiz	Gestion des formulaires, versioning, composition des questions, passation, sauvegarde et correction.	Garantir la comparabilité des évaluations techniques et la traçabilité des réponses.
Candidatures et parcours public	Création du dossier, vérification d'email, suivi candidat, demandes et consultation des statuts.	Maintenir une expérience publique simple sans sacrifier la sécurité ni la cohérence métier.
Entretiens, évaluations et décisions	Planification, réponse candidat, saisie d'évaluation, consolidation et décision.	Coordonner la fin du parcours de recrutement et conserver un historique explicable.
Administration et communication	Paramètres globaux, SMTP, templates, notifications et contenus administrables.	Assurer la gouvernance opérationnelle et la communication du système.

Troisième partie

Réalisation et mise en œuvre

Technologies utilisées, organisation du code et stratégie d'exécution du projet

CHAPITRE 5

Environnement technique et outils

Ce chapitre présente l'environnement réel dans lequel SyneliaQuizApp a été développé, testé et exécuté. Il constitue un point d'entrée concret et factuel pour la lecture technique du mémoire.

5.1 Environnement de développement

Le développement de SyneliaQuizApp a été réalisé dans un environnement de travail orienté poste client, adapté à une production incrémentale et à des validations fréquentes. Les itérations les plus récentes du projet ont été menées sous **Windows**, avec **Visual Studio Code** comme environnement principal d'édition, de navigation dans le code et de pilotage des validations. Ce choix s'explique par la complémentarité des extensions utilisées pour Java, Angular, PowerShell, Markdown et, dans le cadre de la préparation du mémoire, Typst.

L'organisation du travail s'appuie sur une séparation explicite entre le backend, le frontend, la documentation fonctionnelle et les notes de pilotage. Le dépôt contient ainsi le projet Spring Boot dans `quizapp/`, l'application Angular dans `quizapp_front/`, des scripts utilitaires, un historique de notes techniques, ainsi que les documents de cadrage et de mémoire. Cette structuration a favorisé une progression par slices fonctionnelles, chaque évolution étant accompagnée d'un support de cartographie, de validation et de capitalisation.

La gestion de version repose sur **Git**, tandis que l'exécution locale s'appuie sur des commandes simples et reproductibles. Côté backend, la présence de `mvnw` et `mvnw.cmd` permet de standardiser les builds Maven. Côté frontend, les scripts déclarés dans `package.json` centralisent le démarrage, la compilation, le mode watch et les tests. Des fichiers de configuration comme `proxy.conf.js` et `proxy.conf.json` assurent en outre une communication transparente entre l'interface Angular et l'API locale pendant les phases de développement.

5.2 Technologies backend

Le backend de l'application repose sur **Java 21** et sur **Spring Boot 3.5.6**. Le fichier `quizapp/pom.xml` confirme ce positionnement technique à travers la propriété `java.version` fixée à 21 et l'utilisation du parent Spring Boot correspondant. Le projet est packagé au format **WAR**, ce qui permet de conserver une compatibilité de déploiement avec un serveur applicatif tout en gardant la souplesse d'un démarrage local maîtrisé.

La couche web est construite avec **Spring Web**, qui sert de socle à l'exposition des endpoints REST. À cela s'ajoutent **Spring Security** pour l'authentification et l'autorisation, **Spring WebSocket** pour les fonctionnalités temps réel, ainsi que **Spring Data JPA** pour la persistance relationnelle. La classe principale `QuizappApplication` s'appuie également sur `@EnableScheduling`, ce qui montre que l'application intègre des traitements planifiés dans son cycle d'exécution.

La persistance métier s'appuie sur **MySQL** via le connecteur `mysql-connector-j`, tandis que **Flyway** est utilisé pour gérer les migrations de schéma de manière versionnée, conformément aux bonnes pratiques documentées par Red Gate Software dans la référence officielle **Flyway Documentation** (2025). Ce choix est particulièrement important dans un projet dont le modèle de données a fortement évolué, notamment autour de la candidature, des occurrences de pipeline, des quiz, des entretiens et des notifications.

Plusieurs bibliothèques complémentaires renforcent la qualité de l'implémentation. **MapStruct** réduit le coût de transformation entre entités et objets d'échange, **Lombok** diminue le code répétitif, **Thymeleaf** est utilisé pour les modèles d'emails, et **Spring Context Support** avec **Jakarta Mail** assurent la gestion des envois. Le backend intègre également des briques spécialisées telles que **Spring LDAP Core**, **Apache POI** pour la manipulation de documents bureautiques, **MinIO** pour la gestion objet via une API compatible S3, dont la documentation officielle **MinIO Object Storage Reference Guide** (MinIO Inc., 2025) décrit l'intégration en environnement Java, **Redis** pour le cache et les données volatiles — dont J. L. Carlson, dans **Redis in Action** (2013), décrit les cas d'usage en complément d'une base relationnelle — et **Springdoc OpenAPI** pour documenter l'API.

Enfin, l'architecture serveur reste cohérente avec un projet d'ingénierie orienté maintenabilité. Le serveur Tomcat est déclaré avec le scope `provided`, la pile de sécurité est centralisée, et les règles métier sont isolées dans des classes spécialisées de la couche business. Cette organisation facilite la lisibilité du code, l'écriture de tests ciblés et l'évolution progressive du produit.

5.3 Technologies frontend

L'interface utilisateur repose sur **Angular 20.2**, comme l'indiquent les dépendances `@angular/core`, `@angular/router`, `@angular/forms` et les autres modules clés du fichier `quizapp_front/package.json`. Le projet est compilé avec **TypeScript 5.9.2**, ce qui permet de bénéficier d'un typage strict et d'une meilleure sécurisation des contrats de données entre l'interface et le backend.

Le fonctionnement réactif de l'application s'appuie sur **RxJS 7.8**, tandis que **Zone.js** reste présent pour l'intégration au modèle de détection de changements d'Angular. Le build frontend est piloté par le builder `@angular/build:application`, configuré dans `angular.json`. Les scripts `start`, `build`, `watch` et `test` permettent respectivement de lancer le serveur de développement, de produire un build final, de travailler en compilation continue et d'exécuter les tests unitaires.

L'interface est stylée en **SCSS**, choix confirmé à la fois dans les schémas Angular et dans la configuration du projet. Le cadre réactif de l'application exploite par ailleurs les **Signals**, nouveau modèle de réactivité fine introduit par Angular et documenté par Google dans la référence **Angular Signals: Fine-Grained Reactivity Model** (2025), qui permet d'optimiser le rafraîchissement du DOM sans déclencher une détection de changements globale — un atout pour les tableaux de bord de suivi en temps réel. La migration vers cette architecture moderne est également commentée par KodMatrix dans son article **Angular 20: Everything You Need to Know** (2025) comme une évolution majeure réduisant la complexité du code pour les applications à forte densité fonctionnelle. Cette approche facilite la factorisation des styles, l'expression de variations visuelles et la structuration des pages. Le frontend adopte par ailleurs une organisation par écrans métier, visible dans le dossier `src/app/pages`, avec des modules tels que `apply-position`, `dashboard`, `quiz`, `track-application` ou encore `verify-email`. Ce découpage favorise une lecture directe du produit par parcours utilisateur.

Le projet frontend intègre également une couche de composants mutualisés, un espace partagé, des services applicatifs, des directives, des pipes et des éléments de configuration. Cette structuration est cohérente avec une application monopage de taille intermédiaire à forte densité fonctionnelle, dans laquelle la réutilisation d'éléments d'interface et la cohérence des interactions constituent des enjeux centraux.

5.4 Base de données

La base de données principale de SyneliaQuizApp est une base **MySQL 8**, choisie pour sa robustesse, sa large diffusion en contexte web et son adéquation avec un modèle relation-

nel riche. Le domaine métier du recrutement manipule en effet des objets fortement liés entre eux : campagnes, formulaires, candidatures, étapes de pipeline, tentatives de quiz, entretiens, évaluations, décisions, notifications et permissions. Un SGBDR relationnel s'impose donc naturellement pour garantir l'intégrité des relations et la cohérence des transactions.

Le couplage entre le code Java et la base repose sur **JPA**, avec une organisation en entités et repositories. La présence du dossier `quizapp/src/main/java/tech/synelia/quizapp/dao/entity` d'un côté et du dossier `repository` de l'autre montre clairement cette séparation entre modélisation des tables et accès aux données. Cette approche permet d'exprimer des règles métier complexes tout en gardant un point d'entrée structuré vers la couche de persistance.

L'écosystème local n'est pas limité à MySQL. Le fichier `docker-compose.yml` montre que des services complémentaires comme **Redis 7** et **MinIO** sont également prévus dans l'environnement de développement. Redis peut servir de support aux besoins de cache ou de coordination légère, tandis que MinIO apporte une solution compatible stockage objet pour les pièces et contenus associés à l'application. Cette composition technique reflète une architecture pragmatique : un cœur transactionnel relationnel, enrichi par des services spécialisés lorsque cela apporte une valeur concrète.

5.5 Outils complémentaires

Plusieurs outils complémentaires participent à la qualité et à l'exploitabilité du projet. Côté backend, **Maven** reste l'outil de référence pour la gestion du cycle de vie, des dépendances et des tests Java. L'utilisation du wrapper Maven évite de dépendre d'une installation globale homogène sur tous les postes. Côté frontend, **npm** pilote l'installation des paquets et l'exécution des scripts Angular.

Les validations API s'appuient sur **Postman**, comme le montre le document `quizapp/POSTMAN_API_GUIDE.md`. Cet outil joue un rôle important dans un projet à forte densité d'endpoints, car il permet de rejouer rapidement des scénarios métier, de contrôler les contrats d'entrée et de sortie et de vérifier le comportement des routes publiques et privées. La présence de cartographies fonctionnelles, de notes de debug et de guides de validation montre d'ailleurs que les outils de test manuel ont été intégrés à la démarche d'ingénierie et non utilisés de manière ponctuelle.

Enfin, **Docker Compose** simplifie la mise à disposition des services auxiliaires, tandis que **Git** structure l'historique de travail et la traçabilité des évolutions. Dans le cadre de la

rédaction du mémoire, l'introduction de **Typst** complète cet environnement en fournissant une chaîne de composition adaptée à un document long, structuré et versionnable.

Couche	Technologies principales	Rôle dans SyneliaQuizApp
Backend	Java 21, Spring Boot 3.5.6	Exposition de l'API REST, sécurité, logique métier et orchestration des traitements du recrutement.
Frontend	Angular 20.2, TypeScript 5.9.2	Construction de l'interface monopage, gestion des parcours candidat et recruteur, communication avec l'API.
Données	MySQL 8, Flyway	Persistance relationnelle, évolution versionnée du schéma et maintien de la cohérence métier.
Services auxiliaires	Redis 7, MinIO, Docker Compose	Support aux besoins transverses comme le cache, le stockage objet et l'environnement d'exécution local.
Validation	Maven, npm, Postman, Typst	Build, vérifications ciblées, validation des scénarios métier et production du mémoire.

CHAPITRE 6

Implémentation de la solution

Ce chapitre détaille la manière dont la solution a été concrètement construite, tant du côté backend que du côté frontend.

6.1 Organisation du code source

Le workspace du projet est organisé autour d'une séparation nette entre les composants applicatifs et les artefacts de support. Le dossier `quizapp/` contient le backend Spring Boot, tandis que `quizapp_front/` regroupe l'application Angular. Autour de ces deux noyaux se trouvent des éléments de support comme `docker-compose.yml` et des scripts utilitaires facilitant le développement et le déploiement local.

Le backend adopte une organisation classique mais efficace pour un projet de gestion métier. Le point d'entrée applicatif est la classe `QuizappApplication`, et les responsabilités sont ensuite réparties entre plusieurs packages : `business` pour la logique métier, `dao` pour la persistance, `rest` pour l'exposition web, `security` pour la sécurité, `config` pour la configuration transverse, `utils` pour les utilitaires, et `websocket` pour les mécanismes temps réel. Cette architecture favorise un découplage clair entre exposition, orchestration métier et accès aux données.

Le frontend suit une logique équivalente de séparation des responsabilités. Le dossier `src/app` contient notamment des sous-espaces dédiés aux pages, aux composants réutilisables, au noyau applicatif, aux services et aux éléments partagés. Le dossier `pages` reflète directement les grands parcours métiers de l'application, alors que `shared` concentre les briques génériques utilisées dans plusieurs écrans, comme les panneaux de détail, les sélecteurs de filtre ou les composants de saisie plus spécialisés.

6.2 Implémentation du backend

L'implémentation backend repose sur une logique en couches. Les contrôleurs REST reçoivent les requêtes et délèguent le traitement aux classes de la couche `business`, lesquelles centralisent les règles applicatives. Cette couche métier constitue le cœur réel de l'application : elle orchestre les transitions d'état, les validations fonctionnelles, les

notifications, les interactions entre objets métier et les politiques de sécurité appliquées aux différentes opérations.

La couche de persistance est structurée autour des entités JPA et des repositories. Le package `dao` contient à la fois la modélisation des objets persistés et les interfaces d'accès aux données. Cette organisation permet de faire évoluer les règles métier sans mélanger les considérations de persistance avec la logique d'orchestration. Elle est particulièrement utile dans un contexte où les modèles de candidature, de quiz et d'entretien ont connu des évolutions importantes.

La configuration transverse est isolée dans des espaces dédiés, notamment pour la sécurité, les intercepteurs REST, les WebSockets et le comportement applicatif global. Cette structuration favorise la maintenabilité et rend plus explicite le rôle de chaque couche dans le système.

6.3 Implémentation du frontend

L'implémentation frontend s'inscrit dans le modèle d'une application monopage orientée parcours utilisateur. Angular est utilisé pour structurer l'interface en pages, composants et services, avec une séparation nette entre les écrans métier et les éléments réutilisables. Les pages couvrent les grands usages de la plateforme : candidature, quiz, suivi public, authentification, tableau de bord et administration métier.

Les composants partagés du dossier `shared` constituent un socle important de cohérence visuelle et fonctionnelle. Ils évitent de redévelopper les mêmes interactions dans plusieurs modules et facilitent l'homogénéité de l'interface. Les services applicatifs portent quant à eux les appels HTTP et les logiques transverses nécessaires au dialogue avec le backend, tandis que le noyau applicatif héberge les mécanismes plus globaux comme les guards, la configuration et certains comportements d'infrastructure côté client.

Cette architecture frontend présente un avantage majeur dans un projet comme SyneliaQuizApp : elle permet de faire évoluer une fonctionnalité locale sans remettre en cause l'ensemble de l'application, tout en conservant une expérience cohérente pour les différents profils d'utilisateurs.

Zone du dépôt	Contenu principal	Responsabilité dans la solution
quizapp/	Application Spring Boot	Centralise la logique métier, la sécurité, les accès aux données, les

		endpoints REST et les services transverses.
quizapp_front/	Application Angular	Porte l'expérience utilisateur, les formulaires, les écrans métier et les flux de navigation.
docker-compose.yml	Services d'infrastructure locale	Décrit les dépendances techniques utiles au développement et aux validations locales.

6.4 Implémentation des modules fonctionnels

L'implémentation fonctionnelle de SyneliaQuizApp ne s'est pas limitée à juxtaposer des écrans ou des endpoints isolés. Elle a consisté à faire converger plusieurs modules interdépendants autour d'un même objectif : rendre le processus de recrutement plus lisible, plus traçable et plus pilotable, depuis la publication d'une campagne jusqu'à la décision finale.

6.4.1 Gestion des campagnes et des formulaires

Le module des campagnes constitue le point d'entrée organisationnel du système. Il permet de créer une campagne, de la configurer, de lui associer un ou plusieurs formulaires et de contrôler les droits des collaborateurs qui peuvent la consulter, la modifier, suivre ses candidatures ou intervenir dans les étapes d'évaluation. Le projet ne se limite donc pas à une simple liste d'offres : la campagne devient une unité de pilotage métier qui structure les permissions, les parcours et les indicateurs.

La gestion des formulaires a été conçue avec une logique de réutilisation et de versionnement. Le rattachement entre campagne et formulaire repose sur une relation dédiée, ce qui permet de mutualiser un même formulaire entre plusieurs campagnes tout en gardant la possibilité de dupliquer ou d'isoler une version lorsqu'une divergence devient nécessaire. Les stratégies d'affectation prises en charge, comme l'attribution aléatoire, séquentielle, conditionnée par un source token ou aléatoire sans répétition, montrent que le questionnaire technique n'est pas un bloc figé mais un sous-système adaptable au contexte de recrutement.

6.4.2 Moteur de pipeline et progression des candidatures

Le pipeline de recrutement constitue l'épine dorsale du système : il définit l'ordre des étapes par lesquelles transite une candidature et conditionne les actions disponibles à chaque stade. L'implémentation repose sur une modélisation explicite des gabarits de

pipeline (`PipelineTemplate`), des étapes ordonnées (`PipelineStage`) et de la progression effective de chaque dossier (`ApplicationProgress`). Cette architecture s'inspire des principes de machine d'états et de workflow configurables, tels que décrits par Martin Fowler dans **Domain-Specific Languages** (2010) et par le standard BPMN 2.0 (Object Management Group, 2013) pour la modélisation des processus métier.

Concrètement, un gabarit de pipeline est rattaché à une campagne et décrit une succession typée d'étapes : candidature, quiz, entretien, évaluation, décision. Chaque étape possède un type, un ordre, des critères d'évaluation optionnels et une configuration dynamique stockée en `config_json`. Lorsqu'une candidature progresse, le système crée une occurrence de progression rattachée à l'étape concernée, y inscrit les métadonnées utiles (score de quiz, notes d'entretien, statut de décision) et recalcule l'état consolidé du dossier. Ce mécanisme garantit que l'historique complet du parcours candidat reste consultable, y compris en cas de retour arrière ou de reprise après annulation.

L'intérêt de cette approche dépasse la simple organisation séquentielle. Elle permet de décorréler le cycle de vie générique défini par le gabarit des comportements spécifiques portés par chaque étape. Ainsi, une étape de type quiz active automatiquement les composants de passation et de correction, tandis qu'une étape de type décision verrouille certaines actions publiques et déclenche les notifications de fin de parcours. Cette architecture par machines d'états et règles contextuelles confère au système une souplesse importante pour s'adapter à de futurs processus de recrutement sans remettre en cause le noyau applicatif.

6.4.3 Gestion des candidatures et parcours public

Le module des candidatures couvre l'ensemble du flux candidat, depuis la soumission initiale jusqu'au suivi postérieur à l'envoi. Une candidature peut être déposée à partir d'un lien public, associée à une campagne donnée, puis enrichie par des pièces et des informations métier utiles au traitement du dossier. Un mécanisme de vérification d'email sécurise ensuite l'accès aux étapes sensibles, notamment au quiz technique.

Le projet intègre également un véritable parcours public de suivi. Chaque candidature reçoit un code de suivi permettant au candidat de consulter l'état d'avancement de son dossier, de reprendre un quiz encore actif, de répondre à une proposition d'entretien ou, dans certains cas, de retirer sa candidature. Cette continuité entre l'espace public et le back-office recruteur constitue un apport important de la solution, car elle réduit les ruptures d'information et améliore la transparence vis-à-vis du candidat.

6.4.4 Quiz technique et correction

Le quiz technique représente l'un des modules les plus structurants de l'application. Il a été conçu pour prendre en charge plusieurs types de questions, un chronométrage éventuel, la randomisation selon la configuration du formulaire et une gestion fine des tentatives. Le passage du quiz n'est plus rattaché à un simple jeton global mais à une occurrence explicite de tentative, ce qui améliore la traçabilité et prépare l'application à des scénarios plus riches de reprise ou de multi-occurrence.

La correction combine deux logiques complémentaires. D'un côté, les questions objectives sont notées automatiquement à partir des réponses soumises. De l'autre, les questions nécessitant une appréciation humaine peuvent passer par un circuit de correction manuelle, avec des statuts dédiés et des permissions précises pour les recruteurs habilités. Ce compromis entre automatisation et intervention humaine est essentiel pour préserver à la fois l'efficacité opérationnelle et la pertinence de l'évaluation technique.

6.4.5 Sécurisation du quiz et intégrité des évaluations

Dans un contexte où le quiz technique joue un rôle objectivant dans la présélection des candidats, la sécurisation du processus d'évaluation constitue un enjeu à part entière. Le système intègre plusieurs mécanismes destinés à garantir l'intégrité des résultats, en s'appuyant sur les travaux de A. Nigam, R. Pasricha, T. Singh et P. Joseph, dans leur revue **A systematic review of research on cheating in online exams** (2021), qui recensent les approches de prévention et de détection de la triche dans les examens en ligne.

Le premier niveau de sécurisation repose sur un verrouillage par token. L'accès au quiz est conditionné à la possession d'un jeton de source valide et à la vérification préalable de l'adresse email du candidat, ce qui empêche les soumissions anonymes ou non authentifiées. Le second niveau concerne la traçabilité temporelle : le système enregistre l'horodatage de début de chaque tentative, le temps écoulé et un instantané horaire au moment de la soumission, ce qui permet de détecter des anomalies de durée ou des soumissions trop rapides pour être crédibles. Un troisième niveau, plus fin, porte sur la détection d'inactivité : le frontend surveille les événements utilisateur (mouvements de souris, frappes clavier, défilement) et signale au backend les périodes d'absence prolongée, offrant ainsi un indice supplémentaire pour l'appréciation de la passation par les recruteurs.

Ces dispositions ne visent pas à transformer le quiz en un dispositif de surveillance intrusive, mais à fournir un faisceau d'indices suffisant pour étayer une décision de correction manuelle ou de vérification approfondie lorsque cela s'avère nécessaire. Elles

s'inscrivent dans la philosophie plus large du projet : objectiver ce qui peut l'être, tout en conservant une place pour le jugement humain éclairé par des données fiables.

6.4.6 Entretiens, évaluations et décisions

Le module des entretiens prolonge naturellement le quiz dans le pipeline de recrutement. Il permet de planifier une convocation, de notifier le candidat, de suivre sa réponse et de repositionner l'entretien si nécessaire. L'implémentation distingue clairement l'entretien planifié de la réponse candidat, ce qui rend possible une gestion plus réaliste des propositions, refus, reports et archivages.

Les évaluations associées aux entretiens vont au-delà d'une note globale. Elles peuvent porter des critères détaillés, des pondérations et des éléments de justification, puis alimenter les agrégats utilisés dans les vues de suivi. La décision finale, qu'elle soit intermédiaire ou terminale, s'appuie ainsi sur un continuum cohérent : candidature, quiz, entretien, évaluation, décision. Cette chaîne métier renforce la lisibilité du dossier candidat pour les recruteurs et limite les pertes d'information entre étapes.

6.4.7 Notifications, permissions et pilotage

Les notifications et la gestion des permissions constituent la couche transverse qui rend l'application réellement exploitable en contexte collaboratif. Les droits ne sont pas gérés uniquement par rôle global (RBAC), mais aussi par permissions contextualisées sur une campagne ou sur une entité donnée. Cette granularité s'inspire des principes de contrôle d'accès par attributs (ABAC) formalisés par le NIST (V. C. Hu et al., **Guide to Attribute Based Access Control (ABAC)**, NIST SP 800-162, 2014), où la décision d'autorisation ne dépend plus seulement du rôle de l'utilisateur mais également de son lien avec l'entité cible et du contexte opérationnel. Cette approche permet d'ouvrir certaines actions à des recruteurs ou évaluateurs ciblés sans leur accorder un accès total à l'ensemble du système.

Sur le plan des notifications, l'architecture repose sur une logique de publication/souscription inspirée du patron de conception Observateur formalisé par E. Gamma, R. Helm, R. Johnson et J. Vlissides dans **Design Patterns** (1994). Les événements métier — nouvelle candidature, planification d'entretien, réponse candidat, décision — jouent le rôle de déclencheurs auxquels réagissent des composants spécialisés : templates d'email Thymeleaf pour les notifications externes, services de notification in-app pour les alertes internes, et journal de messagerie (`MailLog`) pour la traçabilité des envois. Cette séparation entre l'occurrence d'un événement et les actions qu'il provoque permet d'ajouter de nouveaux canaux de notification sans modifier la logique métier sous-jacente.

En parallèle, les notifications internes et les emails transactionnels assurent la circulation des informations critiques : nouvelles candidatures, convocations à entretien, retours

sur demandes candidat, décisions ou alertes d'administration. Le tableau de bord unifie enfin l'accès à ces modules en proposant une lecture opérationnelle de l'activité, des listes métier, des filtres transverses et des écrans spécialisés par domaine.

6.4.8 Traçabilité, audit et historisation

La traçabilité constitue l'un des fils conducteurs du projet et mérite d'être traitée comme un module à part entière. Dans un système où des décisions de recrutement sont prises, où des évaluations sont produites et où des permissions fines sont accordées, il ne suffit pas de stocker l'état final d'un dossier : il faut pouvoir reconstituer comment et par qui cet état a été atteint. Cette exigence rejoint les principes d'audit trail en entreprise, tels que formalisés par R. Johnston dans **Designing an Enterprise Application Audit Trail** (2019).

L'implémentation repose sur un service d'audit transverse (`AuditEventService`) capable d'intercepter les mutations significatives des entités métier. Chaque événement audité enregistre l'entité concernée, l'action réalisée, l'acteur à l'origine de l'opération, un horodatage et, surtout, une représentation JSON de l'état avant et après la modification. Ce mécanisme de snapshot permet de répondre à des questions concrètes : qui a modifié le statut de cette candidature ? Quand cette évaluation a-t-elle été saisie ? Quel était le score du candidat avant la correction manuelle ? La présence de ces informations renforce à la fois la gouvernance interne du recrutement et la confiance des utilisateurs dans le système.

Au-delà de l'audit des entités, la traçabilité s'étend à d'autres dimensions. Les logs de messagerie (`MailLog`) conservent l'historique des envois avec leur statut de délivrance, ce qui permet de diagnostiquer un défaut de communication avec un candidat. Les progressions de pipeline (`ApplicationProgress`) matérialisent chaque franchissement d'étape avec les métadonnées associées. Enfin, le versionnement des formulaires garantit qu'une campagne clôturée reste lisible dans son contexte d'origine, même si le questionnaire a évolué depuis. Cette approche multicouche de la traçabilité transforme la contrainte réglementaire en avantage fonctionnel : le système ne se contente pas d'exécuter un processus, il le documente.

6.5 Intégration et communication

La communication entre le frontend Angular et le backend Spring Boot repose sur une architecture API clairement séparée. Le frontend agit comme client de l'application et délèguer l'essentiel des traitements métier au serveur via des appels HTTP centralisés dans des services applicatifs. En développement local, cette communication est simplifiée

par les fichiers de proxy Angular, qui redirigent les requêtes vers l'API tout en évitant les frictions liées au CORS lors des itérations quotidiennes.

Les échanges de données se font principalement au format **JSON**, à travers des DTO qui stabilisent les contrats entre l'interface et les couches métier. Ce choix facilite la sérialisation, la validation côté client et la réutilisation des mêmes objets dans plusieurs écrans. Le projet suit par ailleurs une convention homogène d'API centrée sur des endpoints explicites, majoritairement exposés sous forme d'actions métier plutôt que comme simples opérations CRUD génériques. Cette approche se justifie par la richesse des cas d'usage : avancer une candidature, répondre à un entretien, publier une version de formulaire ou recalculer une évaluation ne relèvent pas tous d'une simple mise à jour standard.

Les flux de fichiers, en particulier autour des candidatures et des pièces associées, sont pris en charge par des points d'entrée dédiés. La solution sépare ainsi la donnée métier visible dans les écrans, comme le statut d'une candidature ou le lien de suivi public, des ressources binaires effectivement téléversées. Cette dissociation est importante car elle permet de faire évoluer le stockage sans remettre en cause l'interface fonctionnelle exposée aux utilisateurs.

Sur le plan de la synchronisation, l'application repose moins sur un rafraîchissement brut de toutes les données que sur des contrats ciblés et des miroirs métier calculés côté backend. Par exemple, certaines informations de synthèse sur l'état courant d'une candidature, d'un entretien ou d'une évaluation sont recalculées puis renvoyées pour faciliter l'affichage dans les listes et tableaux de bord. Cette stratégie améliore la réactivité perçue du front tout en conservant un centre de vérité unique côté serveur. Elle est complétée par des mécanismes transverses comme les notifications, les WebSockets et les emails, qui assurent la diffusion rapide d'informations importantes aux différents acteurs du processus.

6.5.1 Communication temps réel et tableaux de bord synoptiques

Au-delà des échanges HTTP classiques, SyneliaQuizApp intègre une couche de communication temps réel fondée sur les WebSockets. Cette brique technique, documentée notamment par Y. Fain et A. Moiseev dans **Angular Development with TypeScript** (2e édition, 2018) pour les architectures combinant un backend Java et un frontend Angular, permet d'établir un canal bidirectionnel persistant entre le serveur et les clients connectés.

L'usage principal de ce canal concerne le module QEEL (tableau de bord temps réel), qui affiche des statistiques actualisées sur l'activité de la plateforme : nombre

d'utilisateurs en ligne, candidatures en cours, quiz actifs et autres indicateurs de supervision. Côté backend, un handler WebSocket dédié gère les connexions au sein d'un pool thread-safe et diffuse les mises à jour aux clients abonnés. Côté frontend, un service de statistiques en ligne maintient la connexion et bascule automatiquement sur un mécanisme de polling HTTP en cas d'indisponibilité du canal WebSocket, garantissant ainsi une continuité de service.

Un mécanisme de heartbeat complète ce dispositif : le client émet périodiquement un signal de présence, ce qui permet au serveur de distinguer les utilisateurs réellement actifs des connexions abandonnées. Cette approche, simple dans son principe, améliore significativement la fiabilité des indicateurs temps réel sans introduire de complexité excessive.

6.6 Exécution locale et chaîne DevOps

Au-delà du code lui-même, la maintenabilité d'un projet de la taille de SyneliaQuizApp dépend très fortement de la reproductibilité de son environnement d'exécution. Le dépôt embarque pour cela un fichier `docker-compose.yml` qui décrit deux services d'infrastructure indispensables : une instance **Redis 7** (image `redis:7-alpine`, port `6379`) et une instance **MinIO** exposée sur les ports `9000` (API S3) et `9001` (console d'administration), associées à des volumes persistants `redis_data` et `minio_data`. Chaque service déclare un *healthcheck* (`redis-cli ping`, `mc ready local`), ce qui permet à un orchestrateur de connaître l'état réel des dépendances avant de lancer l'application.

La base **MySQL** n'est pas embarquée dans la même composition : elle est consommée comme un service externe paramétrable via les propriétés Spring (`spring.datasource.url`, `spring.flyway.*`). Ce choix simplifie la transition vers un environnement de production où la base est généralement managée séparément, tout en autorisant en développement un démarrage MySQL standalone ou conteneurisé selon les préférences du développeur. L'isolement des dépendances via Docker Compose, couplé à une base externalisée, s'inscrit dans la logique de livraison continue défendue par J. Humble et D. Farley dans **Continuous Delivery** (2010) : un environnement reproductible, versionné et indépendant de la machine hôte.

Le démarrage local suit une chaîne reproductible. Le backend est lancé via `mvnw spring-boot:run` (ou via le script `start-backend.bat`), Flyway applique automatiquement les migrations versionnées, puis le frontend est servi par `npm start` qui utilise les fichiers `proxy.conf.js` / `proxy.conf.json` pour rediriger `/api/**` vers le backend local. Cette séparation explicite — services d'infrastructure dans Docker Compose, backend

et frontend exécutés directement sur la machine hôte — privilégie une boucle de retour rapide tout en restant proche d'une topologie de production.

Service	Port(s)	Rôle dans l'exécution locale
Redis 7	6379	Support de cache et de coordination légère pour les composants transverses (sessions, données volatiles, mécanismes temps réel).
MinIO	9000, 9001	Stockage objet compatible S3 pour les pièces jointes et contenus binaires, avec console d'administration accessible en HTTP.
MySQL 8 (externe)	3306	Base de données relationnelle pivot, gouvernée par les migrations Flyway versionnées du backend.
Spring Boot	8080	API REST, sécurité, orchestration métier et envoi des notifications.
Angular dev-server	4200	Interface utilisateur servie en mode HMR, proxifiée vers l'API backend.

6.7 Démarche qualité et boucle de validation continue

La taille du dépôt — backend Spring Boot multi-packages, frontend Angular structuré en pages et composants, plus de cent migrations Flyway successives, ainsi que des dossiers volumineux de cartographie fonctionnelle et de notes techniques — impose une discipline de validation continue plutôt qu'une recette unique en fin de cycle. Trois pratiques structurantes ont été instituées tout au long du projet.

La première pratique est la **cartographie vivante**. Avant chaque évolution significative, un document fonctionnel est rédigé dans un dossier dédié pour décrire l'intention métier, les impacts sur les écrans, les routes API, le modèle de données et les migrations Flyway attendues. Ce document sert ensuite de contrat partagé entre l'analyse fonctionnelle, l'implémentation et la recette, ce qui réduit fortement le risque d'écart entre la spécification et le code livré.

La deuxième pratique est le **journal de fixes** capitalisé dans un dossier dédié. Chaque correctif non trivial fait l'objet d'une documentation décrivant le symptôme, la cause racine, la correction appliquée et le scénario de validation. Cette mémoire opérationnelle

permet de comprendre la genèse d'une zone de code, de rejouer un test manuel et d'éviter qu'une régression réintroduise un défaut déjà traité.

La troisième pratique est la **validation outillée**. Côté backend, les commandes `mvnw test` et `mvnw verify` sont rejouées avant tout merge significatif pour garantir la compilation, la cohérence Flyway et la non-régression des tests de slice. Côté frontend, l'enchaînement `npm run build` et `npm test -- --watch=false --browsers=ChromeHeadless` sécurise la compilation TypeScript, la résolution des règles SCSS et l'exécution des specs Karma sur un navigateur headless. Cette double boucle est complétée par des collections Postman documentées qui rejouent les scénarios d'API critiques.

Pratique	Artefact	Bénéfice principal
Cartographie vivante	`cartographie-fonctionnelle-front/evolution/`	Aligner spécification, code et recette autour d'un même document métier.
Journal de fixes	`z_notes/FIX_*.md`, `z_notes/DEBUG_*.md`	Capitaliser les causes racines et prévenir les régressions sur les zones sensibles.
Validation backend	`mvnw test`, `mvnw verify`	Sécuriser la compilation, les migrations Flyway et les tests de slice.
Validation frontend	`npm run build`, specs Karma headless	Vérifier le typage strict, le build SCSS et le comportement des composants critiques.
Validation API	Collections Postman documentées	Rejouer rapidement les scénarios métier de bout en bout sur l'API REST.

CHAPITRE 7

Tests et validation

Ce chapitre démontre la rigueur apportée à la vérification du système et à la validation des comportements attendus.

7.1 Stratégie de tests

La stratégie de validation adoptée dans le projet repose sur une combinaison pragmatique de vérifications automatiques et de contrôles manuels guidés par les parcours métier. L'objectif n'était pas de mettre en place une usine à tests exhaustive dès le départ, mais de sécuriser progressivement les zones les plus sensibles du système : permissions, contrats d'API, évolutions du modèle de données, parcours candidat et opérations de recrutement à forte valeur métier.

Cette stratégie s'organise selon plusieurs niveaux, dans l'esprit de la pyramide des tests et des patterns de validation formalisés par G. Meszaros dans **xUnit Test Patterns** (2007). Le premier niveau vérifie la stabilité technique du code par la compilation, les builds et les tests unitaires ciblés. Le deuxième niveau sécurise les interactions entre couches, notamment via des tests de contrôleurs, des validations de contrats JSON et des scénarios API rejoués avec Postman. Le troisième niveau correspond aux validations fonctionnelles menées sur l'interface et sur le parcours public, afin de s'assurer que les enchaînements réels restent cohérents du point de vue utilisateur.

Niveau	Support principal	Objectif de validation
Technique	Compilation, build, tests ciblés	Détecter rapidement une rupture de code, un contrat cassé ou une régression locale.
API	Tests backend et Postman	Vérifier les contrats HTTP, les règles métier et les réponses des endpoints critiques.
Interface	Specs Angular et vérifications UI	Contrôler les composants partagés, les écrans sensibles et les comportements attendus côté front.

Fonctionnel	Scénarios métier complets	Valider le parcours candidat, recruteur et administrateur dans des conditions proches de l'usage réel.

7.2 Tests unitaires

Le backend dispose d'un ensemble réel de tests unitaires et de tests de slice couvrant plusieurs zones importantes de l'application. On y trouve notamment des tests orientés métier sur les permissions, les formulaires, les questions, les entretiens, les évaluations, les tokens de source ou encore certains comportements de repository. Des tests de contrat viennent également sécuriser la sérialisation JSON de certains DTO et la cohérence d'endpoints exposés par les contrôleurs REST.

Cette couverture n'est pas uniforme sur toute l'application, mais elle se concentre sur les zones où une évolution peut facilement produire un effet de bord significatif. C'est particulièrement vrai pour les permissions contextualisées, la logique de versionnement des formulaires, les critères d'évaluation et les flux d'entretien. Ce ciblage traduit une approche réaliste : investir les tests là où le coût d'une régression serait élevé.

Côté frontend, plusieurs pages et composants disposent également de fichiers `.spec.ts`, en particulier autour des listes métier du dashboard, des modules de campagne, de décision, d'entretien, d'évaluation et des composants partagés de filtre. Ces tests restent plus modestes que ceux du backend, mais ils jouent un rôle utile pour verrouiller certains comportements d'interface, notamment lorsque des composants mutualisés sont réutilisés dans de nombreuses vues.

7.3 Tests d'intégration

Les validations d'intégration ont principalement porté sur les endpoints et sur la cohérence des échanges entre frontend et backend. Dans ce cadre, Postman a servi d'outil de référence pour rejouer rapidement des scénarios métier : création de campagne, progression de candidature, récupération d'un quiz public, réponse à un entretien, envoi ou consultation de notifications, etc. Cette pratique est particulièrement adaptée à un projet dont plusieurs flux sont publics, tokenisés ou soumis à des permissions fines.

Les tests de contrôleurs présents côté backend jouent aussi un rôle d'intégration partielle, car ils vérifient la bonne réception des payloads HTTP, leur désérialisation et la propagation correcte vers la couche métier. Ils sont complétés, en développement courant, par des builds ciblés du backend et du frontend afin de sécuriser la compatibilité globale du dépôt après chaque série de corrections.

Enfin, l'intégration ne concerne pas seulement l'API. Elle inclut également les dépendances transverses comme la base MySQL, les migrations Flyway, les proxies frontend, la configuration SMTP active, ainsi que les services complémentaires comme Redis ou MinIO lorsqu'ils sont mobilisés par un scénario. Cette vision élargie permet de vérifier que l'application reste cohérente comme système et non comme simple somme de modules indépendants.

7.4 Tests fonctionnels

Les tests fonctionnels ont été organisés autour des parcours réels des trois grandes catégories d'utilisateurs : candidat, recruteur et administrateur. Pour le candidat, les scénarios essentiels couvrent la soumission d'une candidature, la vérification d'email, l'accès au quiz, la reprise éventuelle d'une tentative, la consultation du suivi public et la réponse à une proposition d'entretien. Ces scénarios permettent de contrôler la fluidité du parcours externe, qui constitue la façade la plus visible du système.

Pour le recruteur, les validations ont surtout porté sur la création et la configuration des campagnes, l'affectation des formulaires, la consultation des candidatures, la planification des entretiens, la correction éventuelle des réponses ouvertes, la saisie des évaluations et la prise de décision. La cohérence des permissions a fait l'objet d'une attention particulière, car une même page peut exposer des actions différentes selon les droits globaux ou contextuels de l'utilisateur.

Enfin, les tests administrateur ont concerné les données de référence, les rôles et permissions, la configuration applicative, les modèles d'email, les réglages SMTP et certaines vues techniques du dashboard. Ces validations sont importantes car elles conditionnent indirectement le bon fonctionnement des flux métier. Une application de recrutement peut paraître correcte côté utilisateur final tout en restant fragile si sa configuration transverse n'est pas fiable.

7.5 Tests de performance

Le projet n'a pas donné lieu à une campagne industrielle de tests de charge au sens strict. Il serait donc excessif de prétendre disposer de mesures complètes de montée en charge, de latence sous forte concurrence ou de benchmarks comparatifs. En revanche, plusieurs choix d'architecture et plusieurs contrôles empiriques ont visé à préserver un niveau de performance cohérent avec le contexte d'usage attendu.

D'abord, la séparation frontend/backend permet de répartir proprement les responsabilités de calcul et de présentation. Ensuite, l'usage de DTO ciblés, de filtres de recherche,

de vues de synthèse et d'agrégats calculés évite de surcharger inutilement l'interface avec des graphes de données trop volumineux. Enfin, l'existence de services auxiliaires dédiés, comme Redis ou MinIO, ouvre la voie à une meilleure spécialisation des charges lorsque le produit devra être davantage industrialisé.

Dans le contexte du mémoire, il est donc plus juste de parler d'une *performance raisonnablement maîtrisée* que d'une performance formellement benchmarkée. La priorité a été donnée à la justesse fonctionnelle, à la stabilité des parcours et à la qualité de structuration, tout en laissant ouverte la possibilité d'une optimisation plus poussée lors d'une phase ultérieure de mise en production élargie.

7.6 Tests de sécurité

La sécurité a été traitée à plusieurs niveaux. Le premier niveau concerne l'authentification et l'autorisation, assurées côté backend par Spring Security et prolongées par une logique métier fine de contrôle des permissions. L'application distingue en effet les droits globaux, liés au rôle système, des droits contextuels accordés sur une campagne ou une entité particulière. Cette granularité réduit le risque d'accès excessif et correspond mieux à la réalité collaborative d'un processus de recrutement.

Le deuxième niveau relève des validations d'entrée et de la protection des parcours publics. L'accès au quiz dépend notamment de la vérification d'email, certaines actions publiques sont verrouillées en fonction de l'état de la candidature, et les liens de suivi s'appuient sur des codes dédiés plutôt que sur des identifiants exposés directement. Côté backend, plusieurs garde-fous évitent également de faire confiance à des informations transportées par le client lorsqu'une source authentifiée plus sûre est disponible, comme le principal JWT résolu côté serveur.

Enfin, la sécurité comprend aussi la protection des données sensibles et des services transverses. Les secrets SMTP restent traités comme des données d'écriture non relues en clair, les templates d'email échouent volontairement lorsqu'une variable obligatoire est absente, et les notifications d'administration signalent les défauts de configuration ou d'envoi au lieu de les masquer. Cette approche favorise une sécurité opérationnelle plus réaliste, centrée non seulement sur l'accès, mais aussi sur la fiabilité des traitements critiques.

7.7 Recettage et validation finale

La recette finale a suivi une logique incrémentale. Chaque évolution significative du projet a été confrontée à un petit ensemble de critères de validation : absence d'erreur

de build, cohérence des contrats API, conformité du parcours utilisateur attendu, et non-régression sur les règles métier les plus sensibles. Cette méthode a permis de corriger rapidement les écarts au fil du développement, sans attendre une phase terminale trop lourde et trop risquée.

Les notes techniques, cartographies, guides Postman et check-lists produits pendant le projet ont joué un rôle important dans cette recette. Ils ont servi de mémoire opérationnelle pour rejouer des cas, comprendre l'origine d'un défaut et vérifier qu'un correctif n'en introduisait pas un autre ailleurs. Le recettage ne s'est donc pas limité à cocher des fonctionnalités présentes : il a consisté à confirmer que les comportements livrés restaient cohérents avec le besoin métier et avec les arbitrages pris en cours de projet.

Quatrième partie

Résultats et perspectives

Bilan du projet, valeur créée et axes d'amélioration futurs

CHAPITRE 8

Présentation des résultats

Ce chapitre a pour objectif de mettre en évidence les résultats effectivement obtenus à l'issue du projet, du point de vue fonctionnel, technique et méthodologique.

8.1 Démonstration de l'application

La démonstration de l'application peut être structurée autour de trois parcours complémentaires. Le premier est celui de l'administrateur, qui configure les référentiels, gère les droits, supervise les paramètres applicatifs et maintient les éléments transverses comme les modèles d'email ou la configuration SMTP. Ce parcours montre que la solution ne se limite pas à une interface métier, mais propose également les leviers d'administration nécessaires à une exploitation durable.

Le deuxième parcours est celui du recruteur. Il couvre la création d'une campagne, l'association d'un ou plusieurs formulaires, le partage d'accès avec d'autres intervenants, le suivi des candidatures, la planification des entretiens, la consultation des évaluations et la prise de décision. C'est dans ce parcours que la valeur métier de SyneliaQuizApp apparaît le plus clairement : l'information est regroupée, contextualisée et pilotable sans dépendre d'outils disparates.

Le troisième parcours est celui du candidat. Il comprend la soumission de la candidature, la vérification d'email, l'accès au quiz, le suivi public de l'état d'avancement et les interactions autour des entretiens. Ce parcours est particulièrement important dans une logique de qualité de service, car il traduit concrètement la manière dont l'entreprise communique avec ses candidats et structure leur expérience.

Les figures suivantes présentent trois écrans représentatifs de l'application livrée : la page d'accueil publique, le tableau de bord de synthèse (overview) et l'écran de gestion des campagnes de recrutement.

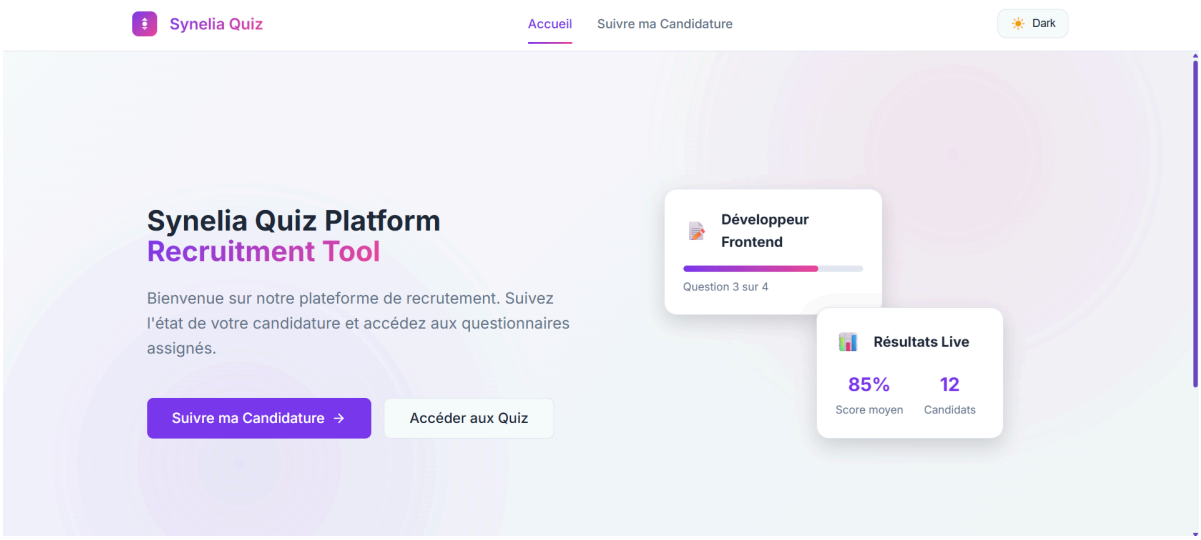


Fig. 35. – Page d'accueil publique de SyneliaQuizApp — présentation des campagnes ouvertes, accès au dépôt de candidature et au suivi candidat.

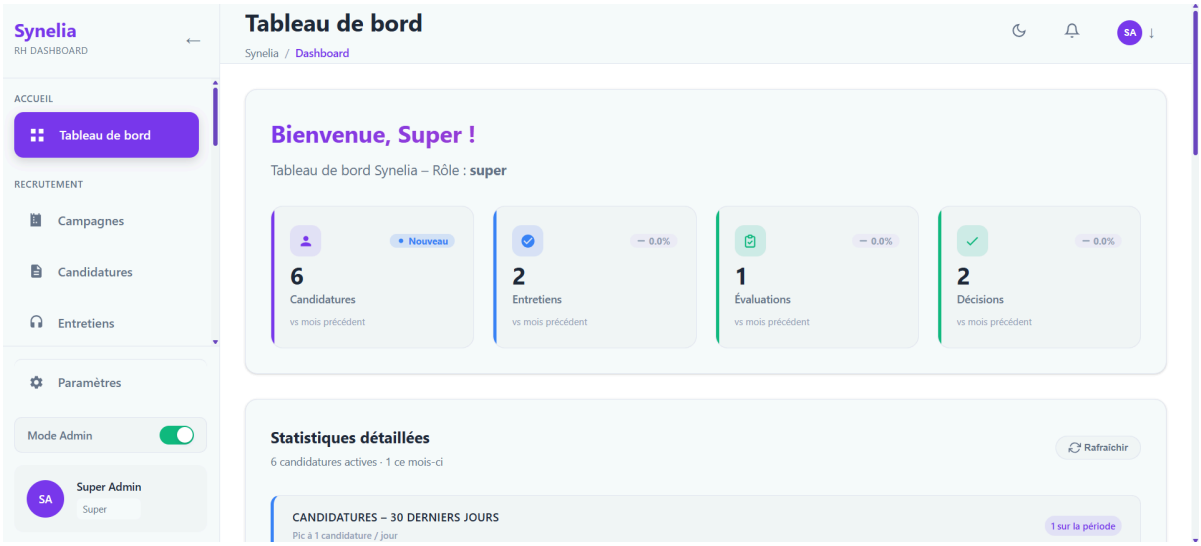


Fig. 36. – Tableau de bord administrateur / recruteur — synthèse des indicateurs clés, tendances de candidatures, entonnoir de conversion et distribution des évaluations.

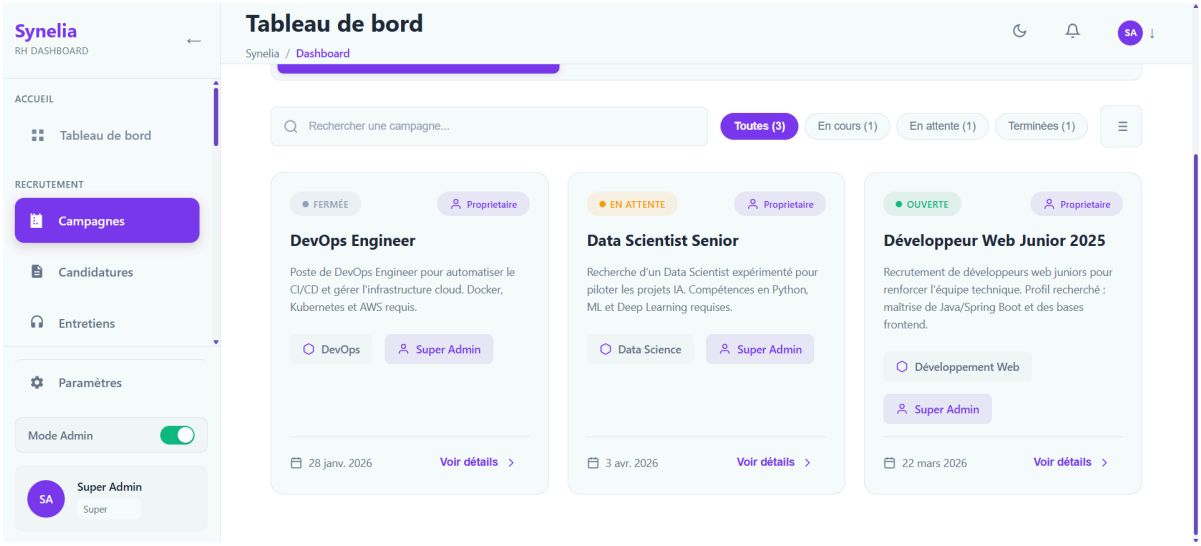


Fig. 37. – Écran de gestion des campagnes de recrutement — création, configuration, partage d'accès et suivi des campagnes actives.

8.2 Fonctionnalités implémentées

Le tableau suivant résume les principales fonctionnalités effectivement prises en charge par la solution au terme du projet.

Module	Niveau de couverture	Synthèse
Campagnes et formulaires	Élevé	Création, configuration, partage d'accès, association de formulaires, gestion de versions et stratégies d'affectation.
Candidatures	Élevé	Soumission publique, vérification d'email, suivi candidat, retrait, consultation et traitement côté recruteur.
Quiz technique	Élevé	Tentatives, chronométrage, randomisation, reprise conditionnelle, notation automatique et correction manuelle partielle.
Entretiens	Élevé	Planification, réponse candidat, reprogrammation, archivage et suivi par occurrence.
Évaluations et décisions	Élevé	Critères détaillés, agrégats d'évaluation, décisions intermédiaires et finales avec notifications associées.

Administration transverse	Moyen à élevé	Permissions, configuration applicative, emails, SMTP, référentiels et vues techniques de supervision.

Cette couverture montre que le projet dépasse largement le stade d'un prototype d'interface. La plateforme prend en charge le cœur du processus de recrutement, avec une densité fonctionnelle suffisante pour soutenir un usage réel, même si certaines zones restent perfectibles ou méritent une industrialisation plus poussée.

8.3 Bilan technique

Sur le plan technique, le projet aboutit à une solution cohérente, fondée sur une séparation nette entre backend Spring Boot, frontend Angular et base relationnelle MySQL pilotée par migrations. Cette architecture présente un bon compromis entre robustesse, lisibilité et capacité d'évolution. Les principaux composants structurants du système sont identifiables : gestion centralisée des permissions, versionnement des formulaires, orchestration des parcours candidat, synchronisation des états métier et administration transverse du produit.

La qualité du code ne doit pas être évaluée uniquement à travers des indicateurs quantitatifs. Dans le cas présent, elle se manifeste surtout par la clarification progressive des responsabilités, la formalisation d'un nombre croissant de règles métier, l'introduction de tests ciblés sur les zones sensibles et la réduction de certains héritages techniques devenus ambiguës. Le projet conserve encore des marges d'amélioration, mais il dispose désormais d'une base technique suffisamment structurée pour accueillir de nouvelles itérations sans repartir de zéro.

Les performances observées dans le cadre de développement et de validation restent compatibles avec un usage interne raisonnable. Les parcours métier essentiels peuvent être exécutés localement de manière fluide, les builds sont reproductibles, et les dépendances d'infrastructure sont suffisamment explicites pour permettre une montée en qualité future. Ce constat reste volontairement prudent : il reflète un produit techniquement crédible, mais encore perfectible sur l'industrialisation avancée.

8.4 Retour d'expérience

Le projet a mis en évidence une réalité fréquente des applications métier : la complexité ne vient pas seulement du volume de code, mais surtout de la finesse des règles à articuler. Les difficultés majeures ont concerné la stabilité des parcours lorsqu'une même donnée devait être visible sous plusieurs angles, par exemple dans les listes, les écrans de détail,

les flux publics et les agrégats de pilotage. Les notions de permission contextuelle, de versionnement des formulaires, d'occurrences d'entretien ou de tentatives de quiz ont exigé un travail d'alignement progressif entre base de données, backend et interface.

Parmi les décisions importantes prises pendant le projet figure la volonté de recentrer plusieurs comportements autour de sources de vérité plus explicites. Cela se traduit par l'usage accru des permissions d'entité, par la formalisation des tentatives de quiz, par la séparation entre formulaires partagés et versions publiées, ou encore par l'amélioration des contrats d'email et de notification. Ces choix ont parfois demandé davantage d'effort à court terme, mais ils renforcent la lisibilité et la solidité de la solution à moyen terme.

D'un point de vue méthodologique, le stage a également confirmé l'intérêt d'une documentation vivante : notes de debug, cartographies fonctionnelles, guides de test, backlog technique et synthèses de décision. Dans un projet riche en règles métier, cette capitalisation devient un levier de qualité à part entière, car elle facilite autant la correction des anomalies que la rédaction du mémoire lui-même.

CHAPITRE 9

Limites et perspectives

Ce dernier chapitre du corps du mémoire permet d'adopter un recul critique sur la solution réalisée et d'ouvrir vers des évolutions réalistes.

9.1 Limites du travail réalisé

Malgré les résultats obtenus, plusieurs limites demeurent. La première tient au niveau d'industrialisation des validations : le projet dispose de tests ciblés et de recettes sérieuses, mais pas encore d'une couverture automatisée exhaustive ni d'une chaîne de validation continue complète sur l'ensemble des parcours. Cette limite est classique dans un projet de stage, mais elle doit être reconnue car elle conditionne la vitesse et la sécurité des évolutions futures.

La deuxième limite concerne la maturité de certaines zones transverses. Les flux d'email, de notification, de configuration et certaines vues administratives ont fortement progressé, mais restent plus sensibles aux écarts de paramétrage que les parcours cœur. De même, l'application contient encore des mécanismes hérités ou des compatibilités temporaires qui ont été conservés pour accompagner la transition d'un modèle plus ancien vers une architecture plus propre.

Enfin, la solution n'a pas encore fait l'objet d'une mise à l'échelle complète dans un environnement de production exigeant. Les performances ont été jugées satisfaisantes dans le cadre du développement et des validations locales, mais la supervision, l'observabilité, le déploiement continu et les garanties de disponibilité devront être approfondis avant une généralisation à plus grande échelle.

9.2 Perspectives d'amélioration

Les perspectives d'amélioration sont nombreuses et crédibles. Sur le plan technique, une première priorité serait de renforcer la chaîne CI/CD, d'automatiser davantage les builds et d'étendre la couverture de tests sur les parcours les plus critiques, notamment ceux qui combinent permissions, états de candidature et interactions publiques. Une deuxième priorité porterait sur l'observabilité : journalisation métier enrichie, tableaux de bord de suivi, alerting sur les flux email ou sur les erreurs applicatives.

Sur le plan fonctionnel, l'application pourrait évoluer vers une meilleure exploitation analytique du recrutement, avec des indicateurs plus riches sur les taux de conversion, les délais de traitement, les performances des formulaires ou l'efficacité des campagnes. L'expérience utilisateur pourrait également être améliorée par une intégration plus poussée des calendriers, des interfaces de suivi encore plus pédagogiques pour les candidats et des outils de collaboration avancés pour les recruteurs.

Enfin, des évolutions structurantes pourraient concerner le déploiement et l'ouverture de la plateforme : séparation plus poussée des environnements, normalisation des procédures d'administration, documentation opérationnelle renforcée et préparation à une exploitation plus large au sein de l'entreprise.

9.3 Recommandations

Au regard des résultats obtenus, il est recommandé à SYNELIA de considérer Synelia-QuizApp comme une base applicative sérieuse à consolider, plutôt que comme un simple prototype ponctuel. Cette consolidation devrait passer par la stabilisation des règles métier les plus structurantes, la formalisation des priorités produit et la réduction progressive des mécanismes de compatibilité devenus moins utiles.

Il est également conseillé d'investir dans trois axes simultanés : la qualité logicielle, l'exploitation opérationnelle et l'expérience utilisateur. La qualité logicielle suppose davantage d'automatisation de test et de contrôle ; l'exploitation opérationnelle suppose des procédures claires pour la configuration, les déploiements et la supervision ; l'expérience utilisateur suppose d'améliorer encore la lisibilité des parcours pour les recruteurs comme pour les candidats.

Enfin, pour les futures évolutions, il conviendra de préserver le principe qui a le plus apporté de valeur au projet : faire reposer les choix techniques sur une lecture fine du besoin métier. C'est cette articulation entre rigueur logicielle et réalité du recrutement qui donnera à la solution sa pertinence durable.

Conclusion

Le présent mémoire avait pour ambition de répondre à une problématique concrète : comment concevoir une plateforme unifiée capable de digitaliser le recrutement, de standardiser l'évaluation technique et de renforcer la traçabilité des décisions, tout en restant adaptée aux contraintes opérationnelles d'une entreprise en croissance ? Cette question, posée dans le contexte réel de SYNELIA, traduisait un besoin de transformation profonde d'un processus métier encore trop fragmenté entre emails, fichiers de suivi et pratiques peu standardisées.

Pour y répondre, une démarche d'ingénierie logicielle complète a été déployée en quatre temps. La phase d'analyse a permis de formaliser les besoins métier, les règles de gestion et les flux du recrutement à partir du cahier des charges et de l'observation des pratiques existantes. La phase de conception a structuré l'architecture en couches, modélisé la base de données selon la méthode Merise et défini les contrats d'API REST. La réalisation a été menée de manière incrémentale en s'appuyant sur Spring Boot pour le backend, Angular pour le frontend et MySQL pour la persistance, avec des migrations versionnées via Flyway. Enfin, une phase de validation continue a permis de confronter la solution aux exigences fonctionnelles et de corriger les écarts au fil du développement.

Les résultats obtenus sont substantiels. La plateforme SyneliaQuizApp couvre aujourd'hui l'ensemble du cycle de recrutement : création et configuration des campagnes, soumission et suivi des candidatures, évaluation technique par quiz automatisé avec chronométrage et correction manuelle, planification et gestion des entretiens, évaluations collaboratives, prise de décision et notifications. L'application distingue quatre profils d'utilisateurs — candidat, recruteur, administrateur et super administrateur — avec des permissions fines combinant rôles système et droits contextualisés par entité. Une interface d'administration transverse complète le dispositif en offrant les leviers de paramétrage, de gouvernance et de supervision nécessaires à une exploitation durable.

Les contributions principales de ce travail se situent à plusieurs niveaux. Sur le plan fonctionnel, le projet apporte à SYNELIA une solution métier unifiée qui remplace avantageusement l'éparpillement antérieur des outils et des pratiques. Sur le plan technique, il démontre la pertinence d'une architecture en couches fondée sur des frameworks modernes et opinionnés, capable de soutenir un domaine métier riche en règles de gestion sans sacrifier la maintenabilité. Sur le plan méthodologique, il illustre comment une

documentation vivante — cartographies fonctionnelles, journaux de correctifs, guides de validation — peut servir de levier de qualité tout au long du cycle de vie d'un projet.

Ce travail comporte néanmoins des limites qu'il convient de reconnaître. La couverture de tests automatisés reste ciblée sur les zones les plus sensibles et mériterait d'être étendue à l'ensemble des parcours critiques. Certaines zones transverses, notamment la gestion des emails, des notifications et de la configuration, demeurent plus sensibles aux écarts de paramétrage que les modules cœur. Enfin, la solution n'a pas encore été confrontée à un environnement de production à grande échelle, ce qui laisse ouverte la question de la montée en charge, de l'observabilité et du déploiement continu.

Les perspectives ouvertes par ce projet sont nombreuses. Sur le plan technique, le renforcement de la chaîne CI/CD, l'automatisation accrue des tests et la mise en place d'une supervision opérationnelle constitueraient des priorités naturelles. Sur le plan fonctionnel, l'enrichissement des indicateurs analytiques, l'amélioration de l'expérience candidat et l'intégration de nouveaux canaux de diffusion renforceraient la valeur d'usage de la plateforme. À plus long terme, l'ouverture vers des modules complémentaires — matching avancé, intégration avec des job boards, application mobile — pourrait élargir le périmètre sans remettre en cause les fondations techniques posées par ce travail.

Au terme de ce mémoire, il apparaît que la transformation numérique d'un processus de recrutement ne se résume pas à l'écriture d'un logiciel. Elle exige une lecture fine des besoins métier, une architecture capable d'absorber la complexité sans la subir, et une démarche de validation ancrée dans le réel. C'est cette triple exigence que le projet SyneliaQuizApp s'est efforcé de satisfaire, en proposant à SYNELIA bien plus qu'un outil : un système structuré, évolutif et porteur de valeur pour l'ensemble de ses parties prenantes.

Références bibliographiques

Ouvrages et articles scientifiques

Digitalisation RH et gestion du recrutement

- Bondarouk, T. & Olivas-Luján, M. R. (2013). **Electronic HRM in the Smart Era**. Emerald Group Publishing.
  Cité p. 6 (Introduction), p. Chapitre 1.1.4 (§Nécessité d'une solution digitale).
- Stone, D. L., Deadrick, D. L., Lukaszewski, K. M. & Johnson, R. (2015). The influence of technology on the future of human resource management. **Human Resource Management Review**, 25(2), 216-231.
  Cité p. Chapitre 1.1.3 (§Défis et limitations actuels).
- Chapman, D. S. & Webster, J. (2003). The use of technologies in the recruiting, selection, and evaluation of job applicants. **Journal of Management**, 29(4), 519-543.
  Cité p. Chapitre 1.1.2 (§Processus de recrutement traditionnel).

Architecture logicielle et développement web

- Walls, C. (2022). **Spring in Action** (6e édition). Manning Publications.
  Cité p. Chapitre 2.4.2 (§Choix du framework backend).
- Freeman, A. (2023). **Pro Angular 16**. Apress.
  Cité p. Chapitre 2.4.3 (§Choix du framework frontend).
- Fielding, R. T. (2000). **Architectural Styles and the Design of Network-based Software Architectures**. Thèse de doctorat, University of California, Irvine.
  Cité p. Chapitre 4.5 (§Conception de l'API REST).
- Spilca, L. (2024). **Spring Start Here** (2e édition). Manning Publications.
  Cité p. Chapitre 5.2 (§Technologies backend) — référence contextuelle.
- Fain, Y. & Moiseev, A. (2018). **Angular Development with TypeScript** (2e édition). Manning Publications.
  Cité p. Chapitre 6.5.1 (§Communication temps réel et WebSocket).

Méthodologie d'ingénierie et conception

- Larman, C. (2005). **UML et les design patterns**. Pearson Education.
  Cité p. Chapitre 3.1 (§Méthodologie d'analyse).
- Martin, R. C. (2017). **Clean Architecture**. Prentice Hall.
  Cité p. Chapitre 4.1 (§Architecture globale du système).
- Fowler, M. (2002). **Patterns of Enterprise Application Architecture**. Addison-Wesley.
  Cité p. Chapitre 5.2 (§Technologies backend — DTO/MapStruct).
- Fowler, M. (2010). **Domain-Specific Languages**. Addison-Wesley Professional.
  Cité p. Chapitre 6.4.2 (§Moteur de pipeline et progression).
- Massé, M. (2011). **REST API Design Rulebook**. O'Reilly Media.
  Cité p. Chapitre 4.5 (§Conception de l'API REST).

Sécurité des systèmes d'information

- Spilca, L. (2024). **Spring Security in Action** (2e édition). Manning Publications.
  Cité p. Chapitre 3.3 (§Besoins non fonctionnels — sécurité).
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). **Design Patterns**. Addison-Wesley.
  Cité p. Chapitre 6.4.7 (§Notifications — pattern Observateur).

Modélisation métier — Méthode Merise

- Tardieu, H., Rochfeld, A. & Colletti, R. (2003). **La méthode Merise : Principes et outils**. Éditions d'Organisation.
  Cité p. Chapitre 4.2.1 (§Démarche Merise retenue).

- Gabay, J. (2008). **Merise et UML : Guide pratique**. Dunod.
  Cité p. Chapitre 4.2.1 (§Démarche Merise retenue).
- Matheron, J. P. (2005). **Comprendre Merise**. Eyrolles.
  Cité p. Chapitre 4.2.1 (§Démarche Merise retenue).

Persistance et bases de données

- Bauer, C., King, G. & Gregory, G. (2016). **Java Persistence with Hibernate** (2e édition). Manning Publications.
  Cité p. Chapitre 2.4.4 (§Choix du SGBD), p. Chapitre 5.2 (§Technologies backend), p. Chapitre 5.4 (§Base de données).
- Carlson, J. L. (2013). **Redis in Action**. Manning Publications.
  Cité p. Chapitre 5.2 (§Technologies backend — cache Redis).

Ergonomie et expérience utilisateur

- Nogier, J.-F. & Bouillot, J. (2020). **Ergonomie du logiciel et design web** (6e édition). Dunod.
  Cité p. Chapitre 4.4 (§Maquettes et interfaces).
- Bastien, J. C. & Scapin, D. L. (1993). **Ergonomical criteria for the evaluation of user interfaces**. Rapport technique INRIA.
  Cité p. Chapitre 3.3 (§Besoins non fonctionnels — ergonomie).

Qualité logicielle et tests

- Meszaros, G. (2007). **xUnit Test Patterns**. Addison-Wesley Professional.
  Cité p. Chapitre 7.1 (§Stratégie de tests).

Urbanisation des SI et transformation digitale

- Longépé, C. (2009). **Le projet d'urbanisation du S.I.**. Dunod.
  Cité p. 6 (Introduction).

DevOps et déploiement continu

- Humble, J. & Farley, D. (2010). **Continuous Delivery**. Addison-Wesley.
  Cité p. Chapitre 6.6 (§Exécution locale et chaîne DevOps).

Sécurisation des examens en ligne

- Nigam, A., Pasricha, R., Singh, T. & Joseph, P. (2021). A systematic review of research on cheating in online exams. **Journal of Computing in Higher Education**.
  Cité p. Chapitre 6.4.5 (§Sécurisation du quiz et intégrité des évaluations).

Traçabilité et audit des systèmes

- Johnston, R. (2019). Designing an Enterprise Application Audit Trail. **Journal of Systems Architecture**.
  Cité p. Chapitre 6.4.8 (§Traçabilité, audit et historisation).

Contrôle d'accès avancé

- Hu, V. C. et al. (2014). **Guide to Attribute Based Access Control (ABAC)**. NIST SP 800-162.
  Cité p. Chapitre 6.4.7 (§Notifications, permissions et pilotage — permissions ABAC).

Documentations techniques et ressources en ligne

- Broadcom / Spring Team. (2025). **Spring Boot Reference Documentation (Version 3.5.x)**. Consulté le 12 mars 2026.
  Cité p. Chapitre 5.2 (§Technologies backend).
- Google. (2025). **Angular Documentation: Concepts and Architecture (v20)**. Consulté le 18 avril 2026.
  Cité p. Chapitre 5.3 (§Technologies frontend).
- Google. (2025). **Angular Signals: Fine-Grained Reactivity Model**. Consulté le 20 avril 2026.
  Cité p. Chapitre 5.3 (§Technologies frontend — Signals).

- Red Gate Software. (2025). **Flyway Documentation**. Consulté le 3 mars 2026.
  Cité p. Chapitre 5.2 (§Technologies backend), p. Chapitre 5.4 (§Base de données).
- MinIO Inc. (2025). **MinIO Object Storage Reference Guide**. Consulté le 15 février 2026.
  Cité p. Chapitre 5.2 (§Technologies backend), p. Chapitre 6.6 (§Exécution locale).
- OWASP Foundation. (2025). **OWASP Top 10**. Consulté le 8 mars 2026.
  Cité p. Chapitre 3.3 (§Besoins non fonctionnels — sécurité).
- KodMatrix. (2025). Angular 20: Everything You Need to Know. **DEV Community**. Consulté le 22 avril 2026.
  Cité p. Chapitre 5.3 (§Technologies frontend).
- Object Management Group. (2013). **BPMN 2.0**. Consulté le 5 mars 2026.
  Cité p. Chapitre 6.4.2 (§Moteur de pipeline et progression).

Démarche de recherche documentaire

La bibliographie de ce mémoire a été constituée selon une démarche thématique progressive. Les références ont d'abord été identifiées à partir des besoins propres au projet SyneliaQuizApp, puis regroupées par domaine : processus métier du recrutement, architecture logicielle, sécurité, modélisation Merise, ergonomie, persistance, tests, urbanisation des SI et DevOps. Chaque référence a été sélectionnée pour son aptitude à éclairer un choix technique, une méthode de conception ou une règle de gestion documentée dans le corps du mémoire. Les ressources en ligne ont été consultées entre février et mai 2026 ; les dates exactes de consultation sont indiquées pour chaque référence web.

Annexes

Annexe A — Captures d'écran complémentaires

Les figures suivantes présentent des écrans additionnels de la plateforme qui n'apparaissent pas dans le corps du mémoire. Elles illustrent la richesse fonctionnelle de l'application à travers les différents modules du tableau de bord recruteur et administrateur.

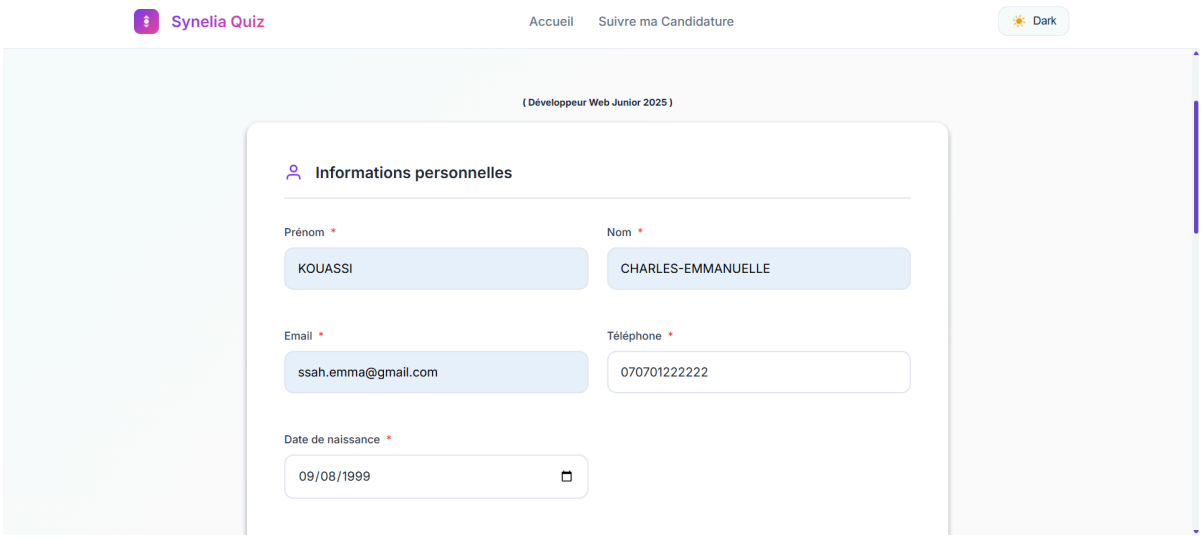


Fig. 38. – Page publique de dépôt de candidature — formulaire de soumission pour une campagne.

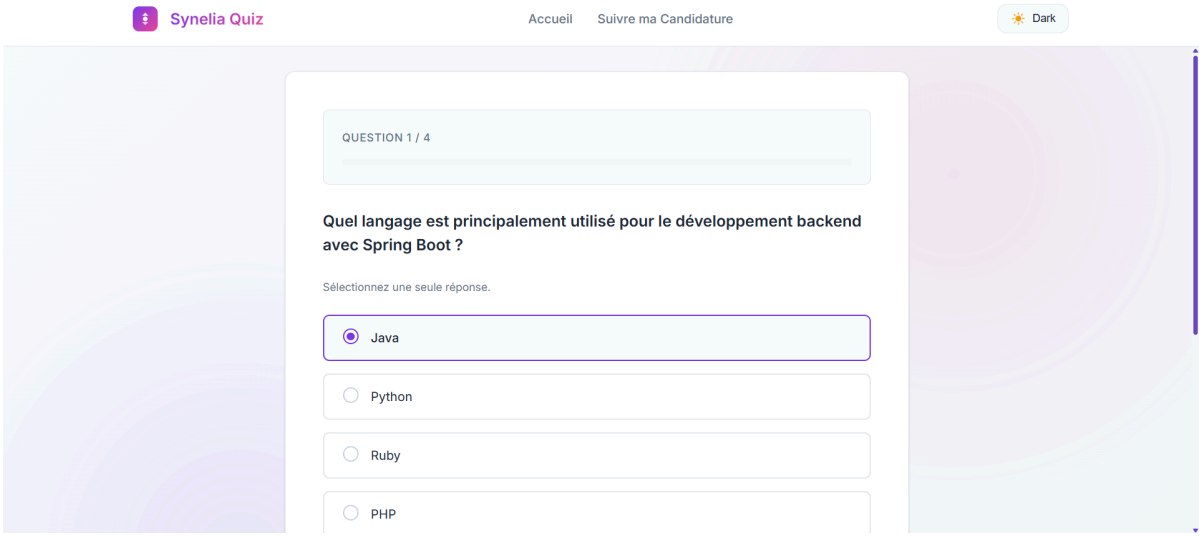


Fig. 39. – Interface de passation du quiz technique — question à choix multiples avec chronomètre.

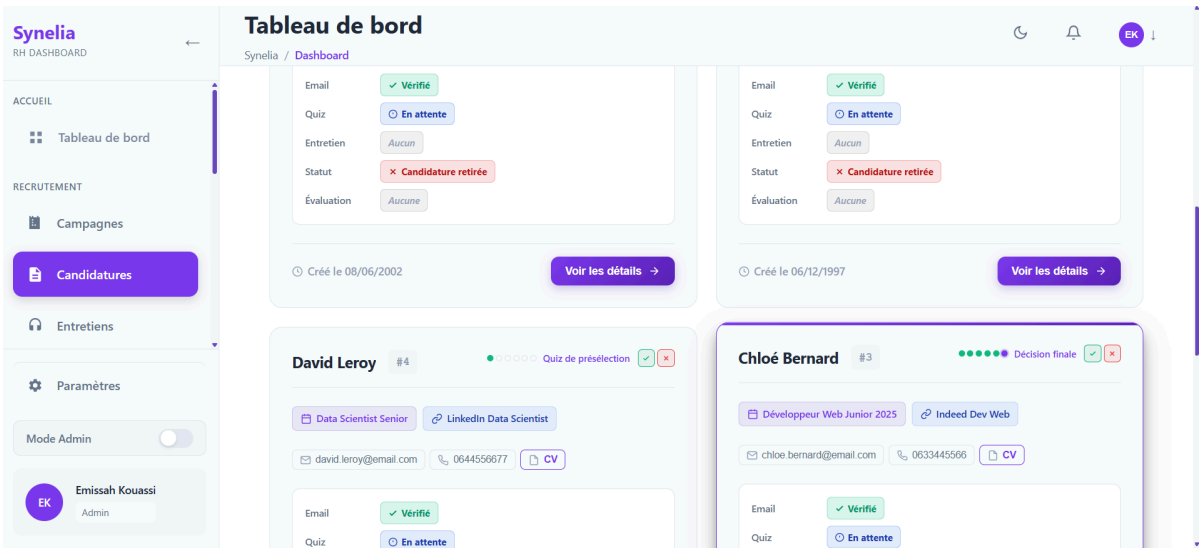


Fig. 40. – Liste des candidatures — filtrage, recherche et actions contextualisées par dossier.

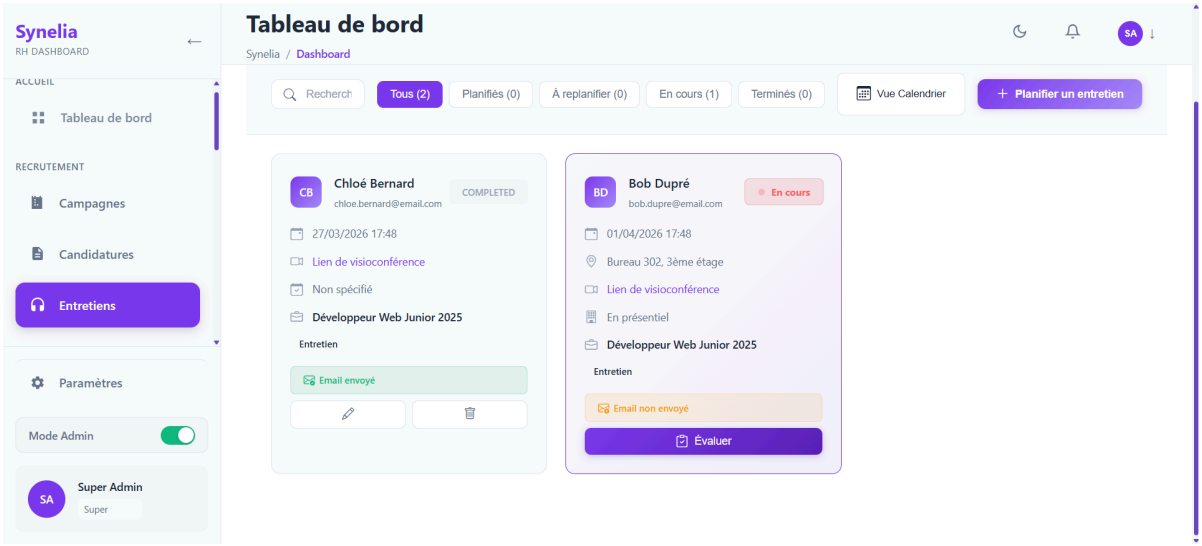


Fig. 41. – Gestion des entretiens — planification, états (terminé, en cours) et actions associées.

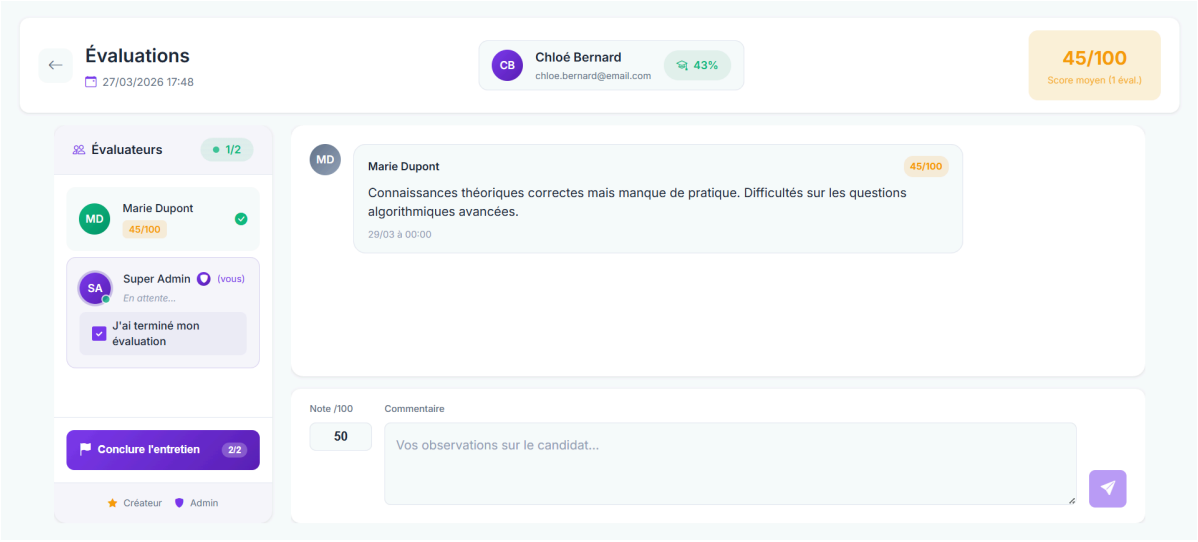


Fig. 42. – Écran d'évaluation collaboratif — critères détaillés, notation et commentaires par évaluateur.

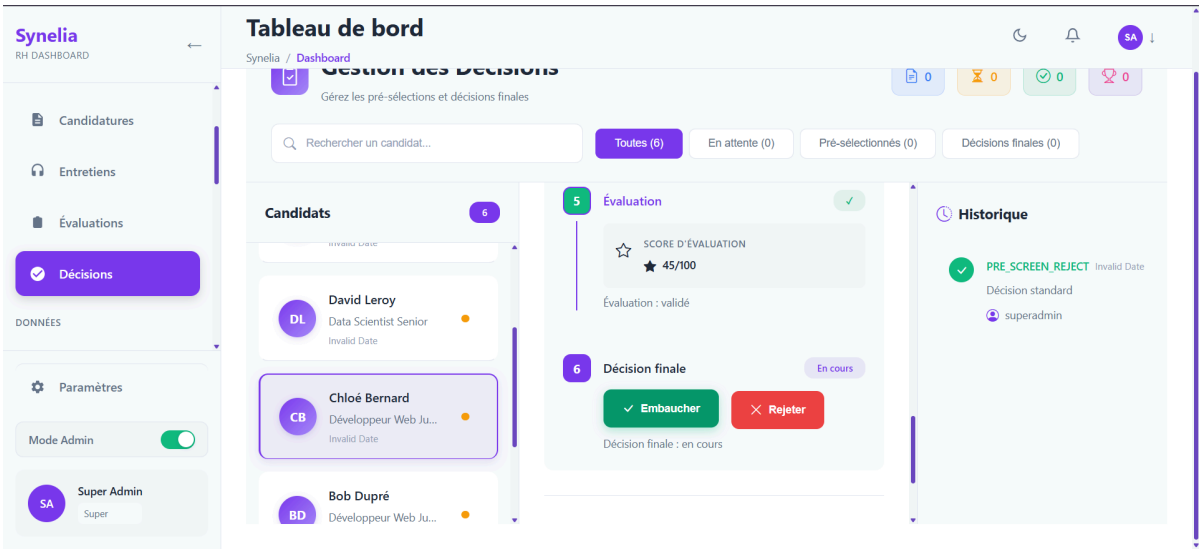


Fig. 43. – Module de décisions — présélection, décision finale, annulation et historique.

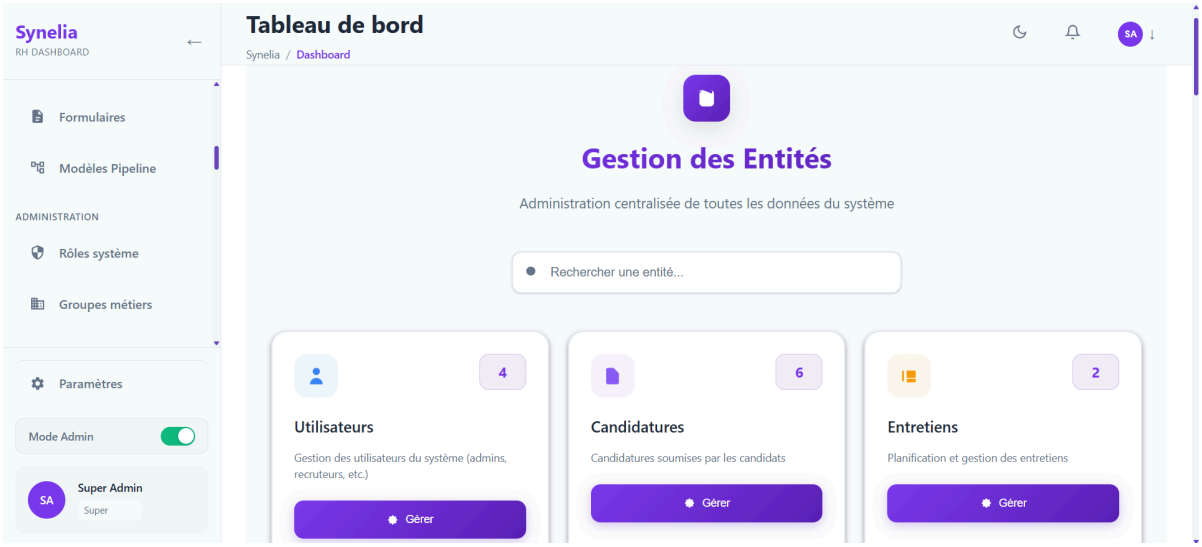


Fig. 44. – Vue d'administration des entités — gestion des données de référence (départements, spécialités, etc.).

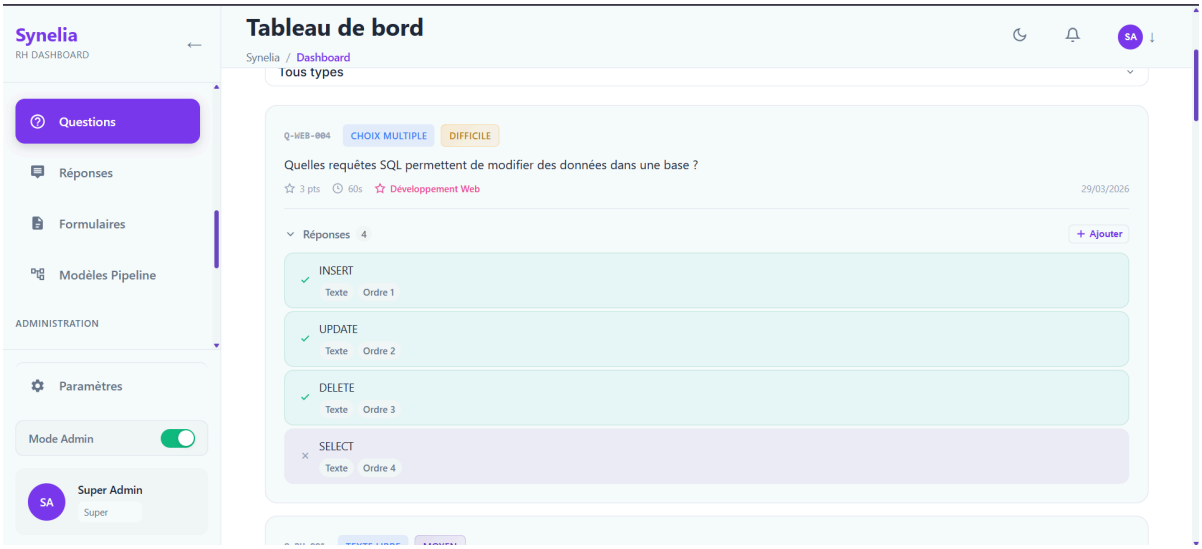


Fig. 45. – Banque de questions — création, types de questions, réponses et niveau de difficulté.

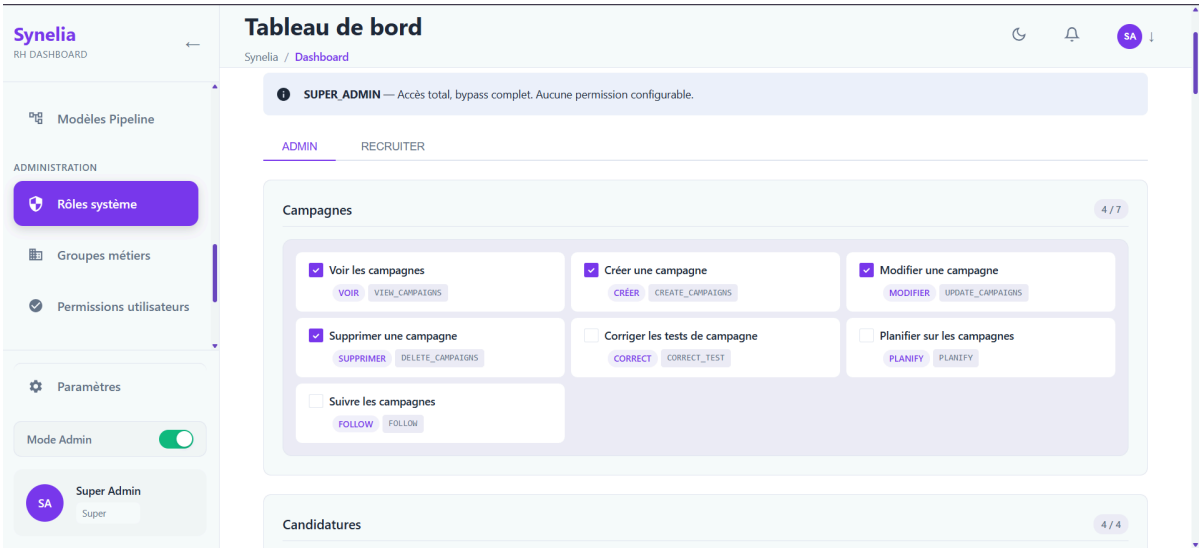


Fig. 46. – Gestion des permissions système — rôles, droits et overrides utilisateur.

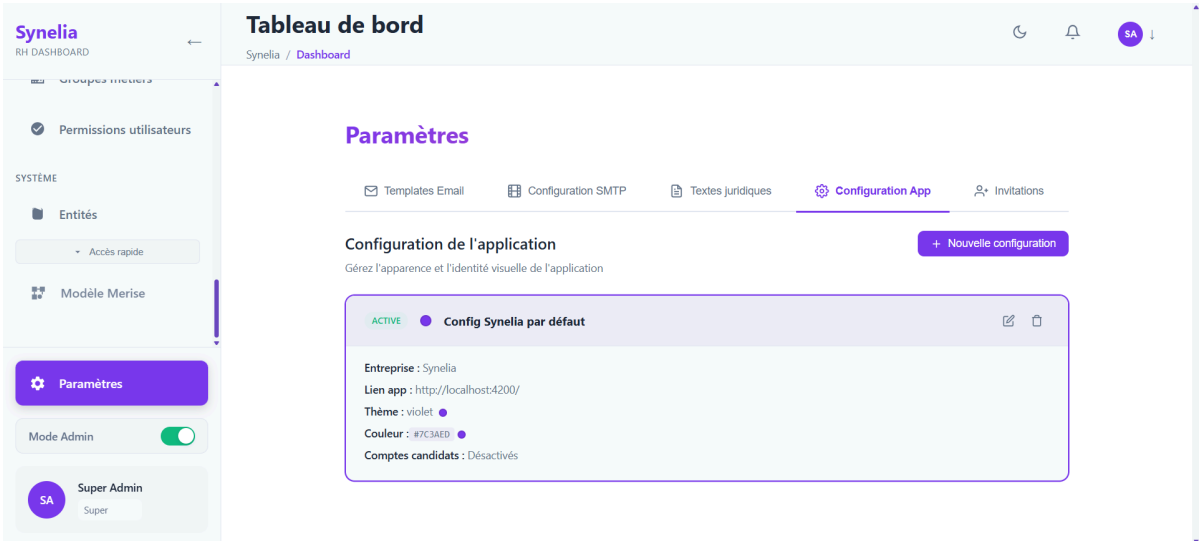


Fig. 47. – Paramétrage de l'application — configuration SMTP, modèles d'email et constantes globales.

Annexe B — Diagrammes de séquence complémentaires

Les trois diagrammes de séquence suivants, mentionnés dans le corps du mémoire (Partie 2, § Compléments éventuels en UML), sont présentés ici en détail. Ils complètent le diagramme de soumission d'une candidature publique en documentant les interactions dynamiques entre le frontend Angular, l'API REST Spring Boot, les services métier, la couche de persistance JPA/MySQL et les mécanismes de notification.

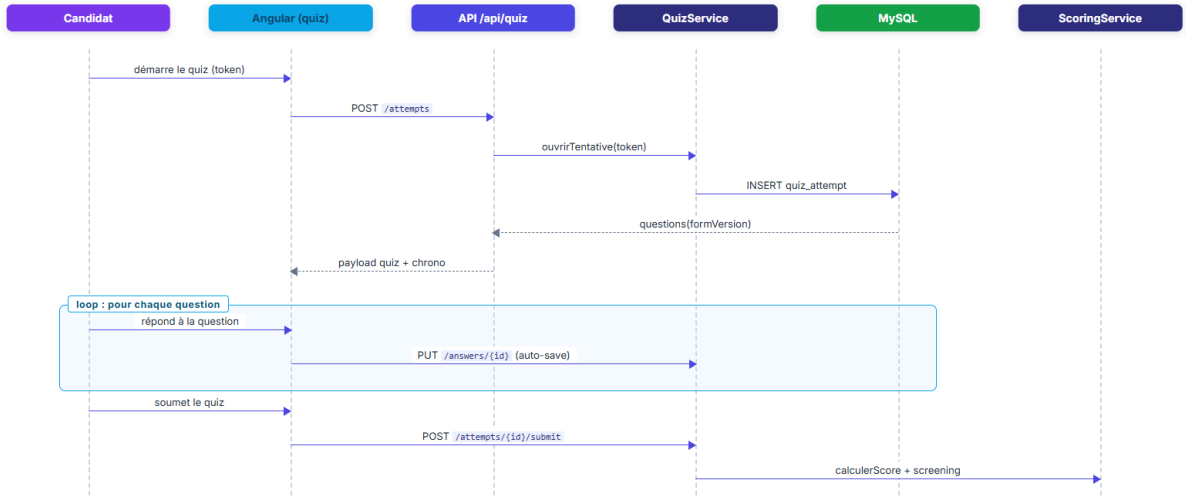


Fig. 48. – Séquence — passation et soumission du quiz technique : chargement des questions, enregistrement progressif des réponses et clôture du quiz.

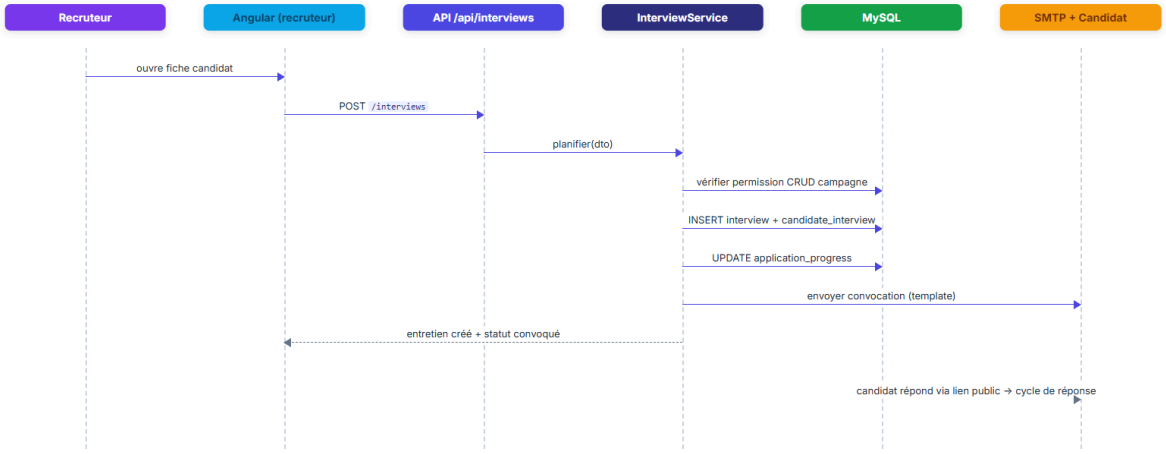


Fig. 49. – Séquence — planification et convocation d'un entretien : rattachement à la candidature, production de la convocation et mise à jour du pipeline.

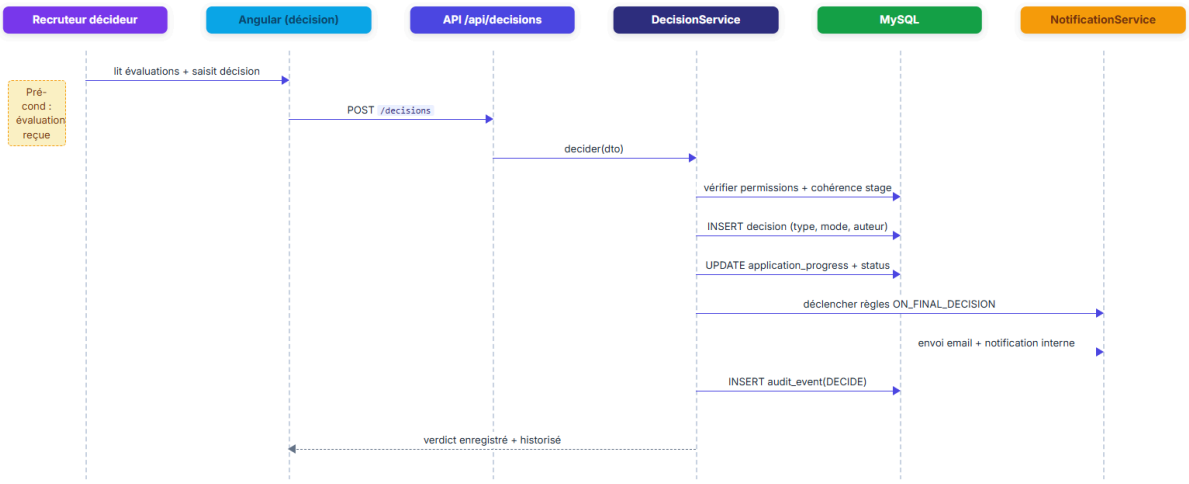


Fig. 50. – Séquence — saisie et propagation d'une décision finale : agrégation des évaluations, décision et synchronisation des états du dossier.

Annexe C — Extraits de code sources

Cette annexe présente quelques extraits représentatifs du code développé dans le cadre du projet. Ils illustrent les choix d'architecture et les motifs de conception récurrents dans l'implémentation de SyneliaQuizApp.

Extrait C.1 — Filtre d'authentification JWT (Spring Security)

Le filtre ci-dessous intercepte chaque requête HTTP entrante, extrait le jeton JWT de l'en-tête `Authorization`, valide sa signature et sa date d'expiration, puis peuple le contexte de sécurité Spring avec l'identité et les autorités de l'utilisateur. La hiérarchie implicite des rôles — SUPER héritant de ADMIN et RECRUITER, ADMIN héritant de RECRUITER — est résolue directement dans le filtre.

```
@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain)
        throws ServletException, IOException {
    String authHeader = request.getHeader("Authorization");

    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        String token = authHeader.substring(7);
        try {
            if (jwtService.isTokenValid(token)) {
                Integer userId = jwtService.extractUserId(token);
                String systemRole = jwtService.extractSystemRole(token);
                UserRole normalizedRole = UserRole.fromValue(systemRole);

                List<SimpleGrantedAuthority> authorities = new ArrayList<>();
                Set<String> authorityNames = new LinkedHashSet<>();
                if (normalizedRole != null) {
                    authorityNames.add("ROLE_" + normalizedRole.getSystemRole());
                    if (UserRole.SUPER.equals(normalizedRole)) {
                        authorityNames.add("ROLE_ADMIN");
                        authorityNames.add("ROLE_RECRUITER");
                    } else if (UserRole.ADMIN.equals(normalizedRole)) {
                        authorityNames.add("ROLE_RECRUITER");
                    }
                }
                for (String authorityName : authorityNames) {
                    authorities.add(new SimpleGrantedAuthority(authorityName));
                }

                var authToken = new UsernamePasswordAuthenticationToken(
                    userId, null, authorities);
                SecurityContextHolder.getContext()
                    .setAuthentication(authToken);
```

```
            }
        } catch (Exception e) {
            logger.debug("JWT validation failed: {}", e.getMessage());
        }
    }
    filterChain.doFilter(request, response);
}
```

Extrait C.2 — Création d'une candidature (couche métier Spring)

La méthode `create` de la classe `ApplicationsBusiness` orchestre l'ensemble des opérations nécessaires à la soumission d'une candidature : validation des champs obligatoires, vérification de l'existence du jeton de source et de la campagne associée, contrôle des limites de candidatures par campagne, instanciation de l'entité, initialisation de la progression dans le pipeline, génération du code de suivi, création de la tentative de quiz et envoi de l'email de vérification. L'annotation `@Transactional` garantit l'atomicité de l'ensemble.

```
@Transactional
public Response<ApplicationsDto> create(
        Request<ApplicationsDto> request, Locale locale)
        throws ParseException, JsonProcessingException {

    platformReadinessService.requireEmailDeliveryAvailable(
            EmailActionType.APPLICATION_VERIFICATION, locale);

    for (ApplicationsDto dto : request.getDatas()) {
        // Validation des champs obligatoires
        Map<String, Object> fieldsToVerify = new HashMap<>();
        fieldsToVerify.put("candidateFirstName", dto.getCandidateFirstName());
        fieldsToVerify.put("candidateLastName", dto.getCandidateLastName());
        fieldsToVerify.put("candidateEmail", dto.getCandidateEmail());
        fieldsToVerify.put("candidatePhone", dto.getCandidatePhone());

        if (!Validate.RequiredValue(fieldsToVerify).isGood()) {
            throw new InvalidEntityException(
                functionalError.FIELD_EMPTY(
                    Validate.getValidate().getField(), locale));
        }

        // Vérification du jeton de source et de la campagne
        SourceToken existingSourceToken = sourceTokenRepository
            .findOne(dto.getSourceTokenId(), false);
        if (existingSourceToken == null) {
            throw new EntityNotFoundException(
                functionalError.DATA_NOT_EXIST(
                    "sourceToken", locale));
        }
```

```
        Campaigns campaign = existingSourceToken.getCampaigns();
        // ... vérification des limites, instanciation, pipeline, quiz ...
    }

    return response;
}
```

Extrait C.3 — DashboardController : endpoint d'agrégation des statistiques

Le contrôleur ci-dessous illustre le pattern d'agrégation utilisé pour le tableau de bord. Un endpoint unique `GET /api/dashboard/overview-stats` consolide les données de plusieurs repositories — applications, entretiens, évaluations, décisions — et produit un objet JSON structuré contenant les indicateurs clés (KPIs avec deltas mensuels), les distributions par statut, les tendances sur 30 jours, l'entonnoir de conversion, le top 5 des campagnes et la distribution des scores de quiz.

```
@GetMapping("/overview-stats")
public Map<String, Object> getOverviewStats() {
    Map<String, Object> result = new LinkedHashMap<>();

    List<Applications> apps = applicationsRepository.findAll().stream()
            .filter(a -> !Boolean.TRUE.equals(a.getIsDeleted()))
            .toList();

    // KPIs avec delta mois en cours vs mois précédent
    Map<String, Object> kpis = new LinkedHashMap<>();
    kpis.put("applications", buildKpi(apps, Applications::getCreatedAt,
            startOfPrevMonth, startOfMonth));
    // ... interviews, evaluations, decisions ...
    result.put("kpis", kpis);

    // Distribution par statut, tendance 30 jours, entonnoir, top campagnes...
    result.put("statusDistribution",
        toEntryList(apps.stream()
            .collect(Collectors.groupingBy(Applications::getApplicationStatus,
                    LinkedHashMap::new, Collectors.counting()))));

    result.put("applicationsTrend", build30DayTrend(apps));
    result.put("conversionFunnel", buildFunnel(apps));
    result.put("topCampaigns", buildTopCampaigns(apps, 5));

    return result;
}
```

Table des matières

Introduction 1
1. Présentation générale du projet 5
    1.1. Contexte du projet 5
        1.1.1. L'entreprise SYNELIA et ses besoins 5
        1.1.2. Le processus de recrutement traditionnel 8
        1.1.3. Les défis et limitations actuels 8
        1.1.4. La nécessité d'une solution digitale 9
    1.2. Objectifs du projet 10
        1.2.1. Objectifs généraux 10
        1.2.2. Objectifs spécifiques 10
    1.3. Périmètre fonctionnel 11
        1.3.1. Modules couverts par l'application 11
        1.3.2. Acteurs du système 11
        1.3.3. Frontières du projet 11
    1.4. Contraintes du projet 12
2. Étude de l'existant et état de l'art 13
    2.1. Étude de l'existant 13
    2.2. Benchmark des solutions existantes 13
    2.3. État de l'art technologique 14
    2.4. Justification des choix technologiques 15
        2.4.1. Critères de sélection retenus 15
        2.4.2. Choix du framework backend 16
        2.4.3. Choix du framework frontend 17
        2.4.4. Choix du système de gestion de base de données 18
        2.4.5. Synthèse comparative 19
    2.5. Critique de l'existant et solution proposée 20
3. Analyse des besoins 23
    3.1. Méthodologie d'analyse 23
        3.1.1. Parties prenantes et périmètre étudié 23
        3.1.2. Démarche de formalisation 24
    3.2. Besoins fonctionnels 24
        3.2.1. Authentification et gestion des utilisateurs 25
        3.2.2. Gestion des données de référence 25

3.2.3. Campagnes de recrutement 25
3.2.4. Candidatures et suivi candidat 26
3.2.5. Quiz technique 26
3.2.6. Entretiens, évaluations et décisions 26
3.2.7. Notifications et dashboard 27
3.2.8. Matrice synthétique des besoins fonctionnels 27
3.2.9. Matrice acteurs et responsabilités 28
3.3. Besoins non fonctionnels 30
3.4. Diagrammes des cas d'utilisation 31
3.4.1. Diagramme global 32
3.4.2. Description textuelle des cas d'utilisation principaux 33
3.4.3. Lecture par acteur 34
3.5. Priorisation des besoins 34
3.6. Règles de gestion 35
4. Conception de la solution 38
4.1. Architecture globale du système 38
4.1.1. Vue logique en couches 40
4.2. Conception de la base de données 40
4.2.1. Démarche Merise retenue 41
4.2.2. Modèle Conceptuel de Données (MCD) 42
4.2.3. Modèle Logique de Données (MLD) 46
4.2.4. Modèle Physique de Données (MPD) 50
4.2.5. Dictionnaire de données du noyau métier 54
4.2.6. Correspondance entre objets Merise et tables physiques 56
4.2.7. Dépendances fonctionnelles 57
4.2.8. Structure d'accès théorique (SAT) 57
4.2.9. Modèle Conceptuel des Traitements (MCT) 59
4.2.10. Modèle Physique des Traitements (MPT) 63
4.2.11. Modèle Conceptuel de Communication (MCC) 65
4.3. Compléments éventuels en UML 67
4.3.1. Diagrammes de classes 67
4.3.2. Diagrammes de séquence 69
4.3.3. Diagrammes d'activités et d'états 70
4.4. Maquettes et interfaces 71
4.5. Conception de l'API REST 72
5. Environnement technique et outils 75

5.1. Environnement de développement 75
5.2. Technologies backend 76
5.3. Technologies frontend 77
5.4. Base de données 77
5.5. Outils complémentaires 78
6. Implémentation de la solution 80
6.1. Organisation du code source 80
6.2. Implémentation du backend 80
6.3. Implémentation du frontend 81
6.4. Implémentation des modules fonctionnels 82
6.4.1. Gestion des campagnes et des formulaires 82
6.4.2. Moteur de pipeline et progression des candidatures 82
6.4.3. Gestion des candidatures et parcours public 83
6.4.4. Quiz technique et correction 84
6.4.5. Sécurisation du quiz et intégrité des évaluations 84
6.4.6. Entretiens, évaluations et décisions 85
6.4.7. Notifications, permissions et pilotage 85
6.4.8. Traçabilité, audit et historisation 86
6.5. Intégration et communication 86
6.5.1. Communication temps réel et tableaux de bord synoptiques 87
6.6. Exécution locale et chaîne DevOps 88
6.7. Démarche qualité et boucle de validation continue 89
7. Tests et validation 91
7.1. Stratégie de tests 91
7.2. Tests unitaires 92
7.3. Tests d'intégration 92
7.4. Tests fonctionnels 93
7.5. Tests de performance 93
7.6. Tests de sécurité 94
7.7. Recettage et validation finale 94
8. Présentation des résultats 97
8.1. Démonstration de l'application 97
8.2. Fonctionnalités implémentées 99
8.3. Bilan technique 100
8.4. Retour d'expérience 100
9. Limites et perspectives 102

9.1. Limites du travail réalisé 102
9.2. Perspectives d'amélioration 102
9.3. Recommandations 103
Conclusion 104
Références bibliographiques 106
Annexes 109
Annexe A — Captures d'écran complémentaires 109
Annexe B — Diagrammes de séquence complémentaires 113
Annexe C — Extraits de code sources 115